

Unlocking the lookup singularity with Lasso

Srinath Setty*

Justin Thaler†

Riad Wahby‡

Abstract

This paper introduces **Lasso**, a new family of lookup arguments, which allow an untrusted prover to commit to a vector $a \in \mathbb{F}^m$ and prove that all entries of a reside in some predetermined table $t \in \mathbb{F}^n$. **Lasso**’s performance characteristics unlock the so-called “lookup singularity”. **Lasso** works with any multilinear polynomial commitment scheme, and provides the following efficiency properties.

- For m lookups into a table of size n , **Lasso**’s prover commits to just $m + n$ field elements. Moreover, the committed field elements are *small*, meaning that, no matter how big the field $\mathbb{F}$ is, they are all in the set $\{0, \dots, m\}$. When using a multiexponentiation-based commitment scheme, this results in the prover’s costs dominated by only $O(m + n)$ group *operations* (e.g., elliptic curve point additions), plus the cost to prove an evaluation of a multilinear polynomial whose evaluations over the Boolean hypercube are the table entries. This represents a significant improvement in prover costs over prior lookup arguments (e.g., plookup, Halo2’s lookups, lookup arguments based on logarithmic derivatives).
- Unlike all prior lookup arguments, if the table t is structured (in a precise sense that we define), then no party needs to commit to t , enabling the use of much larger tables than prior works (e.g., of size 2^{128} or larger). Moreover, **Lasso**’s prover only “pays” in runtime for table entries that are accessed by the lookup operations. This applies to tables commonly used to implement range checks, bitwise operations, big-number arithmetic, and even transitions of a full-fledged CPU such as RISC-V. Specifically, for any integer parameter $c > 1$, **Lasso**’s prover’s dominant cost is committing to $3 \cdot c \cdot m + c \cdot n^{1/c}$ field elements. Furthermore, all these field elements are “small”, meaning they are in the set $\{0, \dots, \max\{m, n^{1/c}, q\} - 1\}$, where q is the maximum value in a .

Lasso’s starting point is **Spark**, a time-optimal polynomial commitment scheme for sparse polynomials in Spartan (CRYPTO 2020). We first provide a stronger security analysis for **Spark**. Spartan’s security analysis assumed that certain metadata associated with a sparse polynomial is committed by an honest party (this is acceptable for its purpose in Spartan, but not for **Lasso**). We prove that **Spark** remains secure even when that metadata is committed by a malicious party. This provides the first “standard” commitment scheme for sparse multilinear polynomials with optimal prover costs. We then generalize **Spark** to directly support a lookup argument for both structured and unstructured tables, with the efficiency characteristics noted above.

*Microsoft Research

†a16 crypto research and Georgetown University

‡Carnegie Mellon University

1 Introduction

Suppose that an untrusted prover $\mathcal{P}$ claims to know a witness w satisfying some property. For example, w might be a pre-image of a designated value y of a cryptographic hash function h , i.e., a w such that $h(w) = y$. A trivial proof is for $\mathcal{P}$ to send w to the verifier $\mathcal{V}$, who checks that w satisfies the claimed property.

A zero-knowledge succinct non-interactive argument of knowledge (zkSNARK) achieves the same, but with better verification costs (and proof sizes) and privacy properties. Succinct means that verifying a proof is much faster than checking the witness directly (this also implies that proofs are much smaller than the size of the statement proven). Zero-knowledge means that the verifier does not learn anything about the witness beyond the validity of the statement proven.

Fast algorithms via lookup tables. A common technique in the design of fast algorithms is to use *lookup tables*. These are pre-computed tables of values that, once computed, enable certain operations to be computed quickly. For example, in *tabulation-based universal hashing* [PT12, PT13], the hashing algorithm is specified via some small number c of tables $T_1, \dots, T_c$, each of size $n^{1/c}$. Each cell of each table is filled with a random q -bit number in a preprocessing step. To hash a key x of length n , the key is split into c “chunks” $x_1, \dots, x_c \in \{0, 1\}^{n/c}$, and the hash value is defined to be the bitwise XOR of c *table lookups* i.e., $\oplus_{i=1}^c T_i[x_i]$.

Lookup tables are also useful in the context of SNARKs. Recall that to apply SNARKs to prove the correct execution of computer programs, one must express the execution of the program in a specific form that is amenable to probabilistic checking (e.g., as arithmetic circuits or generalizations thereof). Lookup tables can facilitate the use of substantially smaller circuits.

For example, imagine that a prover wishes to establish that at no point in a program’s execution did any integer ever exceed 2^{128} , say, because were that to happen then an uncorrected “overflow error” would occur. A naive approach to accomplish this inside a circuit-satisfiability instance is to have the circuit take as part of its “non-deterministic advice inputs” 128 field elements for each number x arising during the execution. If the prover is honest, these 128 advice elements will be set to the binary representation of x . The circuit must check that all of the 128 advice elements are in $\{0, 1\}$ and that they indeed equal the binary representation of x , i.e., $x = \sum_{i=0}^{127} 2^i \cdot b_i$, where $b_0, \dots, b_{127}$ denotes the advice elements. This is very expensive: a simple overflow check turns into at least 129 constraints and an additional 128 field elements in the prover’s witness that must be cryptographically committed by the prover.¹

Lookup tables offer a better approach. Imagine for a moment that the prover and the verifier initialize a lookup table containing all integers between 0 and $2^{128} - 1$. Then the overflow check above amounts to simply confirming that x is in the table, i.e., the overflow check *is* a single table lookup. Of course, a table of size 2^{128} is far too large to be explicitly represented—even by the prover. This paper describes techniques to enable such a table lookup without requiring a table such as this to ever be explicitly materialized, by either the prover or the verifier.

Table lookups are now used pervasively in deployed applications that employ SNARKs. They are very useful for representing “non-arithmetic” operations efficiently inside circuits [BCG⁺18, GW20b, GW20a]. The above example is often called a *range check* for the range $\{0, 1, \dots, 2^{128} - 1\}$. Other example operations for which lookups are useful include bitwise operations such as XOR and AND [BCG⁺18], and any operations that require big-number arithmetic.

Lookup arguments. To formalize the above discussion regarding the utility of lookup tables in SNARKs, a (non-interactive) *lookup argument* is a SNARK for the following claim made by the prover.

Definition 1.1 (Statement proven in a lookup argument). *Given a commitment cm_a and a public set T of N field elements, represented as vector $t = (t_0, \dots, t_{N-1}) \in \mathbb{F}^N$ to which the verifier has (possibly) been provided*

¹As we explain later (Remark 1.2), for certain commitment schemes, the prover’s cost to commit to vectors consisting of many $\{0, 1\}$ values can be much cheaper than if the vectors contain arbitrary field elements. However, other SNARK prover costs (e.g., number of field operations) will grow linearly with the number of advice elements and constraints in the circuit to which the SNARK is applied, irrespective of whether the advice elements are $\{0, 1\}$ -valued.

a commitment cm_t , the prover knows an opening $a = (a_0, \dots, a_{m-1}) \in \mathbb{F}^m$ of cm_a such that all elements of a are in T . That is, for each $i = 0, \dots, m-1$, there is a $j \in \{0, \dots, N-1\}$ such that $a_i = t_j$.

The set T in Definition 1.1 is the contents of a lookup table and the vector a is the sequence of “lookups” into the table. The prover in the lookup argument proves to the verifier that every element of a is in T .

A recent flurry of works (Caulk [ZBK⁺22], Caulk+ [PK22], flookup [GK22], Baloo [ZGK⁺22], and cq [EFG22]) have sought to give lookup arguments in which the prover’s runtime is sublinear in the table size N . This is important in applications where the lookup table itself is much larger than the number of lookups into that table. As a simple and concrete example, if the verifier wishes to confirm that $a_0, \dots, a_{m-1}$ are all in a large range (say, in $\{0, 1, \dots, 2^{32} - 1\}$), then performing a number of cryptographic operations linear in N will be slow or possibly untenable. For performance reasons, these papers also express a desire for the commitment scheme used to commit to a and t to be additively homomorphic. However, these prior works all require generating a structured reference string of size N as well as an additional pre-processing work of $O(N \log N)$ group exponentiations. This limits the size of the tables to which they can be applied. For example, the largest structured reference strings generated today are many gigabytes in size and still only support $N < 2^{30}$.²

Indexed lookup arguments. Definition 1.1 is a standard formulation of lookup arguments in SNARKs (e.g., see [ZGK⁺22]). It treats the table as an unordered list of values— T is a *set* and, accordingly, reordering the vector t does not alter the validity of the prover’s claim. However, for reasons that will become apparent shortly (see Section 1.3), we consider a variant notion to be equally natural. We refer to this variant as an *indexed lookup argument* (and refer to the standard variant in Definition 1.1 as an *unindexed lookup argument*.) In an indexed lookup argument, in addition to a commitment to $a \in \mathbb{F}^m$, the verifier is also handed a commitment to a second vector $b \in \mathbb{F}^m$. The prover claims that for all $i = 1, \dots, m$, $a_i = t_{b_i}$. We refer to a as the vector of *looked-up values*, and b as the vector of *indices*.

Definition 1.2 (Statement proven in an indexed lookup argument). *Given commitment cm_a and cm_b , and a public array T of N field elements, represented as vector $t = (t_0, \dots, t_{N-1}) \in \mathbb{F}^N$ to which the verifier has (possibly) been provided a commitment cm_t , the prover knows an opening $a = (a_0, \dots, a_{m-1}) \in \mathbb{F}^m$ of cm_a and $b = (b_0, \dots, b_{m-1}) \in \mathbb{F}^m$ of cm_b such that for each $i = 0, \dots, m-1$, $a_i = T[b_j]$, where $T[b_j]$ is short hand for the b_j ’th entry of t .*

Any indexed lookup argument can easily be turned into an unindexed lookup argument: the unindexed lookup argument prover simply commits to a vector b such that $a_i = T[b_j]$ for all i , and then applies the indexed lookup argument to prove that indeed this holds. There is also a generic transformation that turns any unindexed lookup argument into an indexed one, at least in fields of large enough characteristic (see Appendix A). However, the protocols we describe in this work directly yield indexed lookup arguments, without having to invoke this transformation. Accordingly, our primary focus in this work is on indexed lookup arguments.

1.1 Lasso: A new lookup argument

We describe a new lookup argument, **Lasso**.³ Lasso’s starting point is a polynomial commitment scheme for sparse multilinear polynomials. In particular, Lasso builds on **Spark**, an optimal polynomial commitment scheme for sparse multilinear polynomials from Spartan [Set20]. Spark itself is based on the linear-time sum-check protocol [LFKN90] and offline memory checking [BEG⁺91].

Lasso can be instantiated with any multilinear polynomial commitment scheme. Furthermore, Lasso can be used with any SNARK, including those that prove R1CS or Plonkish satisfiability. This is particularly seamless for SNARKs that have the prover commit to the witness using a multilinear polynomial commitment scheme. This includes many known prover-efficient SNARKs [Set20, GLS⁺21, XZS22, CBBZ23, STW23]. If a SNARK does not natively use multilinear polynomial commitments (e.g., Marlin [CHM⁺20] and Plonk

²See, for example, <https://setup.aleo.org/stats>.

³Lasso is short for LASSO-of-Truth: Lookup Arguments via Sparse-polynomial-commitments and the Sum-check protocol, including for Oversized Tables.

[GWC19], which use univariate polynomial commitments), then one would need an auxiliary argument that the commitment cm_a used in Lasso is a commitment to the multilinear extension of the vector of all lookups performed in the SNARK.

Below, we provide an overview of Lasso’s technical components.

(1) A stronger analysis of Spark, an optimal commitment scheme for sparse polynomials. A sparse polynomial commitment allows an untrusted prover to cryptographically commit to a *sparse* multilinear polynomial g and later provide a requested evaluation $g(r)$ along with a proof that the provided value is indeed equal to the committed polynomial’s evaluation at r . Crucially, we require that the prover’s runtime depends only on the sparsity of the polynomial.⁴ Spartan [Set20] provides such a commitment scheme, which it calls **Spark**. Spartan assumed that certain metadata associated with the sparse polynomial is committed honestly, which was sufficient for its purposes. But, as we see later, Lasso requires an *untrusted* prover to commit to sparse polynomials (and the associated metadata).

A naive extension Spark to handle a maliciously committed metadata incurs concrete and asymptotic overheads, which is undesirable. Nevertheless, we prove that Spark in fact satisfies a stronger security property without any modifications (i.e., it is secure even if the metadata is committed by a potentially malicious party). This provides the first “standard” sparse polynomial commitment scheme with optimal prover costs, a result of independent interest. Furthermore, we specialize Spark for Lasso’s use to obtain concrete efficiency benefits.

(2) Surge: A generalization of Spark. We reinterpret Spark sparse polynomial commitment scheme as a technique for computing the inner product of an m -sparse committed vector of length N with a dense—but highly structured—lookup table of size N (the table is represented as a vector of size N). Specifically, in the sparse polynomial commitment scheme, the table consists of all $(\log N)$ -variate Lagrange basis polynomials evaluated at a specific point $r \in \mathbb{F}^{\log N}$. Furthermore, this table is a *tensor product* of $c \geq 2$ smaller tables, each of size $N^{1/c}$ (here, c can be set to any desired integer in $\{1, \dots, \log N\}$). We further observe that many other lookup tables can similarly be decomposed as product-like expressions of $O(c)$ tables of size $N^{1/c}$, and that Spark extends to support all such tables.

Exploiting this perspective, we describe Surge, a generalization of Spark that allows an untrusted prover to commit to any sparse vector and establish the sparse vector’s inner product with any dense, structured vector. We refer to the structure required for this to work as **Spark-only structure (SOS for short)**. We also refer to this property as *decomposability*. In more detail, an SOS table T is one that can be decomposed into $\alpha = O(c)$ “sub-tables” $\{T_1, \dots, T_\alpha\}$ of size $N^{1/c}$ satisfying the following two properties. First, any entry $T[j]$ of T can be expressed as a simple expression of a corresponding entry into each of $T_1, \dots, T_\alpha$. Second, the so-called *multilinear extension polynomial* of each T_i can be evaluated quickly (for any such table, we call T_i *MLE-structured*, where MLE stands for multilinear extension). For example, as noted above, the table T arising in Spark itself is simply the tensor product of MLE-structured sub-tables $\{T_1, \dots, T_\alpha\}$, where $\alpha = c$.

(3) Lasso: A lookup argument for SOS tables and small/unstructured tables. We observe that Surge directly provides a lookup argument for tables with SOS structure. We call the resulting lookup argument **Lasso**. Lasso has the important property that *all* field elements committed by the prover are “small”, meaning they are in the set $\{0, 1, \dots, \max\{m, N^{1/c}, q\} - 1\}$, where q is such that $\{T_1, \dots, T_\alpha\}$ all have entries in the set $\{0, 1, \dots, q - 1\}$. As elaborated upon shortly (Section 1.2), this property of Lasso has substantial implications for prover efficiency.

Lasso has new and attractive costs when applied to small and unstructured tables in addition to large SOS ones. Specifically, by setting $c = 1$, the Lasso prover commits to only about $m + N$ field elements, and all of

⁴For multilinear polynomials, m -sparse refers to polynomials $g: \mathbb{F}^\ell \rightarrow \mathbb{F}$ in ℓ variables such that $g(x) \neq 0$ for at most m values of $x \in \{0, 1\}^\ell$. In other words, g has at most m non-zero coefficients in the so-called multilinear Lagrange polynomial basis. There are $n := 2^\ell$ Lagrange basis polynomials, so if $m \ll 2^\ell$, then only a tiny fraction of the possible coefficients are non-zero. In contrast, if $m = \Theta(2^\ell)$, then we refer to g as a *dense* polynomial.

the committed elements are $\{0, 1, \dots, \max\{m, N, q\}\}$ where q is the size of the largest value in the table.⁵⁶ **Lasso** is the first lookup argument with this property, which substantially speeds up commitment computation when m , N , and q are all much smaller than the size of the field over which the commitment scheme is defined. For $c > 1$, the number of field elements that the **Lasso** prover commits to is $3cm + \alpha \cdot N^{1/c}$.

(4) GeneralizedLasso: Beyond SOS and small/unstructured tables. Finally, we describe a lookup argument that we call **GeneralizedLasso**, which applies to any MLE-structured table, not only decomposable ones.⁷ The main disadvantage of **GeneralizedLasso** relative to **Lasso** is that cm out of the $3cm + cN^{1/c}$ field elements committed by the **GeneralizedLasso** prover are random rather than small. The proofs are also somewhat larger, as **GeneralizedLasso** involves one extra invocation of the sum-check protocol compared to **Lasso**.

GeneralizedLasso is reminiscent of a sum-check based SNARK (e.g., Spartan [Set20]) and is similarly built from a combination of the sum-check protocol and the **Spark** sparse polynomial commitment scheme. There are two key differences: (1) in **GeneralizedLasso**, the (potentially adversarial) prover commits to a sparse polynomial, rather than an honest “setup algorithm” committing to a sparse polynomial in a preprocessing step in the context of Spartan (where the sparse polynomial encodes the circuit or constraint system of interest); and (2) invoking the standard linear-time sum-check protocol [LFKN90, CTY11, Tha13] makes the prover incur costs linear in the *table size* rather than the number of lookups. To address (1), we invoke our stronger security analysis of **Spark**. To address (2), we introduce a new variant of the sum-check protocol tailored for our setting, which we refer to as the *sparse-dense* sum-check protocol. Conceptually, **GeneralizedLasso** can be viewed as using the sparse-dense sum-check protocol to reduce lookups into any MLE-structured table into lookups into a decomposable table (namely, a certain lookup table arising within the **Spark** polynomial commitment scheme).

Additional discussion of the benefits and costs of **GeneralizedLasso** relative to **Lasso** can be found in Section 1.4.

1.2 Additional discussion of Lasso’s costs

Polynomial commitments and MSMs. As indicated above, a central component of most SNARKs is a cryptographic protocol called a *polynomial commitment scheme*. Such a scheme allows an untrusted prover to succinctly commit to a polynomial p and later reveal an evaluation $p(r)$ for a point r chosen by the verifier (the prover will also return a *proof* that the claimed evaluation is indeed equal to the committed polynomial’s evaluation at r). In **Lasso**, the bottleneck for the prover is the polynomial commitment scheme.

Many popular polynomial commitments are based on multiexponentiations (also known as multi-scalar multiplications, or MSMs). This means that the commitment to a polynomial p (with n coefficients $c_0, \dots, c_{n-1}$ over an appropriate basis) is

$$\prod_{i=0}^{n-1} g_i^{c_i},$$

for some public generators $g_1, \dots, g_n$ of a multiplicative group $\mathbb{G}$. Examples include KZG [KZG10], Bulletproofs/IPA [BCC⁺16, BBB⁺18], Hyrax [WTS⁺18], and Dory [Lee21].⁸

The naive MSM algorithm performs n group exponentiations and n group multiplications (note that each group exponentiation is about $400\times$ slower than a group multiplication). But Pippenger’s MSM algorithm saves a factor of about $\log(n)$ relative to the naive algorithm. This factor can be well over $10\times$ in practice.

⁵**Lasso** makes blackbox use of any so-called grand product argument. If using the grand product argument from [SL20, Section 6], a low-order number, say at most $O(m/\log^3 m)$, of large field elements need to be committed (see Section E for discussion).

⁶If **Lasso** is used as an indexed lookup argument, the prover commits to $m + N$ field elements. If used as an unindexed lookup argument, the number can increase to $2m + N$ because in the unindexed setting one must “charge” for the prover to commit to the index vector $b \in \mathbb{F}^m$.

⁷In fact, **GeneralizedLasso** applies to any table with *some* low-degree extension, not necessarily its multilinear one, that is evaluable in logarithmic time.

⁸In Hyrax and Dory, the prover does $\sqrt{n}$ MSMs each of size $\sqrt{n}$.

Working over large fields, but committing to small elements. If all exponents appearing in the multiexponentiation are “small”, one can save another factor of $10\times$ relative to applying Pippenger’s algorithm to an MSM involving random exponents. This is analogous to how computing $g_i^{2^{16}}$ is $10\times$ faster than computing $g_i^{2^{160}}$: the first requires 16 squaring operations, while the second requires 160 such operations.

In other words, if one is promised that all field elements (i.e., exponents) to be committed via an MSM are in the set $\{0, 1, \dots, K\} \subset \mathbb{F}$, the number of group operations required to compute the MSM depend only on K and not on the size of $\mathbb{F}$ ⁹

Quantitatively, if all exponents are upper bounded by some value K , with $K \ll n$, then Pippenger’s algorithm only needs (about) one group *operation* per term in the multiexponentiation.¹⁰ More generally, with any MSM-based commitment scheme, Pippenger’s algorithm allows the prover to commit to roughly $k \cdot \log(n)$ -bit field elements (meaning field elements in $\{0, 1, \dots, n\}$) with only k group operations per committed field element.

Polynomial evaluation proofs. In any SNARK or lookup argument, the prover not only has to commit to one or more polynomials, but also reveal to the verifier an evaluation of the committed polynomials at a point of the verifier’s choosing. This requires the prover to compute a so-called evaluation proof, which establishes that the returned evaluation is indeed consistent with the committed polynomial. For some polynomial commitment schemes, such as Bulletproofs/IPA [BCC⁺16, BBB⁺18], evaluation proofs are quite slow and this cost can bottleneck the prover. However, for others, evaluation proof computation is a low-order cost [WTS⁺18, BBHR18]. In this work, we add another commitment scheme to this list, introducing Sona (Section 1.5), which combines the excellent commitment time of Hyrax, and evaluation proof computation involving sublinear cryptographic work, with the excellent verification costs of Nova.

Moreover, evaluation proofs exhibit excellent batching properties (whereby the prover can commit to many polynomials and only produce a single evaluation proof across all of them) [BGH19, KST22, BDFG20]. So in many contexts, computing opening proofs is not a bottleneck even when a scheme such as Bulletproofs/IPA.

For all of the above reasons, our accounting of prover cost in this work generally ignores the cost of polynomial evaluation proofs.

Summarizing Lasso’s prover costs. Based on the above accounting, Lasso’s prover costs when applied to a lookup table T can be summarized as follows.

- Setting the parameter $c = 1$, the Lasso prover commits to just $m + N$ field elements (using any multilinear polynomial commitment scheme), all of which are in $\{0, \dots, m\}$.¹¹ Using an MSM-based commitment scheme, this translates to very close to $m + N$ group operations.
- For $c > 1$, the Lasso prover applied to any decomposable table commits to $3cm + \alpha N^{1/c}$ field elements, all of which are in the set $\{0, \dots, \max\{m, N^{1/c}, q\} - 1\}$, where q is the largest value in any of the α sub-tables $T_1, \dots, T_\alpha$.
- The GeneralizedLasso prover applies to any MLE-structured table, and commits to the same number of field elements as the Lasso prover, but cm of them are random field elements, instead of small ones.

In all cases above, no party needs to cryptographically commit to the table T or subtables $T_1, \dots, T_\alpha$, so long as they are MLE-structured.

In Appendix B, we compare these costs with those of existing lookup arguments.

⁹Of course, the cost of each group operation depends on the size of the group’s base field, which is closely related to that of the scalar field $\mathbb{F}$. However, the *number* of group operations to compute the MSM depends only on K , not on $\mathbb{F}$.

¹⁰To be very precise, if $K \leq n$, then Pippenger’s algorithm performs only $(1 + o(1))n$ group operations.

¹¹In fact, for any $k \geq 1$, at most m/k of these field elements are larger than k .

1.3 A companion work: Jolt, and the lookup singularity

In the context of SNARKs, a *front-end* is a transformation or compiler that turns any computer program into an *intermediate representation*—typically a variant of circuit-satisfiability—so that a back-end (i.e., a SNARK for circuit-satisfiability) can be applied to establish that the prover correctly ran the computer program on a witness. A companion paper called **Jolt** (for “Just One Lookup Table”) shows that **Lasso**’s ability to handle gigantic tables without either prover or verifier ever materializing the whole table (so long as the table is modestly “structured”) enables substantial improvements in the front-end design.

Jolt’s idea is cleanest to describe in the context of a front-end for a simple virtual machine (VM), which in SNARK design has become synonymous with the notion of a CPU. A VM is defined by a set of primitive instructions (called an instruction set), one of which is executed at each step of the program. Typically, a front-end for a SNARK outputs a circuit, that for each step of the computation, (a) determines which instruction should be executed at that step and (b) executes the instruction. **Jolt** uses **Lasso** to replace part (b) at each step with a single lookup, into a gigantic lookup table. Specifically, consider the popular RISC-V instruction set **RIS**, targeted by the RISC-Zero project.¹² For each of the primitive RISC-V instructions f_i , the idea of **Jolt** is to create a lookup table that contains the entire evaluation table of f_i . For example, if f_i takes two 64-bit inputs, the table will have 2^{128} entries, whose (x, y) ’th entry is $f_i(x, y)$. One can “glue together” the tables for each instruction, into a single table of size 2^{128} times the number of instructions.

Jolt shows that for each of the RISC-V instructions (including multiplication instructions and division and remainder instructions), the resulting table has the structure that we require to apply **Lasso**. This leads to a front-end for VMs such as RISC-V that outputs much smaller circuits than prior front-ends, and has additional benefits such as easier auditability. Preliminary estimates from **Jolt** show that, when applied to the RISC-V instruction set over 64-bit data types, the prover commits to ≤ 65 field elements per step of the RISC-V CPU. Of these field elements, about a third lie in $\{0, 1\}$, only five are larger than about 2^{22} , and none are larger than 2^{64} . This means that **Jolt**’s prover costs when applied to a T -step execution of the RISC-V CPU on 64-bit data types is equivalent to computing roughly 6 multiexponentiations of size T if using a 256-bit field. Put another way, the **Jolt** prover’s runtime is equivalent to committing to about 6 arbitrary field elements per step of the RISC-V CPU.

We believe that **Lasso** and **Jolt** together essentially achieve a vision outlined by Barry Whitehat called *the lookup singularity* **Whi**. The lookup singularity seeks to transform arbitrary computer program into “circuits” that *only* perform lookups. Whitehat’s post outlines many benefits to achieving this vision, from improved performance to auditability and formal verification of the correctness of the front-end.

1.4 Lasso vs. GeneralizedLasso

The relationship between MLE-structured and decomposable tables. For any decomposable table $T \in \mathbb{F}^N$, there is always some low-degree extension polynomial $\hat{T}$ of T (namely, an extension of degree at most k in each variable) that can be evaluated in $O(\log N)$ time. In general, $\hat{T}$ is not necessarily multilinear, so a table being decomposable does not necessarily imply that it is MLE-structured. But **GeneralizedLasso** actually applies to any table with a low-degree extension that is evaluable in logarithmic time. In this sense, decomposability (the condition required to apply **Lasso**) is a stronger condition than what is necessary to apply **GeneralizedLasso**.

Pros and cons of GeneralizedLasso. We currently do not know specific tables of interest for which **GeneralizedLasso** applies but **Lasso** does not. In particular, all lookup tables arising in our companion paper **Jolt** are decomposable. However, there are benefits to **GeneralizedLasso** that may justify its increased costs.

For example, **Jolt** works conceptually by taking one lookup table for each primitive RISC-V instruction and concatenating them together into a single gigantic table. **Jolt** shows that each of the constituent tables (one per instruction) is both MLE-structured and decomposable. It is trivial to show that the concatenation of MLE-structured tables is MLE-structured, and the **GeneralizedLasso** verifier when applied to the concatenated table is essentially no more complicated than the **GeneralizedLasso** verifier when applied to each table individually.

¹²<https://www.risczero.com/>

In contrast, while it is true that the concatenation of decomposable tables is decomposable, implementing the concatenated table’s decomposition can be quite involved (at least, when the decompositions of the constituent tables are all different, as is the case with **Jolt**). This is particularly relevant because any implementation of the **Lasso** verifier applied to a given table depends on the decomposition of the table.

In summary, although **Lasso** is more performative than **GeneralizedLasso** in the context of decomposable tables, for some lookup tables the **Lasso** verifier implementation may be more complicated. Hence, even if future work does not identify MLE-structured tables of interest that are not decomposable, there are nonetheless simplicity and auditability benefits to **GeneralizedLasso** that may compensate for its diminished relative performance.¹³

1.5 Sona: A new transparent polynomial commitment scheme

Hyrax [WTS⁺18] provides a multilinear polynomial commitment scheme (for random evaluation queries) with attractive prover costs. To commit to an ℓ -variate multilinear polynomial (which means the polynomial has $m = 2^\ell$ coefficients), the prover performs $\sqrt{m}$ multiexponentiations each of length $\sqrt{m}$. To compute an evaluation proof, the prover performs $O(m)$ field operations and a $O(\sqrt{m})$ exponentiations (this requires applying Bulletproofs to prove an inner product instance consisting of vectors of length $\sqrt{m}$); an evaluation proof consists of $O(\log m)$ group elements.

The downside of Hyrax’s commitment scheme is that the verification costs are large: commitments consist of $\sqrt{m}$ group elements, and to verify an evaluation proof, the verifier has to perform two multiexponentiations of size $\sqrt{m}$. Dory [Lee21] can be thought of as reducing the Hyrax verifier’s costs from $O(\sqrt{m})$ to $O(\log m)$, at the cost of requiring pairings, and requiring the verifier to perform a logarithmic number of operations in the target group of a pairing-friendly group.

We propose a new polynomial commitment scheme (for random evaluation queries) called **Sona**, which reduces Hyrax’s verification costs in a different way. It uses two tools: Nova [KST22] and BabyHyrax (a simplified version of Hyrax in a manner that we describe next). In particular, BabyHyrax’s evaluation proofs consist of $O(\sqrt{m})$ field elements, but it requires *no* cryptographic operations (BabyHyrax does not invoke Bulletproofs and instead proves the inner product instance by sending the underlying vectors).

With these tools in hand, in **Sona**, rather than sending a commitment $\mathbf{cm}$ consisting of $\sqrt{n}$ group elements as in BabyHyrax, the Sona prover sends the *hash* $a = h(\mathbf{cm})$ of the group elements. And rather than sending an evaluation proof π that consists of $\sqrt{n}$ group elements and convinces the BabyHyrax verifier that $p(r) = v$, the Sona prover uses Nova to prove that it knows:

- A vector $\mathbf{cm}$ in $\mathbb{G}^{\sqrt{m}}$ such that $a = h(\mathbf{cm})$.
- A proof π that would have convinced the BabyHyrax verifier that $\mathbf{cm}$ is a commitment to a polynomial p such that $p(r) = v$.

The primary operations that Nova is applied to in this context are thus hashing a length- $\sqrt{m}$ vector $\mathbf{cm}$, and applying the BabyHyrax verifier’s checks on π , which mainly consists of two multiexponentiations of size $\sqrt{m}$ in $\mathbb{G}$. Applying the Nova prover to these computations results in $O(\sqrt{m} \log(\lambda)/\log(m))$ group operations for the prover. Hence, the prover’s total work to compute an evaluation proof for **Sona** is $O(m)$ field operations and $O(\sqrt{m} \log(\lambda)/\log(m))$ group operations. **Sona**’s evaluation proofs are a constant number of field elements and takes a constant-sized multiexponentiation to verify.

¹³Minimizing the number of field operations done by the *prover* in **GeneralizedLasso** is highly involved, and is the focus of Appendix G.5. However, this does not affect auditability, as the verifier is very simple, and only the verifier needs to be implemented correctly for the SNARK to be secure. Furthermore, there is a relatively simple **GeneralizedLasso** prover implementation that performs $O(m \log N)$ field operations for many lookup tables (Appendix G.4). We believe that this in many applications, including **Jolt**, this will be few enough field operations enough to avoid bottlenecking the prover, relative to commitment costs.

2 Preliminaries

We use λ to denote the security parameter and $\mathbb{F}$ to denote a finite field (e.g., the prime field $\mathbb{F}_p$ for a large prime p). We use “PPT algorithms” to refer to probabilistic polynomial time algorithms. Throughout this manuscript, we consider any field addition or multiplication to require constant time.

2.1 Multilinear extensions

An ℓ -variate polynomial $p: \mathbb{F}^\ell \rightarrow \mathbb{F}$ is said to be *multilinear* if p has degree at most one in each variable. Let $f: \{0, 1\}^\ell \rightarrow \mathbb{F}$ be any function mapping the ℓ -dimensional Boolean hypercube to a field $\mathbb{F}$. A polynomial $g: \mathbb{F}^\ell \rightarrow \mathbb{F}$ is said to *extend* f if $g(x) = f(x)$ for all $x \in \{0, 1\}^\ell$. It is well-known that for any $f: \{0, 1\}^\ell \rightarrow \mathbb{F}$, there is a unique *multilinear* polynomial $\tilde{f}: \mathbb{F}^\ell \rightarrow \mathbb{F}$ that extends f . The polynomial $\tilde{f}$ is referred to as the *multilinear extension* (MLE) of f .

The *total degree* of an ℓ -variate polynomial p refers to the maximum sum of the exponents in any monomial of p . Observe that if p is multilinear, then its total degree is at most ℓ . However, note that *not* all polynomials of total degree ℓ are multilinear.

A particular multilinear extension that arises frequently in the design of proof systems is $\tilde{\text{eq}}$, which is the MLE of the function $\text{eq}: \{0, 1\}^s \times \{0, 1\}^s \rightarrow \mathbb{F}$ defined as follows:

$$\text{eq}(x, e) = \begin{cases} 1 & \text{if } x = e \\ 0 & \text{otherwise.} \end{cases}$$

An explicit expression for $\tilde{\text{eq}}$ is:

$$\tilde{\text{eq}}(x, e) = \prod_{i=1}^s (x_i e_i + (1 - x_i)(1 - e_i)). \quad (1)$$

Indeed, one can easily check that the right hand side of Equation (1) is a multilinear polynomial, and that if evaluated at any input $(x, e) \in \{0, 1\}^s \times \{0, 1\}^s$, it outputs 1 if $x = e$ and 0 otherwise. Hence, the right hand side of Equation (1) is the unique multilinear polynomial extending eq . Equation (1) implies that $\tilde{\text{eq}}(r_1, r_2)$ can be evaluated at any point $(r_1, r_2) \in \mathbb{F}^s \times \mathbb{F}^s$ in $O(s)$ time.

Multilinear extensions of vectors. Given a vector $u \in \mathbb{F}^m$, we will often refer to the *multilinear extension of u* and denote this multilinear polynomial by $\tilde{u}$. $\tilde{u}$ is obtained by viewing u as a function mapping $\{0, 1\}^{\log m} \rightarrow \mathbb{F}$ in the following natural way: the function interprets its $(\log m)$ -bit input $(i_1, \dots, i_{\log m})$ as the binary representation of an integer i between 0 and $m - 1$, and outputs u_i . $\tilde{u}$ is defined to be the multilinear extension of this function.

Lagrange interpolation. An explicit expression for the MLE of any function is given by the following standard lemma (see [Tha22, Lemma 3.6]).

Lemma 1. *Let $f: \{0, 1\}^\ell \rightarrow \mathbb{F}$ be any function. Then the following multilinear polynomial $\tilde{f}$ extends f :*

$$\tilde{f}(x_1, \dots, x_\ell) = \sum_{w \in \{0, 1\}^\ell} f(w) \cdot \chi_w(x_1, \dots, x_\ell), \quad (2)$$

where, for any $w = (w_1, \dots, w_\ell)$,

$$\chi_w(x_1, \dots, x_\ell) := \prod_{i=1}^{\ell} (x_i w_i + (1 - x_i)(1 - w_i)). \quad (3)$$

Equivalently, $\chi_w(x_1, \dots, x_\ell) = \tilde{\text{eq}}(x_1, \dots, x_\ell, w_1, \dots, w_\ell)$.

The polynomials $\{\chi_w : w \in \{0,1\}^\ell\}$ are called the *Lagrange basis polynomials* for ℓ -variate multilinear polynomials. The evaluations $\{\tilde{f}(w) : w \in \{0,1\}^\ell\}$ are sometimes called the coefficients of $\tilde{f}$ in the *Lagrange basis*, terminology that is justified by Equation (2).

Dense representation for multilinear polynomials. Since the MLE of a function is unique, it offers the following method to represent any multilinear polynomial. Given a multilinear polynomial $g : \mathbb{F}^\ell \rightarrow \mathbb{F}$, it can be represented uniquely by the list of tuples L such that for all $i \in \{0,1\}^\ell$, $(\text{to-field}(i), g(i)) \in L$ if and only if $g(i) \neq 0$, where to-field is the canonical injection from $\{0,1\}^\ell$ to $\mathbb{F}$. We denote such a representation of g as $\text{DenseRepr}(g)$.

Definition 2.1. A multilinear polynomial g in ℓ variables is a *sparse multilinear polynomial* if $|\text{DenseRepr}(g)|$ is sub-linear in $O(2^\ell)$. Otherwise, it is a *dense multilinear polynomial*.

As an example, suppose $g : \mathbb{F}^{2^s} \rightarrow \mathbb{F}$. Suppose $|\text{DenseRepr}(g)| = O(2^s)$, then g is a sparse multilinear polynomial because $O(2^s)$ is sublinear in $O(2^{2^s})$.

The sum-check protocol. Let g be some ℓ -variate polynomial defined over a finite field $\mathbb{F}$. The purpose of the sum-check protocol is for prover to provide the verifier with the following sum:

$$H := \sum_{b \in \{0,1\}^\ell} g(b). \quad (4)$$

To compute H unaided, the verifier would have to evaluate g at all 2^ℓ points in $\{0,1\}^\ell$ and sum the results. The sum-check protocol allows the verifier to offload this “hard work” to the prover. It consists of ℓ rounds, one per variable of g . In round i , the prover sends a message consisting of d_i field elements, where d_i is the degree of g in its i 'th variable, and the verifier responds with a single (randomly chosen) field element. The verifier's runtime is $O\left(\sum_{i=1}^\ell d_i\right)$, plus the time required to evaluate g at a single point $r \in \mathbb{F}^\ell$. In the typical case that $d_i = O(1)$ for each round i , this means the total verifier time is $O(\ell)$, plus the time required to evaluate g at a single point $r \in \mathbb{F}^\ell$. This is exponentially faster than the 2^ℓ time that would generally be required for the verifier to compute H . See [AB09, Chapter 8] or [Tha22, §4.1] for details.

SNARKs We adapt the definition provided in [KST22].

Definition 2.2. Consider a relation $\mathcal{R}$ over public parameters, structure, instance, and witness tuples. A non-interactive argument of knowledge for $\mathcal{R}$ consists of PPT algorithms $(\mathcal{G}, \mathcal{P}, \mathcal{V})$ and deterministic $\mathcal{K}$, denoting the generator, the prover, the verifier and the encoder respectively with the following interface.

- $\mathcal{G}(1^\lambda) \rightarrow \text{pp}$: On input security parameter λ , samples public parameters pp .
- $\mathcal{K}(\text{pp}, \mathbf{s}) \rightarrow (pk, vk)$: On input structure $\mathbf{s}$, representing common structure among instances, outputs the prover key pk and verifier key vk .
- $\mathcal{P}(pk, u, w) \rightarrow \pi$: On input instance u and witness w , outputs a proof π proving that $(\text{pp}, \mathbf{s}, u, w) \in \mathcal{R}$.
- $\mathcal{V}(vk, u, \pi) \rightarrow \{0,1\}$: On input the verifier key vk , instance u , and a proof π , outputs 1 if the instance is accepting and 0 otherwise.

A non-interactive argument of knowledge satisfies completeness if for any PPT adversary $\mathcal{A}$

$$\Pr \left[\mathcal{V}(vk, u, \pi) = 1 \mid \begin{array}{l} \text{pp} \leftarrow \mathcal{G}(1^\lambda), \\ (\mathbf{s}, (u, w)) \leftarrow \mathcal{A}(\text{pp}), \\ (\text{pp}, \mathbf{s}, u, w) \in \mathcal{R}, \\ (pk, vk) \leftarrow \mathcal{K}(\text{pp}, \mathbf{s}), \\ \pi \leftarrow \mathcal{P}(pk, u, w) \end{array} \right] = 1.$$

A non-interactive argument of knowledge satisfies knowledge soundness if for all PPT adversaries $\mathcal{A}$ there

exists a PPT extractor $\mathcal{E}$ such that for all randomness ρ

$$\Pr \left[\begin{array}{l} \mathcal{V}(\text{vk}, u, \pi) = 1, \\ (\text{pp}, \text{s}, u, w) \notin \mathcal{R} \end{array} \middle| \begin{array}{l} \text{pp} \leftarrow \mathcal{G}(1^\lambda), \\ (\text{s}, u, \pi) \leftarrow \mathcal{A}(\text{pp}; \rho), \\ (\text{pk}, \text{vk}) \leftarrow \mathcal{K}(\text{pp}, \text{s}), \\ w \leftarrow \mathcal{E}(\text{pp}, \rho) \end{array} \right] = \text{negl}(\lambda).$$

A non-interactive argument of knowledge is succinct if the size of the proof π is polylogarithmic in the size of the statement proven.

Polynomial commitment scheme We adapt the definition from [BFS20]. A polynomial commitment scheme for multilinear polynomials is a tuple of four protocols $\text{PC} = (\text{Gen}, \text{Commit}, \text{Open}, \text{Eval})$:

- $pp \leftarrow \text{Gen}(1^\lambda, \mu)$: takes as input μ (the number of variables in a multilinear polynomial); produces public parameters pp .
- $\mathcal{C} \leftarrow \text{Commit}(pp, g)$: takes as input a μ -variate multilinear polynomial over a finite field $g \in \mathbb{F}[\mu]$; produces a commitment $\mathcal{C}$.
- $b \leftarrow \text{Open}(pp, \mathcal{C}, g)$: verifies the opening of commitment $\mathcal{C}$ to the μ -variate multilinear polynomial $g \in \mathbb{F}[\mu]$; outputs $b \in \{0, 1\}$.
- $b \leftarrow \text{Eval}(pp, \mathcal{C}, r, v, \mu, g)$ is a protocol between a PPT prover $\mathcal{P}$ and verifier $\mathcal{V}$. Both $\mathcal{V}$ and $\mathcal{P}$ hold a commitment $\mathcal{C}$, the number of variables μ , a scalar $v \in \mathbb{F}$, and $r \in \mathbb{F}^\mu$. $\mathcal{P}$ additionally knows a μ -variate multilinear polynomial $g \in \mathbb{F}[\mu]$. $\mathcal{P}$ attempts to convince $\mathcal{V}$ that $g(r) = v$. At the end of the protocol, $\mathcal{V}$ outputs $b \in \{0, 1\}$.

Definition 2.3. A tuple of four protocols $(\text{Gen}, \text{Commit}, \text{Open}, \text{Eval})$ is an extractable polynomial commitment scheme for multilinear polynomials over a finite field $\mathbb{F}$ if the following conditions hold.

- **Completeness.** For any μ -variate multilinear polynomial $g \in \mathbb{F}[\mu]$,

$$\Pr \left\{ \begin{array}{l} pp \leftarrow \text{Gen}(1^\lambda, \mu); \mathcal{C} \leftarrow \text{Commit}(pp, g); \\ \text{Eval}(pp, \mathcal{C}, r, v, \mu, g) = 1 \wedge v = g(r) \end{array} \right\} \geq 1 - \text{negl}(\lambda)$$

- **Binding.** For any PPT adversary $\mathcal{A}$, size parameter $\mu \geq 1$,

$$\Pr \left\{ \begin{array}{l} pp \leftarrow \text{Gen}(1^\lambda, \mu); (\mathcal{C}, g_0, g_1) = \mathcal{A}(pp); \\ b_0 \leftarrow \text{Open}(pp, \mathcal{C}, g_0); b_1 \leftarrow \text{Open}(pp, \mathcal{C}, g_1); \\ b_0 = b_1 \neq 0 \wedge g_0 \neq g_1 \end{array} \right\} \leq \text{negl}(\lambda)$$

- **Knowledge soundness.** Eval is a succinct argument of knowledge for the following NP relation given $pp \leftarrow \text{Gen}(1^\lambda, \mu)$.

$$\mathcal{R}_{\text{Eval}}(pp) = \{ \langle (\mathcal{C}, r, v), (g) \rangle : g \in \mathbb{F}[\mu] \wedge g(r) = v \wedge \text{Open}(pp, \mathcal{C}, g) = 1 \}$$

2.2 Polynomial IOPs and polynomial commitments

Most modern SNARKs work by combining a type of interactive protocol called a *polynomial IOP* [BFS20] with a cryptographic primitive called a *polynomial commitment scheme* [KZG10]. The combination yields a succinct *interactive* argument, which can then be rendered non-interactive via the Fiat-Shamir transformation [FS86], yielding a SNARK. Roughly, a polynomial IOP is an interactive protocol where, in one or more rounds, the prover may “send” to the verifier a large polynomial g . Because g is so large, one does not wish for the verifier to read a complete description of g . Instead, in any efficient polynomial IOP, the verifier only “queries” g at one point (or a handful of points). This means that the only information the verifier needs about g to check that the prover is behaving honestly is one (or a few) evaluations of g .

Scheme	Commit Size	Proof Size	$\mathcal{V}$ time	Commit time	$\mathcal{P}$ time
KZG + Gemini	1 $ \mathbb{G}_1 $	$O(\log N) \mathbb{G}_1 $	$O(\log N) \mathbb{G}_1$	$O(N) \mathbb{G}_1$	$O(N) \mathbb{G}_1$
Brakedown-commit	1 $ \mathbb{H} $	$O(\sqrt{N} \cdot \lambda) \mathbb{F} $	$O(\sqrt{N} \cdot \lambda) \mathbb{F}$	$O(N) \mathbb{F}, \mathbb{H}$	$O(N) \mathbb{F}, \mathbb{H}$
Orion-commit	1 $ \mathbb{H} $	$O(\lambda \log^2 N) \mathbb{H} $	$O(\lambda \log^2 N) \mathbb{H}$	$O(N) \mathbb{F}, \mathbb{H}$	$O(N) \mathbb{F}, \mathbb{H}$
Hyrax-commit	$O(\sqrt{N}) \mathbb{G} $	$O(\sqrt{N}) \mathbb{G} $	$O(\sqrt{N}) \mathbb{G}$	$O(N) \mathbb{G}$	$O(N) \mathbb{F}$
Dory	1 $ \mathbb{G}_T $	$O(\log N) \mathbb{G}_T $	$O(\log N) \mathbb{G}_T$	$O(N) \mathbb{G}_1$	$O(N) \mathbb{F}$
Sona (this work)	1 $ \mathbb{H} $	$O(1) \mathbb{G} $	$O(\sqrt{N}) \mathbb{G}$	$O(1) \mathbb{G}$	$O(N) \mathbb{F}, O(\sqrt{N}) \mathbb{G}$

Figure 1: Costs of polynomial commitment schemes when committing to a multilinear ℓ -variate polynomial over $\mathbb{F}$, with $N = 2^\ell$. All are transparent. $\mathcal{P}$ time refers to the time to compute evaluation proofs. In addition to the reported $O(N)$ field operations, Hyrax and Dory require roughly $O(N^{1/2})$ cryptographic work to compute evaluation proofs. $\mathbb{F}$ refers to a finite field, $\mathbb{H}$ refers to a collision-resistant hash, $\mathbb{G}$ refers to a cryptographic group where DLOG is hard, and $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T)$ refer to pairing-friendly groups. Columns with a suffix of “size” depict to the number of elements of a particular type, and columns with a suffix of “time” depict the number of operations (e.g., field multiplications or the size of multiexponentiations). Orion also requires $O(\sqrt{N})$ pre-processing time for the verifier.

In turn, a polynomial commitment scheme enables an untrusted prover to succinctly *commit* to a polynomial g , and later provide to the verifier any evaluation $g(r)$ for a point r chosen by the verifier, along with a proof that the returned value is indeed consistent with the committed polynomial. Essentially, a polynomial commitment scheme is exactly the cryptographic primitive that one needs to obtain a succinct argument from a polynomial IOP. Rather than having the prover send a large polynomial g to the verifier as in the polynomial IOP, the argument system prover instead cryptographically commits to g and later reveals any evaluations of g required by the verifier to perform its checks.

Whether or not a SNARK requires a trusted setup, as well as whether or not it is plausibly post-quantum secure, is determined by the polynomial commitment scheme used. If the polynomial commitment scheme does not require a trusted setup, neither does the resulting SNARK, and similarly if the polynomial commitment scheme is plausibly secure against quantum adversaries, then the SNARK is plausibly post-quantum sound.

Lasso can make use of any commitment schemes for *multilinear* polynomials. Note that any univariate polynomial commitment scheme can be transformed into a multilinear one, though the transformations introduce some overhead (e.g., [ZXZS20, BCHO22, CBBZ23]). A brief summary of the multilinear polynomial commitment schemes is provided in Figure 2.2. All of the schemes in the figure, except for KZG-based scheme, are transparent; Brakedown-commit and Orion-commit are plausibly post-quantum secure.

3 Technical overview

Suppose that the verifier has a commitment to a table $t \in \mathbb{F}^n$ as well as a commitment to another vector $a \in \mathbb{F}^m$. Suppose that a prover wishes to prove that all entries in a are in the table t . A simple observation in prior works [ZBK⁺22, ZGK⁺22] is that the prover can prove that it knows a sparse matrix $M \in \mathbb{F}^{m \times n}$ such that for each row of M , only one cell has a value of 1 and the rest are zeros and that $M \cdot t = a$, where $\cdot$ is the matrix-vector multiplication. This turns out to be equivalent, up to negligible soundness error, to confirming that

$$\sum_{y \in \{0,1\}^{\log N}} \widetilde{M}(r, y) \cdot \widetilde{t}(y) = \widetilde{a}(r), \quad (5)$$

for an $r \in \mathbb{F}^{\log m}$ chosen at random by the verifier. Here, $\widetilde{M}$, $\widetilde{a}$ and $\widetilde{t}$ are the so-called *multilinear extension polynomials* (MLEs) of M , t , and a (see Section 2.1 for details).

Lasso proves Equation (5) by having the prover commit to the sparse polynomial $\widetilde{M}$ using **Spark** and then prove the equation directly with a generalization of **Spark** called **Surge**. This provides the most efficient lookup argument when either the table t is “decomposable” (we discuss details of this below), or when t is unstructured but small. It turns out most tables that occur in practice (e.g., the ones that arise in **Jolt** are decomposable). When t is not decomposable, but still structured, a generalization of Lasso, which we refer to as **GeneralizedLasso**, proves Equation (5) using a combination of a new form of the sum-check protocol (which

we refer to as the sparse-dense sum-check protocol) and the **Spark** polynomial commitment scheme. We defer further details of **GeneralizedLasso** to Appendix [F](#).

3.1 Lasso’s starting point: The **Spark** sparse polynomial commitment scheme

Lasso’s starting point is **Spark**, an optimal sparse polynomial commitment scheme from Spartan [\[Set20\]](#). It allows an untrusted prover to prove evaluations of a sparse multilinear polynomial with costs proportional to the size of the dense representation of the sparse multilinear polynomial. Spartan established security of **Spark** under the assumption that certain metadata associated with a sparse polynomial is committed honestly, which sufficed for its application in the context of Spartan. In this paper, perhaps surprisingly, we prove that **Spark** remains secure even if that metadata is committed by an untrusted party (e.g., the prover), providing a standard commitment scheme for sparse polynomials.

The **Spark** sparse polynomial commitment scheme works as follows. The prover commits to a unique dense representation of the sparse polynomial g , using any polynomial commitment scheme for “dense” (multilinear) polynomials. The dense representation of g is effectively a list of all of the monomials of g with a non-zero coefficient (and the corresponding coefficient). More precisely, the list specifies all *multilinear Lagrange basis polynomials* with non-zero coefficient. Details as to what are the multilinear Lagrange basis polynomials are not relevant to this overview (but can be found in Section [2.1](#)).

When the verifier requests an evaluation $g(r)$ of the committed polynomial g , the prover returns the claimed evaluation v and needs to prove that v is indeed equal to the committed polynomial evaluated at r . Let c be such that $N = m^c$. As explained below, there is a simple and natural algorithm that takes as input the dense representation of g , and outputs $g(r)$ in $O(c \cdot m)$ time. **Spark** amounts to the bespoke SNARK establishing that the prover correctly ran this sparse-polynomial-evaluation algorithm on the committed description of g . Note that this perspective on **Spark** is somewhat novel, though it is partially implicit in the scheme itself and in an exposition of [\[Tha22\]](#) Section 16.2].

A time-optimal algorithm for evaluating a multilinear polynomial of sparsity m . We first describe a naive solution and then describe an optimal solution used in **Spark**.

A naive solution. Consider an algorithm that iterates over each Lagrange basis polynomials specified in the committed dense representation, evaluates that basis polynomial at r , multiplies by the corresponding coefficient, and adds the result to the evaluation. Unfortunately, a naive evaluation of a $(\log N)$ -variate Lagrange basis polynomial at r would take $O(\log N)$ time, resulting in a total runtime of $O(m \cdot \log N)$.

Eliminating the logarithmic factor. The key to achieving time $O(c \cdot m)$ is to ensure that each Lagrange basis polynomial can be evaluated in $O(c)$ time. This is done via the following procedure. This procedure is reminiscent of Pippenger’s algorithm for multiexponentiation, with m being the size of the multiexponentiation, and Lagrange basis polynomials with non-zero coefficients corresponding to exponents.

Decompose the $\log N = c \cdot \log m$ variables of r into c blocks, each of size $\log m$, writing $r = (r_1, \dots, r_c) \in (\mathbb{F}^{\log m})^c$. Then any $(\log N)$ -variate Lagrange basis polynomial evaluated at r can be expressed as a product of c “smaller” Lagrange basis polynomials, each defined over only $\log m$ variables, with the i ’th such polynomial evaluated at r_i . There are only $2^{\log m} = m$ multilinear Lagrange basis polynomials over $\log m$ variables. Moreover, there are now-standard algorithms that, for any input $r_i \in \mathbb{F}^{\log m}$, run in time m and evaluate all m of the $(\log m)$ -variate Lagrange basis polynomials at r_i . Hence, in $O(c \cdot m)$ total time, one can evaluate *all* m of these basis polynomials at each r_i , storing the results in a (write-once) memory M .

Given M , the time-optimal algorithm can evaluate *any* given $\log(N)$ -variate Lagrange basis polynomial at r by performing c lookups into memory, one for each block r_i , and multiplying together the results.^{[14](#)}

Note that we chose to decompose the $\log N$ variables into c blocks of length $\log m$ (rather than more, smaller blocks, or fewer, bigger blocks) to balance the runtime of the two phases of the algorithm, namely:

¹⁴This is also closely analogous to the behavior of tabulation hashing discussed earlier in the introduction, which is why we chose to highlight this example from algorithm design.

- The time required to “write to memory” the evaluations of all $(\log m)$ -variate Lagrange basis polynomials at $r_1, \dots, r_c$.
- The time required to evaluate $p(r)$ given the contents of memory.

In general, if we break the variables into c blocks of size $\ell = \log(N)/c = \log(m)$, the first phase will require time $c \cdot 2^\ell = cm$, and the second will require time $O(m \cdot c)$.

How the Spark prover proves it correctly ran the above time-optimal algorithm. To enable an untrusted prover to efficiently prove that it correctly ran the above algorithm to compute an evaluation of a sparse polynomial g at r , Spark uses *offline memory checking* [BEG⁺91] to prove read-write consistency. Furthermore, the contents of the memory is determined succinctly by r , so the verifier does not need any commitments to the contents of the memory. Spark effectively forces the prover to commit to the “execution trace” of the algorithm (which has size roughly $c \cdot m$, because the algorithm runs in time $O(c)$ for each of the m Lagrange basis polynomials with non-zero coefficient) plus $c \cdot N^{1/c} = O(c \cdot m)$. The latter term arises because at the end of m operations, the offline memory-checking technique requires the prover to supply certain access counts indicating the number of times a particular memory location was read during the course of the protocol. Moreover, note that this memory has size $c \cdot N^{1/c}$ if the algorithm breaks the $\log N$ variables into c blocks of size $\log(N)/c$. As we will see later, this is why Lasso’s prover winds up cryptographically committing to $3 \cdot c \cdot m + c \cdot N^{1/c}$ field elements.

Remark 1. *The cost incurred by Spark’s prover to “replay” to provide access counts at the very end of the algorithm’s execution can be amortized over multiple sparse polynomial evaluations. In particular, if the prover proves an evaluation of k sparse polynomials in the same number of variables, the aforementioned cost in the offline memory checking is reused across all k sparse polynomials.*

3.2 Surge: A generalization of Spark

Re-imagining Spark. A sparse polynomial commitment scheme can be viewed as having the prover commit to an m -sparse vector u of length N , where m is the number of non-zero coefficients of the polynomial, and N is the number of elements in a suitable basis. For univariate polynomials in the standard monomial basis, N is the degree, m is the number of non-zero coefficients, and u is the vector of coefficients. For an ℓ -variate multilinear polynomial g over the Lagrange basis, $N = 2^\ell$, m is the number of evaluation points over the Boolean hypercube $x \in \{0, 1\}^\ell$ such that $g(x) \neq 0$, and u is the vector of evaluations of g at all evaluation points over the hypercube $\{0, 1\}^\ell$.

An evaluation query to g at input r returns the inner product of the sparse vector u with the dense vector t consisting of the evaluations of all basis polynomials at r . In the multilinear case, for each $S \in \{0, 1\}^\ell$, the S ’th entry of t is $\chi_S(r)$. In this sense, *any* sparse polynomial commitment scheme achieves the following: it allows the prover to establish the value of the inner product $\langle u, t \rangle$ of a sparse (committed) vector u with a dense, structured vector t .

Spark $\rightarrow$ Surge. To obtain Surge from Spark, we critically examine the type of structure in t that is exploited by Spark, and introduce Surge as a natural generalization of Spark that supports any table t with this structure. More importantly, we observe that many lookup tables critically important in practice (e.g., those that arise in Jolt) exhibit this structure.

In more detail, the Surge prover essentially establishes that it correctly ran a natural $O(c \cdot m)$ -time algorithm for computing $\langle u, t \rangle$. This algorithm is a natural analog of the sparse polynomial evaluation algorithm described in Section 3.1: it iterates over every non-zero entry u_i of u , quickly computes $t_i = T[i]$ by performing one lookup into each of $O(c)$ “sub-tables” of size $N^{1/c}$, and quickly “combines” the result of each lookup to obtain t_i and hence $u_i \cdot t_i$. In this way, this algorithm takes just $O(c \cdot m)$ time to compute the desired inner product $\sum_{i: u_i \neq 0} u_i \cdot t_i$.

Details of the structure needed to apply Surge. In the case of Spark itself, the dense vector t is simply the *tensor product* of smaller vectors, $t_1, \dots, t_c$, each of size $N^{1/c}$. Specifically, Spark breaks r into c

“chunks” $r = (r_1, \dots, r_c) \in (\mathbb{F}^{(\log N)/c})^c$, where r is the point at which the **Spark** verifier wants to evaluate the committed polynomial. Then t_i contains the evaluations of all $((\log N)/c)$ -variate Lagrange basis polynomials evaluated at r_i . And for each $S = (S_1, \dots, S_c) \in (\{0, 1\}^{(\log N)/c})^c$, the S ’th entry of t is:

$$\prod_{i=1}^c t_i(r_i).$$

In general, **Spark** applies to any table vector t that is “decomposable” in a manner similar to the above. Specifically, suppose that $k \geq 1$ is an integer and there are $\alpha = k \cdot c$ tables $T_1, \dots, T_\alpha$ of size $N^{1/c}$ and an α -variate multilinear polynomial g such that the following holds. For any $r \in \{0, 1\}^{\log N}$, write $r = (r_1, \dots, r_c) \in (\{0, 1\}^{\log(N)/c})^c$, i.e., break r into c pieces of equal size. Suppose that for every $r \in \{0, 1\}^{\log N}$,

$$T[r] = g(T_1[r_1], \dots, T_k[r_1], T_{k+1}[r_2], \dots, T_{2k}[r_2], \dots, T_{\alpha-k+1}[r_c], \dots, T_\alpha[r_c]). \quad (6)$$

Simplifying slightly, **Surge** allows the prover to commit to a m -sparse vector $u \in \mathbb{F}^N$ and prove that the inner product of u and the table T (or more precisely the associated vector t) equals some claimed value. And the cost for the prover is dominated by the following operations.

- Committing to $3 \cdot \alpha \cdot m + \alpha \cdot N^{1/c}$ field elements, where $2 \cdot \alpha \cdot m + \alpha \cdot N^{1/c}$ of the committed elements are in the set

$$\{0, 1, \dots, \max\{m, N^{1/c}\} - 1\},$$

and the remaining $\alpha \cdot m$ of them are elements of the sub-tables $T_1, \dots, T_\alpha$. For many lookup tables T , these elements are themselves in the set $\{0, 1, \dots, N^{1/c} - 1\}$.

- Let b be the number of monomials in g . Then the **Surge** prover performs $O(k \cdot \alpha N^{1/c}) = O(b \cdot c \cdot N^{1/c})$ field operations. In many cases, the factor of b in the number of prover field operations can be removed.

We refer to tables that can be decomposed into sub-tables of size $N^{1/c}$ as per Equation (6) as having *Spark-only structure* (SOS), or more simply as being *decomposable*.

4 **Spark**: Spartan’s sparse polynomial commitment scheme, with a stronger security analysis

We prove a substantial strengthening of a result from Spartan [Set20, Lemma 7.6]. In particular, we prove that in Spartan’s sparse polynomial commitment scheme, which is called **Spark**, one does not need to assume that certain metadata associated with a sparse polynomial is committed honestly (in the case of Spartan, the metadata is committed by the setup algorithm, so it was sufficient for its purposes). We thereby obtain the first “standard” polynomial commitment scheme (i.e., meeting Definition 2.3) with prover costs *linear* in the number of non-zero coefficients. We prove this result without any substantive changes to **Spark**.

For simplicity of presentation, we make a minor change that does not affect costs nor analysis: we have the prover commit to metadata associated with the sparse polynomial at the time of proving an evaluation rather than when the prover commits to the sparse polynomial (the metadata depends only on the sparse polynomial, and in particular, it is independent of the point at which the sparse polynomial is evaluation, so the metadata can be committed either in the commit phase or when proving an evaluation). Our text below is adapted from an exposition of Spartan’s result by Golovnev et al. [GLS⁺21]. It is natural for the reader to conceptualize the **Spark** sparse polynomial commitment scheme as a bespoke SNARK for a prover to prove it correctly ran the sparse $(\log N)$ -variate multilinear polynomial evaluation algorithm described in Section 3.1 using c memories of size $N^{1/c}$.

4.1 A (slightly) simpler result: $c = 2$

We begin proving a special case of the final result, the proof of which exhibits all of the ideas and techniques. This special case (Theorem 1) describes a transformation from any commitment scheme for dense polynomials

defined over $\log m$ variables to one for sparse multilinear polynomials defined over $\log N = 2 \log m$ variables. It is the bespoke SNARK mentioned above when using $c = 2$ memories of size $N^{1/2}$.

The dominant costs for the prover in **Spark** is committing to 7 dense multilinear polynomials over $\log(m)$ -many variables, and 2 dense multilinear polynomials over $\log(N^{1/c})$ -many variables. In dense ℓ -variate multilinear polynomial commitment schemes, the prover time is roughly linear in 2^ℓ . Hence, so long as $m \geq N^{1/c}$, the prover time is dominated by the commitments to the 7 dense polynomials over $\log(m)$ -many variables. This ensures that the prover time is linear in the sparsity of the committed polynomial as desired (rather than linear in $2^{2 \log m} = m^2$, which would be the runtime of applying a dense polynomial commitment scheme directly to the sparse polynomial over $2 \log m$ variables).

The full result. If we wish to commit to a sparse multilinear polynomial over ℓ variables, let $N := 2^\ell$ denote the dimensionality of the space of ℓ -variate multilinear polynomials. For any desired integer $c \geq 2$, our final, general, result replaces these two memories (each of size equal to $N^{1/2}$) with c memories of size equal to $N^{1/c}$. Ultimately, the prover must commit to $(3c + 1)$ many dense $(\log m)$ -variate multilinear polynomials, and c many dense $(\log(N^{1/c}))$ -variate polynomials.

We begin with the simpler result where c equals 2 before stating and proving the full result.

Theorem 1 (Special case of Theorem 2 with $c = 2$). *Let $M = N^{1/2}$. Given a polynomial commitment scheme for $(\log M)$ -variate multilinear polynomials with the following parameters (where M is a positive integer and $WLOG$ a power of 2):*

- the size of the commitment is $c(M)$;
- the running time of the commit algorithm is $tc(M)$;
- the running time of the prover to prove a polynomial evaluation is $tp(M)$;
- the running time of the verifier to verify a polynomial evaluation is $tv(M)$;
- the proof size is $p(M)$,

there exists a polynomial commitment scheme for multilinear polynomials over $2 \log M = \log N$ variables that evaluate to a non-zero value at at most m locations over the Boolean hypercube $\{0, 1\}^{2 \log M}$, with the following parameters:

- the size of the commitment is $7c(m) + 2c(M)$;
- the running time of the commit algorithm is $O(tc(m) + tc(M))$;
- the running time of the prover to prove a polynomial evaluation is $O(tp(m) + tp(M))$;
- the running time of the verifier to verify a polynomial evaluation is $O(tv(m) + tv(M))$; and
- the proof size is $O(p(m) + p(M))$.

Representing sparse polynomials with dense polynomials. Let D denote a $(2 \log M)$ -variate multilinear polynomial that evaluates to a non-zero value at at most m locations over $\{0, 1\}^{2 \log M}$. For any $r \in \mathbb{F}^{2 \log M}$, we can express the evaluation of $D(r)$ as follows. Interpret $r \in \mathbb{F}^{2 \log M}$ as a tuple (r_x, r_y) in a natural manner, where $r_x, r_y \in \mathbb{F}^{\log M}$. Then by multilinear Lagrange interpolation (Lemma 1), we can write

$$D(r_x, r_y) = \sum_{(i,j) \in \{0,1\}^{\log M} \times \{0,1\}^{\log M} : D(i,j) \neq 0} D(i,j) \cdot \tilde{eq}(i, r_x) \cdot \tilde{eq}(j, r_y). \quad (7)$$

Claim 1. *Let to-field be the canonical injection from $\{0, 1\}^{\log M}$ to $\mathbb{F}$ and to-bits be its inverse. Given a $2 \log M$ -variate multilinear polynomial D that evaluates to a non-zero value at at most m locations over $\{0, 1\}^{2 \log M}$, there exist three $(\log m)$ -variate multilinear polynomials $\text{row}, \text{col}, \text{val}$ such that the following holds*

for all $r_x, r_y \in \mathbb{F}^{\log M}$.

$$D(r_x, r_y) = \sum_{k \in \{0,1\}^{\log m}} \text{val}(k) \cdot \tilde{e}q(\text{to-bits}(\text{row}(k)), r_x) \cdot \tilde{e}q(\text{to-bits}(\text{col}(k)), r_y). \quad (8)$$

Moreover, the polynomials' coefficients in the Lagrange basis can be computed in $O(m)$ time.

Proof. Since D evaluates to a non-zero value at at most m locations over $\{0,1\}^{2 \log M}$, D can be represented uniquely with m tuples of the form $(i, j, D(i, j)) \in (\{0,1\}^{\log M}, \{0,1\}^{\log M}, \mathbb{F})$. By using the natural injection to-field from $\{0,1\}^{\log M}$ to $\mathbb{F}$, we can view the first two entries in each of these tuples as elements of $\mathbb{F}$ (let to-bits denote its inverse). Furthermore, these tuples can be represented with three m -sized vectors $R, C, V \in \mathbb{F}^m$, where tuple k (for all $k \in [m]$) is stored across the three vectors at the k th location in the vector, i.e., the first entry in the tuple is stored in R , the second entry in C , and the third entry in V . Take row as the unique MLE of R viewed as a function $\{0,1\}^{\log m} \rightarrow \mathbb{F}$. Similarly, col is the unique MLE of C , and val is the unique MLE of V . The claim holds by inspection since Equations (7) and (8) are both multilinear polynomials in r_x and r_y and agree with each other at every pair $r_x, r_y \in \{0,1\}^{\log M}$. $\square$

Conceptually, the sum in Equation (8) is *exactly* what the sparse polynomial evaluation algorithm described in Section 3.1 computes term-by-term. Specifically, that algorithm (using $c = 2$ memories) filled up one memory with the quantities $\tilde{e}q(i, r_x)$ as i ranges over $\{0,1\}^{\log M}$ (see Equation (7), and the other memory with the quantities $\tilde{e}q(j, r_x)$, and then computed each term of Equation (8) via one lookup into each memory, to the respective memory cells with (binary) indices $\text{to-bits}(\text{row}(k))$ and $\text{to-bits}(\text{col}(k))$, followed by two field multiplications.

Commit phase. To commit to D , the committer can send commitments to the three $(\log m)$ -variate multilinear polynomials $\text{row}, \text{col}, \text{val}$ from Claim 1. Using the provided polynomial commitment scheme, this costs $O(m)$ finite field operations, and the size of the commitment to D is $O_\lambda(c(m))$.

Intuitively, the commit phase commits to a “dense” representation of the sparse polynomial, which simply lists all the Lagrange basis polynomial with non-zero coefficients (each specified as an element in $\{0, \dots, M-1\}^2$), along with the associated coefficient. This is exactly the input to the sparse polynomial evaluation algorithm described in Section 3.1.

In the evaluation phase described below, the prover proves that it correctly ran the sparse polynomial evaluation algorithm sketched in Section 3.1 on the committed polynomial in order to evaluate it at the requested evaluation point $(r_x, r_y) \in \mathbb{F}^{2 \log M}$.

A first attempt at the evaluation phase. Given $r_x, r_y \in \mathbb{F}^{\log M}$, to prove an evaluation of a committed polynomial, i.e., to prove that $D(r_x, r_y) = v$ for a purported evaluation $v \in \mathbb{F}$, consider the polynomial IOP in Figure 2 where the polynomial IOP assumes that the verifier has oracle access to the three $(\log m)$ -variate multilinear polynomial oracles that encode D (namely $\text{row}, \text{col}, \text{val}$).

Here, the oracles E_{r_x} and E_{r_y} should be thought of as the (purported) multilinear extensions of the values returned by each memory reads that the algorithm of Section 3.1 performed into each of its two memories, step-by-step over the course of its execution.

If the prover is honest, it is easy to see that it can convince the verifier about the correct of evaluations of D . Unfortunately, the two oracles that the prover sends in the first step of the depicted polynomial IOP can be completely arbitrary. To fix, this, $\mathcal{V}$ must *additionally* check that the following two conditions hold.

- $\forall k \in \{0,1\}^{\log m}$, $E_{r_x}(k) = \tilde{e}q(\text{to-bits}(\text{row}(k)), r_x)$; and
- $\forall k \in \{0,1\}^{\log m}$, $E_{r_y}(k) = \tilde{e}q(\text{to-bits}(\text{col}(k)), r_y)$.

A core insight of Spartan [Set20] is to check these two conditions using memory-checking techniques [BEG⁺91]. These techniques amount to an efficient randomized procedure to confirm that every memory read over the course of an algorithm's execution returns the value last written to that location.

1. $\mathcal{P} \rightarrow \mathcal{V}$: two $(\log m)$ -variate multilinear polynomials E_{rx} and E_{ry} as oracles. These polynomials are purported to respectively equal the multilinear extensions of the functions mapping $k \in \{0, 1\}^{\log m}$ to $\tilde{eq}(\text{to-bits}(\text{row}(k)), r_x)$ and $\tilde{eq}(\text{to-bits}(\text{col}(k)), r_y)$.
2. $\mathcal{V} \leftrightarrow \mathcal{P}$: run the sum-check reduction to reduce the check that

$$v = \sum_{k \in \{0, 1\}^{\log m}} \text{val}(k) \cdot E_{rx}(k) \cdot E_{ry}(k)$$

to checking if the following hold, where $r_z \in \mathbb{F}^{\log m}$ is chosen at random by the verifier over the course of the sum-check protocol:

- $\text{val}(r_z) \stackrel{?}{=} v_{\text{val}}$;
 - $E_{rx}(r_z) \stackrel{?}{=} v_{E_{rx}}$ and $E_{ry}(r_z) \stackrel{?}{=} v_{E_{ry}}$. Here, v_{val} , $v_{E_{rx}}$, and $v_{E_{ry}}$ are values provided by the prover at the end of the sum-check protocol.
3. $\mathcal{V}$: check if the three equalities hold with an oracle query to each of val , E_{rx} , E_{ry} .

Figure 2: A first attempt at a polynomial IOP for revealing a requested evaluation of a $(2 \log(M))$ -variate multilinear polynomial p over $\mathbb{F}$ such that $p(x) \neq 0$ for at most m values of $x \in \{0, 1\}^{2 \log(M)}$.

We take a detour to introduce new results that we rely on here.

Detour: Offline memory checking. Recall that in the offline memory checking algorithm of [BEG⁺91], a *trusted checker* issues operations to an untrusted memory. For our purposes, it suffices to consider only operation sequences in which each memory address is initialized to a certain value, and all subsequent operations are read operations. To enable efficient checking using multiset-fingerprinting techniques, the memory is modified so that in addition to storing a value at each address, the memory also stores a timestamp with each address. Moreover, each read operation is followed by a write operation that updates the timestamp associated with that address (but not the value stored there).

In prior descriptions of offline memory checking [BEG⁺91, CDD⁺03, SAGL18], the trusted checker maintains a single timestamp counter and uses it to compute write timestamps, whereas in *Spark* and our description below, the trusted checker does not use any local timestamp counter; rather, each memory cell maintains its own counter, which is incremented by the checker every time the cell is read.¹⁵ For this reason, we depart from the standard terminology in the memory-checking literature and henceforth refer to these quantities as *counters* rather than timestamps.

The memory-checking procedure is captured in the codebox below.

Local state of the checker: Two sets: RS and WS , which are initialized as follows.¹⁶ $\text{RS} = \{\}$, and for an M -sized memory, WS is initialized to the following set of tuples: for all $i \in [N^{1/c}]$, the tuple $(i, v_i, 0)$ is included in WS , where v_i is the value stored at address i , and the third entry in the tuple, 0, is an “initial count” associated with the value (intuitively capturing the notion that when v_i was written to address i , it was the first time that address was accessed). Here, $[M]$ denotes the set $\{0, 1, \dots, M-1\}$.

Read operations and an invariant. For a read operation at address a , suppose that the untrusted memory responds with a value-count pair (v, t) . Then the checker updates its local state as follows:

¹⁵The same timestamp update procedure was used in *Spartan*’s use of *Spark* [Set20, §7.2.3]. The purpose was to achieve a concrete efficiency benefit. In particular, *Spartan* used a separate timestamp counter for each cell and considered the case where all read timestamps were guaranteed to be computed honestly. In this case, the write timestamp is the result of incrementing an honestly returned read timestamp, which allows *Spartan* to not explicitly materialize write timestamps. Here, we are interested in the case where read timestamps themselves are not computed honestly.

¹⁶The checker in [BEG⁺91] maintains a fingerprint of these sets, but for our exposition, we let the checker maintain full sets.

1. $RS \leftarrow RS \cup \{(a, v, t)\};$
2. store $(v, t + 1)$ at address a in the untrusted memory; and
3. $WS \leftarrow WS \cup \{(a, v, t + 1)\}.$

The following claim captures the invariant maintained on the sets of the checker:

Claim 2. *Let $\mathbb{F}$ be a prime order field. Assuming that the domain of counts is $\mathbb{F}$ and that m (the number of reads issued) is smaller than the field characteristic $|\mathbb{F}|$. Let WS and RS denote the multisets maintained by the checker in the above algorithm at the conclusion of m read operations. If for every read operation, the untrusted memory returns the tuple last written to that location, then there exists a set S with cardinality M consisting of tuples of the form (k, v_k, t_k) for all $k \in [M]$ such that $WS = RS \cup S$. Moreover, S is computable in time linear in M .*

Conversely, if the untrusted memory ever returns a value v for a memory call $k \in [M]$ such v does not equal the value initially written to cell k , then there does not exist any set S such that $WS = RS \cup S$.

Proof. If for every read operation, the untrusted memory returns the tuple last written to that location, then it is easy to see the existence of the desired set S . It is simply the current state of the untrusted memory viewed as the set of address-value-count tuples.

We now prove the other direction in the claim. For notational convenience, let WS_i and RS_i ($0 \leq i \leq m$) denote the multisets maintained by the trusted checker at the conclusion of the i th read operation (i.e., WS_0 and RS_0 denote the multisets before any read operation is issued). Suppose that there is some read operation i that reads from address k , and the untrusted memory responds with a tuple (v, t) such that v differs from the value initially written to address k . This ensures that $(k, v, t) \in RS_j$ for all $j \geq i$, and in particular that $(k, v, t) \in RS$, where recall that RS is the read set at the conclusion of the m read operations. Hence, to ensure that there exists a set S such that $RS \cup S = WS$ at the conclusion of the procedure (i.e., to ensure that $RS \subseteq WS$), there must be some other read operation during which address k is read, and the untrusted memory returns tuple $(k, v, t - 1)$.¹⁷ This is because we have assumed that the value v was not written in the initialization phase, and outside of the initialization phase, the only way that the checker writes (k, v, t) to memory is if a read to address k returns tuple $(v, t - 1)$.

Accordingly, the same reasoning as above applies to tuple $(k, v, t - 1)$. That is, to ensure that $RS = WS$ at the conclusion of the procedure, there must be some other read operation at which address k is read, and the untrusted memory returns tuple $(k, v, t - 2)$. And so on. We conclude that for *every* field element in $\mathbb{F}$ of the form $t - i$ for $i = 1, 2, \dots, \text{char}(\mathbb{F})$, there is some read operation that returns (k, v, t') . Since there are m many read operations and the characteristic of field is greater than m , we obtain a contradiction. $\square$

Remark 2. Claim 2 assumes that the characteristic of the field is at least the number of read operations (if this is not the case, there is no contradiction in the conclusion that tuples of the form $(v, t - i)$ were written for all $i \in 1, \dots, \text{char}(\mathbb{F})$). We can nonetheless work over fields of smaller characteristic by modifying the procedure by which the checker updates the counts returned by each read operation. Specifically, rather than initializing counts to 0 and replacing a count t returned by a read operation with $t + 1$, we instead initialize the counts to 1, and replace a returned count t with $t \cdot g$, where g is a fixed generator of the multiplicative group of the field $\mathbb{F}$. With this modification, Claim 2 applies so long as $|\mathbb{F}| > m$.

Remark 3. The proof of Claim 2 implies that, if the checker ever performs a read to an “invalid” memory cell k , meaning a cell indexed by $k \notin [M]$, then regardless of the value and timestamp returned by the untrusted prover in response to that read, there does not exist any set S such that $WS = RS \cup S$.

¹⁷Recall here that counter arithmetic is done over $\mathbb{F}$, i.e., t and $t - 1$ are in $\mathbb{F}$.

Counter polynomials. To aid the polynomial evaluation proof of the sparse polynomial the prover commits to additional multilinear polynomials beyond E_{rx} and E_{ry} . We now describe these additional polynomials and how they are constructed.

Observe that given the size M of memory and a list of m addresses involved in read operations, one can compute two vectors $C_r \in \mathbb{F}^m, C_f \in \mathbb{F}^M$ defined as follows. For $k \in [m]$, $C_r[k]$ stores the count that would have been returned by the untrusted memory if it were honest during the k th read operation. Similarly, for $j \in [M]$, let $C_f[j]$ store the final count stored at memory location j of the untrusted memory (if the untrusted memory were honest) at the termination of the m read operations. Computing these three vectors requires computation comparable to $O(m)$ operations over $\mathbb{F}$.

Let $\text{read_ts} = \widetilde{C}_r, \text{write_cts} = \widetilde{C}_r + 1, \text{final_cts} = \widetilde{C}_f$. We refer to these polynomials as *counter polynomials*, which are unique for a given memory size M and a list of m addresses involved in read operations.

The actual evaluation proof. To prove the evaluation of a given a $(2 \log M)$ -variate multilinear polynomial D that evaluates to a non-zero value at at most m locations over $\{0, 1\}^{2 \log M}$, the prover sends the following polynomials in addition to E_{rx} and E_{ry} : two $(\log m)$ -variate multilinear polynomials as oracles ($\text{read_ts}_{\text{row}}, \text{read_ts}_{\text{col}}$), and two $(\log M)$ -variate multilinear polynomials ($\text{final_cts}_{\text{row}}, \text{final_cts}_{\text{col}}$), where $(\text{read_ts}_{\text{row}}, \text{final_cts}_{\text{row}})$ and $(\text{read_ts}_{\text{col}}, \text{final_cts}_{\text{col}})$ are respectively the counter polynomials for the m addresses specified by row and col over a memory of size M .

After that, in addition to performing the polynomial IOP depicted earlier in the proof (Figure 2), the core idea is to check if the two oracles sent by the prover satisfy the conditions identified earlier using Claim 2

Claim 3. *Given a $(2 \log M)$ -variate multilinear polynomial, suppose that $(\text{row}, \text{col}, \text{val})$ denote multilinear polynomials committed by the commit algorithm. Furthermore, suppose that*

$$(E_{rx}, E_{ry}, \text{read_ts}_{\text{row}}, \text{final_cts}_{\text{row}}, \text{read_ts}_{\text{col}}, \text{final_cts}_{\text{col}})$$

denote the additional polynomials sent by the prover at the beginning of the evaluation proof.

For any $r_x \in \mathbb{F}^{\log M}$, suppose that

$$\forall k \in \{0, 1\}^{\log m}, E_{rx}(k) = \tilde{eq}(\text{to-bits}(\text{row}(k)), r_x). \quad (9)$$

Then the following holds: $\text{WS} = \text{RS} \cup S$, where

- $\text{WS} = \{(\text{to-field}(i), \tilde{eq}(i, r_x), 0) : i \in \{0, 1\}^{\log(M)}\} \cup \{(\text{row}(k), E_{rx}(k), \text{write_cts}_{\text{row}}(k) = \text{read_ts}_{\text{row}}(k) + 1) : k \in \{0, 1\}^{\log m}\};$
- $\text{RS} = \{(\text{row}(k), E_{rx}(k), \text{read_ts}_{\text{row}}(k)) : k \in \{0, 1\}^{\log m}\};$ and
- $S = \{(\text{to-field}(i), \tilde{eq}(i, r_x), \text{final_cts}_{\text{row}}(i)) : i \in \{0, 1\}^{\log(M)}\}.$

Meanwhile, if Equation (9) does not hold, then there is no set S such that $\text{WS} = \text{RS} \cup S$, where WS and RS are defined as above.

Similarly, for any $r_y \in \mathbb{F}^{\log M}$, checking that $\forall k \in \{0, 1\}^{\log m}, E_{ry}(k) = \tilde{eq}(\text{to-bits}(\text{col}(k)), r_y)$ is equivalent (in the sense above) to checking that $\text{WS}' = \text{RS}' \cup S'$, where

- $\text{WS}' = \{(\text{to-field}(j), \tilde{eq}(j, r_y), 0) : j \in \{0, 1\}^{\log(M)}\} \cup \{(\text{col}(k), E_{ry}(k), \text{write_cts}_{\text{col}}(k) = \text{read_ts}_{\text{col}}(k) + 1) : k \in \{0, 1\}^{\log m}\};$
- $\text{RS}' = \{(\text{col}(k), E_{ry}(k), \text{read_ts}_{\text{col}}(k)) : k \in \{0, 1\}^{\log m}\};$ and
- $S' = \{(\text{to-field}(j), \tilde{eq}(j, r_y), \text{final_cts}_{\text{col}}(j)) : j \in \{0, 1\}^{\log(M)}\}.$

Proof. The result follows from an application of the invariant in Claim 2

Here, we clarify the following subtlety. The expression $\text{to-bits}(\text{row}(k))$ appearing in Equation (9) is not defined if $\text{row}(k)$ is outside of $[M]$ for any $k \in \{0, 1\}^{\log m}$. But in this event, Remark 3 nonetheless implies the conclusion of the theorem, namely that there is no set S such that $\text{WS} = \text{RS} \cup S$. The analogous conclusion holds by the same reasoning if $\text{col}(k)$ is outside of $[M]$ for any $k \in \{0, 1\}^{\log m}$. $\square$

There is no direct way to prove that the checks on sets in Claim 3 hold. Instead, we rely on public-coin, multiset hash functions to compress RS, WS, and S into a single element of $\mathbb{F}$ each. Specifically:

Claim 4 ([Set20]). *Given two multisets A, B where each element is from $\mathbb{F}^3$, checking that $A = B$ is equivalent to checking the following, except for a soundness error of $O(|A| + |B|)/|\mathbb{F}|$ over the choice of γ, τ : $\mathcal{H}_{\tau, \gamma}(A) = \mathcal{H}_{\tau, \gamma}(B)$, where $\mathcal{H}_{\tau, \gamma}(A) = \prod_{(a, v, t) \in A} (h_\gamma(a, v, t) - \tau)$, and $h_\gamma(a, v, t) = a \cdot \gamma^2 + v \cdot \gamma + t$. That is, if $A = B$, $\mathcal{H}_{\tau, \gamma}(A) = \mathcal{H}_{\tau, \gamma}(B)$ with probability 1 over randomly chosen values τ and γ in $\mathbb{F}$, while if $A \neq B$, then $\mathcal{H}_{\tau, \gamma}(A) = \mathcal{H}_{\tau, \gamma}(B)$ with probability at most $O(|A| + |B|)/|\mathbb{F}|$.*

Intuitively, Claim 4 gives an efficient randomized procedure for checking whether two sequences of tuples are permutations of each other. First, the procedure Reed-Solomon fingerprints each tuple (see [Tha22, Section 2.1] for an exposition). This is captured by the function h_γ and intuitively replaces each tuple with a single field element, such that distinct tuples are unlikely to collide. Second, the procedure applies a permutation-independent fingerprinting procedure $H_{\tau, \gamma}$ to confirm that the resulting two sequences of fingerprints are permutations of each other.

We are now ready to depict a polynomial IOP for proving evaluations of a committed sparse multilinear polynomial. Given $r_x, r_y \in \mathbb{F}^{\log M}$, to prove that $D(r_x, r_y) = v$ for a purported evaluation $v \in \mathbb{F}$, consider the polynomial IOP given in Figure 3, which assumes that the verifier has an oracle access to multilinear polynomial oracles that encode D (namely, row, col, val)

Completeness. Perfect completeness follows from perfect completeness of the sum-check protocol and the fact that the multiset equality checks using their fingerprints hold with probability 1 over the choice of τ, γ if the prover is honest.

Soundness. Applying a standard union bound to the soundness error introduced by probabilistic multiset equality checks with the soundness error of the sum-check protocol [LFKN90], we conclude that the soundness error for the depicted polynomial IOP is at most $O(m)/|\mathbb{F}|$.

Round and communication complexity. There are three invocations of the sum-check protocol. First, the sum-check protocol is applied on a polynomial with $\log m$ variables where the degree is at most 3 in each variable, so the round complexity is $O(\log m)$ and the communication cost is $O(\log m)$ field elements. Second, four sum-check-based “grand product” protocols are computed in parallel. Two of the grand products are over vectors of size M and the remaining two are over vectors of size m . Third, the depicted IOP runs four additional “grand products”, which incurs the same costs as above. In total, with the protocol of [SL20, Section 6] for grand products, the round complexity of the depicted IOP is $\tilde{O}(\log m + \log(N))$ and the communication cost is $\tilde{O}(\log m + \log N)$ field elements, where the $\tilde{O}$ notation hides doubly-logarithmic factors. The prover commits to an extra $O(m/\log^3 m)$ field elements.

Verifier time. The verifier’s runtime is dominated by its runtime in the grand product sum-check reductions, which is $\tilde{O}(\log m)$ field operations.

Prover Time. Using linear-time sum-checks [Tha13] in all three sum-check reductions (and using the linear-time prover in the grand product protocol [Tha13, SL20]), the prover’s time is $O(N)$ finite field operations for unstructured tables.

Finally, to prove Theorem 1, applying the compiler of [BFS20] to the depicted polynomial IOP with the given dense polynomial commitment primitive, followed by the Fiat-Shamir transformation [FS86], provides the desired non-interactive argument of knowledge for proving evaluations of committed sparse multilinear polynomials, with efficiency claimed in the theorem statement. Appendix E provides additional details of the grand product argument.

- //During the commit phase, $\mathcal{P}$ has committed to three $(\log m)$ -variate multilinear polynomials $\text{row}, \text{col}, \text{val}$.
1. $\mathcal{P} \rightarrow \mathcal{V}$: four $(\log m)$ -variate multilinear polynomials $E_{rx}, E_{ry}, \text{read_ts_row}, \text{read_ts_col}$ and two $(\log M)$ -variate multilinear polynomials $\text{final_cts_row}, \text{final_cts_col}$.
 2. Recall that Claim 1 (see Equation 8) shows that $D(r_x, r_y) = \sum_{k \in \{0,1\}^{\log m}} \text{val}(k) \cdot E_{rx}(k) \cdot E_{ry}(k)$ assuming that
 - $\forall k \in \{0,1\}^{\log m}, E_{rx}(k) = \tilde{eq}(\text{to-bits}(\text{row}(k)), r_x)$; and
 - $\forall k \in \{0,1\}^{\log m}, E_{ry}(k) = \tilde{eq}(\text{to-bits}(\text{col}(k)), r_y)$.
 Hence, $\mathcal{V}$ and $\mathcal{P}$ apply the sum-check protocol to the polynomial $\text{val}(k) \cdot E_{rx}(k) \cdot E_{ry}(k)$, which reduces the check that $v = \sum_{k \in \{0,1\}^{\log m}} \text{val}(k) \cdot E_{rx}(k) \cdot E_{ry}(k)$ to checking that the following equations hold, where $r_z \in \mathbb{F}^{\log m}$ chosen at random by the verifier over the course of the sum-check protocol:
 - $\text{val}(r_z) \stackrel{?}{=} v_{\text{val}}$; and
 - $E_{rx}(r_z) \stackrel{?}{=} v_{E_{rx}}$ and $E_{ry}(r_z) \stackrel{?}{=} v_{E_{ry}}$. Here, $v_{\text{val}}, v_{E_{rx}}$ and $v_{E_{ry}}$ are values provided by the prover at the end of the sum-check protocol.
 3. $\mathcal{V}$: check if the three equalities above hold with one oracle query each to each of $\text{val}, E_{rx}, E_{ry}$.
 4. // The following checks if E_{rx} is well-formed as per the first bullet in Step 2 above.
 5. $\mathcal{V} \rightarrow \mathcal{P}$: $\tau, \gamma \in_R \mathbb{F}$.
 6. $\mathcal{V} \leftrightarrow \mathcal{P}$: run a sum-check-based protocol for “grand products” ([Tha13, Proposition 2] or [SL20, Section 5 or 6]) to reduce the check that $\mathcal{H}_{\tau, \gamma}(\text{WS}) = \mathcal{H}_{\tau, \gamma}(\text{RS}) \cdot \mathcal{H}_{\tau, \gamma}(S)$, where RS, WS, S are as defined in Claim 3 and $\mathcal{H}$ is defined in Claim 4 to checking if the following hold, where $r_M \in \mathbb{F}^{\log M}, r_m \in \mathbb{F}^{\log m}$ are chosen at random by the verifier over the course of the sum-check protocol:
 - $\tilde{eq}(r_M, r_x) \stackrel{?}{=} v_{eq}$
 - $E_{rx}(r_m) \stackrel{?}{=} v_{E_{rx}}$
 - $\text{row}(r_m) \stackrel{?}{=} v_{\text{row}}; \text{read_ts_row}(r_m) \stackrel{?}{=} v_{\text{read_ts_row}}; \text{and } \text{final_cts_row}(r_M) \stackrel{?}{=} v_{\text{final_cts_row}}$
 7. $\mathcal{V}$: directly check if the first equality holds, which can be done with $O(\log M)$ field operations; check the remaining equations hold with an oracle query to each of $E_{rx}, \text{row}, \text{read_ts_row}, \text{final_cts_row}$.
 8. // The following steps check if E_{ry} is well-formed as per the second bullet in Step 2 above.
 9. $\mathcal{V} \rightarrow \mathcal{P}$: $\tau', \gamma' \in_R \mathbb{F}$.
 10. $\mathcal{V} \leftrightarrow \mathcal{P}$: run a sum-check-based reduction for “grand products” ([Tha13, Proposition 2] or [SL20, Sections 5 and 6]) to reduce the check that $\mathcal{H}_{\tau', \gamma'}(\text{WS}') = \mathcal{H}_{\tau', \gamma'}(\text{RS}') \cdot \mathcal{H}_{\tau', \gamma'}(S')$, where $\text{RS}', \text{WS}', S'$ are as defined in Claim 3 and $\mathcal{H}$ is defined in Claim 4 to checking if the following hold, where $r'_M \in \mathbb{F}^{\log M}, r'_m \in \mathbb{F}^{\log m}$ are chosen at random by the verifier in the sum-check protocol:
 - $\tilde{eq}(r'_M, r_y) \stackrel{?}{=} v'_{eq}$
 - $E_{ry}(r'_m) \stackrel{?}{=} v_{E_{ry}}$
 - $\text{col}(r'_m) \stackrel{?}{=} v_{\text{col}}; \text{read_ts_col}(r'_m) \stackrel{?}{=} v_{\text{read_ts_col}}; \text{and } \text{final_cts_col}(r'_M) \stackrel{?}{=} v_{\text{final_cts_col}}$
 11. $\mathcal{V}$: directly check if the first equality holds, which can be done with $O(\log M)$ field operations; check the remaining equations hold with an oracle query to each of $E_{ry}, \text{col}, \text{read_ts_col}, \text{final_cts_col}$.

Figure 3: Evaluation procedure of the Spark sparse polynomial commitment scheme.

Additional discussion and intuition. As previously discussed, the protocol in Figure 3 allows the prover to prove that it correctly ran the sparse polynomial evaluation algorithm described in Section 3.1 on the committed representation of the sparse polynomial. The core of the protocol lies in the memory-checking procedure, which enables the untrusted prover to establish that it produced the correct value upon every one of the algorithm’s reads into the $c = 2$ memories of size $M = N^{1/2}$. Intuitively, the values that the prover cryptographically commits to in the protocol are simply the values and counters returned by the aforementioned read operations (including a final “read pass” over both memories, which is required by the offline memory-checking procedure).

A key and subtle aspect of the above is that the prover does *not* have to cryptographically commit to the values written to memory in the algorithm’s first phase, when it initializes the two memories (aka lookup tables, albeit dynamically determined by the evaluation point (r_x, r_y)), of size $M = N^{1/2}$. This is because these lookup tables are MLE-structured, meaning that the verifier can evaluate the multilinear extension of these tables on its own. The whole point of cryptographically committing to these values is to let the verifier evaluate the multilinear extension thereof at a randomly chosen point in the grand product argument. Since the verifier can perform this evaluation quickly on its own, there is no need for the prover in the protocol of Figure 3 to commit to these values.

4.2 The general result

Theorem 1 gives a commitment scheme for m -sparse multilinear polynomials over $\log N = 2 \log(M)$ many variables, in which the prover commits to 7 dense multilinear polynomials over $\log m$ many variables, and 2 dense polynomials over $\log(M)$ many variables.

Suppose we want to support sparse polynomials over $c \log(M)$ variables for constant $c > 2$, while ensuring that the prover still only commits to $3c + 1$ many dense multilinear polynomials over $\log m$ many variables, and c many over $\log(N^{1/c})$ many variables. We can proceed as follows.

The function eq and its tensor structure. Recall that $\text{eq}_s: \{0, 1\}^s \times \{0, 1\}^s \rightarrow \{0, 1\}$ takes as input two vectors of length s and outputs 1 if and only if the vectors are equal. (In this section, we find it convenient to make explicit the number of variables over which eq is defined by including a subscript s .) Recall from Equation (1) that $\tilde{\text{eq}}_s(x, e) = \prod_{i=1}^s (x_i e_i + (1 - x_i)(1 - e_i))$.

Equation (7) expressed the evaluation $\tilde{D}(r_x, r_y)$ of a sparse $2 \log(M)$ -variate multilinear polynomial $\tilde{D}$ as

$$\tilde{D}(r_x, r_y) = \sum_{(i,j) \in \{0,1\}^{\log(M)} \times \{0,1\}^{\log(M)}} D(i, j) \cdot \tilde{\text{eq}}_{\log(M)}(i, r_x) \cdot \tilde{\text{eq}}_{\log(M)}(j, r_y). \quad (10)$$

The last two factors on the right hand side above have effectively factored $\tilde{\text{eq}}_{2 \log(M)}((i, j), (r_x, r_y))$ as the product of two terms that each test equality over $\log(M)$ many variables, namely:

$$\tilde{\text{eq}}_{2 \log(M)}((i, j), (r_x, r_y)) = \tilde{\text{eq}}_{\log(M)}(i, r_x) \cdot \tilde{\text{eq}}_{\log(M)}(j, r_y).$$

Within the sparse polynomial commitment scheme, this ultimately led to checking two different memories, each of size M , one of which we referred to as the “row” memory, and one as the “column” memory. For each memory checked, the prover had to commit to three $(\log m)$ -variate polynomials, e.g., E_{rx} , row , read_ts_row , and one $\log(M)$ -variate polynomial, e.g., final_cts_row .

Supporting $\log N = c \log M$ variables rather than $2 \log M$. If we want to support polynomials over $c \log(M)$ variables for $c > 2$, we simply factor $\tilde{\text{eq}}_{c \log(M)}$ into a product of c terms that test equality over $\log(M)$ variables each. For example, if $c = 3$, then we can write:

$$\tilde{\text{eq}}_{3 \log(M)}((i, j, k), (r_x, r_y, r_z)) = \tilde{\text{eq}}_{\log(M)}(i, r_x) \cdot \tilde{\text{eq}}_{\log(M)}(j, r_y) \cdot \tilde{\text{eq}}_{\log(M)}(k, r_z).$$

Hence, if D is a $(3 \log M)$ -variate polynomial, we obtain the following analog of Equation (10):

$$\tilde{D}(r_x, r_y, r_z) = \sum_{(i,j,k) \in \{0,1\}^{\log(M)} \times \{0,1\}^{\log(M)} \times \{0,1\}^{\log(M)}} D(i, j, k) \cdot \tilde{eq}_{\log(M)}(i, r_x) \cdot \tilde{eq}_{\log(M)}(j, r_y) \cdot \tilde{eq}_{\log(M)}(k, r_z). \quad (11)$$

Based on the above equation, straightforward modifications to the sparse polynomial commitment scheme lead to checking c different untrusted memories, each of size M , rather than two. For example, when $c = 3$, the first memory stores all evaluations of $\tilde{eq}_{\log(M)}(i, r_x)$ as i ranges over $\{0, 1\}^{\log m}$, the second stores $\tilde{eq}_{\log(M)}(j, r_y)$ as j ranges over $\{0, 1\}^{\log m}$, and the third stores $\tilde{eq}_{\log(M)}(k, r_z)$ as k ranges over $\{0, 1\}^{\log m}$. These are exactly the contents of the three lookup tables of size $N^{1/c}$ used by the sparse polynomial evaluation algorithm of Section 3.1 when $c = 3$.

For each memory checked, the prover has to commit to three multilinear polynomials defined over $\log(m)$ -many variables, and one defined over $\log(M) = \log(N)/c$ variables. We obtain the following theorem.

Theorem 2. *Given a polynomial commitment scheme for $(\log M)$ -variate multilinear polynomials with the following parameters (where M is a positive integer and $WLOG$ a power of 2):*

- the size of the commitment is $c(M)$;
- the running time of the commit algorithm is $tc(M)$;
- the running time of the prover to prove a polynomial evaluation is $tp(M)$;
- the running time of the verifier to verify a polynomial evaluation is $tv(M)$;
- the proof size is $p(M)$,

there exists a polynomial commitment scheme for $(c \log M)$ -variate multilinear polynomials that evaluate to a non-zero value at at most m locations over the Boolean hypercube $\{0, 1\}^{c \log M}$, with the following parameters:

- the size of the commitment is $(3c + 1)c(m) + c \cdot c(M)$;
- the running time of the commit algorithm is $O(c \cdot (tc(m) + tc(M)))$;
- the running time of the prover to prove a polynomial evaluation is $O(c(tp(m) + tp(M)))$;
- the running time of the verifier to verify a polynomial evaluation is $O(c(tv(m) + tv(M)))$;
- the proof size is $O(c(p(m) + p(M)))$.

Many polynomial commitment schemes have efficient batching properties for evaluation proofs. For such schemes, the factor c can be omitted in the final three bullet points of Theorem 2 (i.e., prover and verifier costs for verifying polynomial evaluation do not grow with c).

4.3 Specializing the Spark sparse commitment scheme to Lasso

In **Lasso**, if the prover is honest then the sparse polynomial commitment scheme is applied to the multilinear extension of a matrix M with m rows and N columns, where m is the number of lookups and N is the size of the table. If the prover is honest then each row of M is a unit vector.

In fact, we require the commitment scheme to enforce these properties even when the prover is potentially malicious. Achieving this simplifies the commitment scheme and provides concrete efficiency benefits. It also keeps **Lasso**'s polynomial IOP simple as it does not need additional invocations of the sum-check protocol to prove that M satisfies these properties.

First, the multilinear polynomial $\text{val}(k)$ is fixed to 1, and it is not committed by the prover. Recall from Claim 1 that $\text{val}(k)$ extends the function that maps a bit-vector $k \in \{0, 1\}^{\log m}$ to the value of the k 'th non-zero evaluation of the sparse function. Since M is a $\{0, 1\}$ -valued matrix, $\text{val}(k)$ is just the constant polynomial that evaluates to 1 at all inputs.

//During the commit phase applied to the multilinear extension $\widetilde{M}$ of $m \times N$ matrix M with each row a unit vector, $\mathcal{P}$ has committed to c different ℓ -variate multilinear polynomials $\text{dim}_1, \dots, \text{dim}_c$, where $\ell = \log(N^{1/c})$. These are analogs of the polynomials `row` and `col` from Figure 3. dim_i is purported to provide the indices of the cells of the i 'th memory that are read by the sparse polynomial evaluation algorithm of Section 3.1. Note that these indices depend only on the the locations of the non-zero entries of $\widetilde{M}$.

//If $\mathcal{P}$ is honest, then each dim_i maps $\{0, 1\}^{\log m}$ to $\{0, \dots, N^{1/c} - 1\}$. For each $j \in \{0, 1\}^{\log m}$, $(\text{dim}_1(j), \dots, \text{dim}_c(j))$ is interpreted as specifying the identity of the unique non-zero entry of row j of M .

// $\mathcal{V}$ requests to evaluate $\widetilde{M}$ at input (r, r') where $r' = (r'_1, \dots, r'_c) \in (\mathbb{F}^\ell)^c$.

1. $\mathcal{P} \rightarrow \mathcal{V}$: $2c$ different $(\log m)$ -variate multilinear polynomials $E_1, \dots, E_c$, `read_ts1`, `read_tsc` and c different ℓ -variate multilinear polynomials `final_cts1`, `final_ctsc`.

//If $\mathcal{P}$ is honest, then `read_ts1`, `read_tsc` and `final_cts1`, `final_ctsc` map $\{0, 1\}^{\log m}$ to $\{0, \dots, m - 1\}$, as these are “counter polynomials” for each of the c memories.

//If $\mathcal{P}$ is honest, then $E_1, \dots, E_c$ contain the values returned by each read operation that the sparse polynomial evaluation algorithm of Section 3.1 makes to each of the c memories.

2. Recall (Equation 11) that $\widetilde{M}(r, r') = \sum_{k \in \{0, 1\}^{\log m}} \widetilde{\mathbf{eq}}(r, k) \cdot \prod_{i=1}^c E_i(k)$, assuming that

- $\forall k \in \{0, 1\}^{\log m}, E_i(k) = \widetilde{\mathbf{eq}}(\text{to-bits}(\text{dim}_i(k)), r'_i)$.

Hence, $\mathcal{V}$ and $\mathcal{P}$ apply the sum-check protocol to the polynomial $g(k) := \widetilde{\mathbf{eq}}(r, k) \cdot \prod_{i=1}^c E_i(k)$, which reduces the check that $v = \sum_{k \in \{0, 1\}^{\log m}} \widetilde{\mathbf{eq}}(r, k) \prod_{i=1}^c E_i(k)$ to checking that the following equations hold, where $r_z \in \mathbb{F}^{\log m}$ chosen at random by the verifier over the course of the sum-check protocol:

- $E_i(r_z) \stackrel{?}{=} v_{E_i}$ for $i = 1, \dots, c$. Here, $v_{E_1}, \dots, v_{E_c}$ are values provided by the prover at the end of the sum-check protocol.

3. $\mathcal{V}$: check if the above equalities hold with one oracle query to each E_i .

// The following checks if E_i is well-formed as per the first bullet in Step 2 above.

4. $\mathcal{V} \rightarrow \mathcal{P}$: $\tau, \gamma \in_R \mathbb{F}$.

//In practice, one would apply a single sum-check protocol to a random linear combination of the below polynomials. For brevity, we describe the protocol as invoking c independent instances of sum-check.

5. $\mathcal{V} \leftrightarrow \mathcal{P}$: For $i = 1, \dots, c$, run a sum-check-based protocol for “grand products” ([Tha13, Proposition2] or [SL20, Section 5 or 6]) to reduce the check that $\mathcal{H}_{\tau, \gamma}(\text{WS}) = \mathcal{H}_{\tau, \gamma}(\text{RS}) \cdot \mathcal{H}_{\tau, \gamma}(S)$, where RS, WS, S are as defined in Claim 3 and $\mathcal{H}$ is defined in Claim 4 to checking if the following hold, where $r''_i \in \mathbb{F}^\ell, r'''_i \in \mathbb{F}^{\log m}$ are chosen at random by the verifier over the course of the sum-check protocol:

- $E_i(r'''_i) \stackrel{?}{=} v_{E_i}$
- $\text{dim}_i(r'''_i) \stackrel{?}{=} v_i$; `read_tsi`(r'''_i) $\stackrel{?}{=} v_{\text{read_ts}_i}$; and `final_ctsi`(r'''_i) $\stackrel{?}{=} v_{\text{final_cts}_{\text{row}}}$

6. $\mathcal{V}$: check that the remaining equations hold with an oracle query to each of $E_i, \text{dim}_i, \text{read_ts}_i, \text{final_cts}_i$.

Figure 4: Evaluation procedure of the Spark sparse polynomial commitment scheme, optimized for its application to M in Lasso.

Second, for any $k = (k_1, \dots, k_{\log m}) \in \{0, 1\}^{\log m}$, the k 'th non-zero entry of M is in row $\text{to-field}(k) = \sum_{j=1}^{\log m} 2^{j-1} \cdot k_j$. Hence, in Equation (8) of Claim 1, $\text{to-bits}(\text{row}(k))$ is simply k .¹⁸ This means that $E_{rx}(k) = \text{eq}(k, r_x)$, which the verifier can evaluate on its own in logarithmic time. With this fact in hand, the prover does not commit to E_{rx} nor prove that it is well-formed.

In terms of costs in the resulting sparse polynomial commitment scheme applied to $\widetilde{M}$, this effectively removes the contribution of the first $\log m$ variables of $\widetilde{M}$ to the costs. Hence, the costs are that of applying the commitment scheme to an m -sparse $\log(N)$ -variate polynomial (with val fixed to 1).

This means that, setting $c = 2$ for illustration, the prover commits to 6 multilinear polynomials with $\log(m)$ variables each and to two multilinear polynomials with $(1/2) \log N$ variables each.

Figure 4 describes **Spark** specialized for **Lasso** to commit to $\widetilde{M}$. The prover commits to $3c$ dense $(\log(m))$ -variate multilinear polynomials, called $\text{dim}_1, \dots, \text{dim}_c$ (the analogs of the **row** and **col** polynomials of Section 4.1), $E_1, \dots, E_c$, and $\text{read_ts}_1, \dots, \text{read_ts}_c$, as well as c dense multilinear polynomials in $\log(N^{1/c}) = \log(N)/c$ variables, called $\text{final_cts}_1, \dots, \text{final_cts}_c$. Each dim_i is purported to be the memory cell from the i 'th memory that the sparse polynomial evaluation algorithm (§3.1) reads at each of its m timesteps, $E_1, \dots, E_c$ the values returned by those reads, and $\text{read_ts}_1, \dots, \text{read_ts}_c$ the associated counts. $\text{final_cts}_1, \dots, \text{final_cts}_c$ are purported to be to counts returned by the memory checking procedure's final pass over each of the c memories.

If the prover is honest, then $\text{dim}_1, \dots, \text{dim}_c$ each map $\{0, 1\}^{\log m}$ to $\{0, \dots, N^{1/c} - 1\}$, and $\text{read_ts}_1, \dots, \text{read_ts}_c$ each map $\{0, 1\}^{\log m}$ to $\{0, \dots, m - 1\}$ and $\text{final_cts}_1, \dots, \text{final_cts}_c$ each map $\{0, 1\}^{\log m}$ to $\{0, \dots, m - 1\}$. In fact, for any integer $j > 0$, at most m/j out of the m evaluations of each counter polynomial read_ts_i and final_cts_i can be larger than j .

5 Surge: A generalization of Spark, providing Lasso

The technical core of the **Lasso** lookup argument is **Surge**, a generalization of **Spark**. In particular, **Lasso** is simply a straightforward use of **Surge**.

Recall from Section 4 and Figure 4 that **Spark** allows the untrusted **Lasso** prover to commit to $\widetilde{M}$, purported to be the multilinear extension of an $m \times N$ matrix M , with each row equal to a unit vector, such that $M \cdot t = a$. The commitment phase of **Surge** is same as that of **Spark**. **Surge** generalizes **Spark** in that the **Surge** prover proves a larger class of statements about the committed polynomial $\widetilde{M}$ (**Spark** focused only on proving *evaluations* of the sparse polynomial $\widetilde{M}$).

Overview of Lasso. In **Lasso**, after committing to $\widetilde{M}$, the **Lasso** verifier picks a random $r \in \mathbb{F}^{\log m}$ and seeks to confirm that

$$\sum_{j \in \{0, 1\}^{\log N}} \widetilde{M}(r, j) \cdot t(j) = \widetilde{a}(r). \quad (12)$$

Indeed, if $M \cdot t$ and a are the same vector, then Equation (12) holds for every choice of r , while if $Mt \neq a$, then by the Schwartz-Zippel lemma, Equation (12) holds with probability at most $\frac{\log m}{|\mathbb{F}|}$. So up to soundness error $\frac{\log m}{|\mathbb{F}|}$, checking that $Mt = a$ is equivalent to checking that Equation (12) holds.

In **Lasso**, the verifier obtains $\widetilde{a}(r)$ via the polynomial commitment to $\widetilde{a}$. Then, the prover establishes Equation (12) using **Surge**. Specifically, **Surge** generalizes **Spark**'s procedure for generating evaluation proofs, to directly produce a proof as to the value of the left hand side of Equation (12). Essentially, the proof *proves* that the prover correctly ran a (very efficient) algorithm for evaluating the left hand side of Equation (12).

A roughly $O(\alpha m)$ -time algorithm for computing the LHS of Equation (12). From Equation (7),

$$\widetilde{M}(r, y) = \sum_{(i, j) \in \{0, 1\}^{\log m + \log N}} M_{i, j} \cdot \widetilde{eq}(i, r) \cdot \widetilde{eq}(j, y).$$

¹⁸More precisely, this holds if we define r_x to be in $\mathbb{F}^{\log m}$ and r_y to be in $\mathbb{F}^{\log N}$, rather than defining them both to be in $\mathbb{F}^{\log M} = \mathbb{F}^{(1/2)(\log m + \log n)}$.

Hence, letting $\text{nz}(i)$ denote the unique column in row i of M that contains a non-zero value (namely, the value 1), the left hand side of Equation (12) equals

$$\sum_{i \in \{0,1\}^{\log m}} \tilde{eq}(i, r) \cdot T[\text{nz}(i)]. \quad (13)$$

Suppose that T is a SOS table. This means that there is an integer $k \geq 1$ and $\alpha = k \cdot c$ tables $T_1, \dots, T_\alpha$ of size $N^{1/c}$, as well as an α -variate multilinear polynomial g such that the following holds. Suppose that for every $r = (r_1, \dots, r_c) \in (\{0, 1\}^{\log(N)/c})^c$,

$$T[r] = g(T_1[r_1], \dots, T_k[r_1], T_{k+1}[r_2], \dots, T_{2k}[r_2], \dots, T_{\alpha-k+1}[r_c], \dots, T_\alpha[r_c]). \quad (14)$$

For each $i \in \{0, 1\}^{\log m}$, decompose $\text{nz}(i)$ and $(\text{nz}_1(i), \dots, \text{nz}_c(i)) \in [N^{1/c}]^c$. Then Expression (13) equals

$$\sum_{i \in \{0,1\}^{\log m}} \tilde{eq}(i, r) \cdot g(T_1[\text{nz}_1(i)], \dots, T_k[\text{nz}_1(i)], T_{k+1}[\text{nz}_2(i)], \dots, T_{2k}[\text{nz}_2(i)], \dots, T_{\alpha-k+1}[\text{nz}_c(i)], \dots, T_\alpha[\text{nz}_c(i)]). \quad (15)$$

The algorithm to compute Expression (15) simply initializes all tables $T_1, \dots, T_\alpha$, then iterates over every $i \in \{0, 1\}^m$ and computes the i 'th term of the sum with a single lookup into each table (of course, the algorithm evaluates g at the results of the lookups into $T_1, \dots, T_\alpha$, and multiplies the result by $\tilde{eq}(i, r)$).

Description of Surge. The commitment to $\widetilde{M}$ in *Surge* consists of commitments to c multilinear polynomials $\text{dim}_1, \dots, \text{dim}_c$, each over $\log m$ variables. dim_i is purported to be the multilinear extension of nz_i .

The verifier chooses $r \in \{0, 1\}^{\log m}$ at random and requests that the *Surge* prover prove that the committed polynomial $\widetilde{M}$ satisfy Equation (13). The prover does so by proving it ran the aforementioned algorithm for evaluating Expression (15). Following the memory-checking procedure in Section 4 with each table $T_i: i = 1, \dots, \alpha$ viewed as a memory of size $N^{1/c}$, this entails committing for each i to $\log(m)$ -variate multilinear polynomials E_i and read_ts_i (purported to capture the value and count returned by each of the m lookups into T_i) and a $\log(N^{1/c})$ -variate multilinear polynomial final_cts_i (purported to capture the final count for each memory cell of T_i).

Let $\tilde{t}_i$ be the multilinear extension of the vector t_i whose j 'th entry is $T_i[j]$. The sum-check protocol is applied to compute

$$\sum_{j \in \{0,1\}^{\log m}} \tilde{eq}(r, j) \cdot g(E_1(j), \dots, E_\alpha(j)). \quad (16)$$

At the end of the sum-check protocol, the verifier needs to evaluate $\tilde{eq}(r, r') \cdot g(E_1(r'), \dots, E_\alpha(r'))$ at a random point $r' \in \mathbb{F}^{\log m}$, which it can do with one evaluation query to each E_i (the verifier can compute $\tilde{eq}(r, r')$ on its own in $O(\log m)$ time).

The verifier must still check that each E_i is well-formed, in the sense that $E_i(j)$ equals $T_i[\text{dim}_i(j)]$ for all $j \in \{0, 1\}^{\log m}$. This is done exactly as in *Spark* to confirm that for each of the α memories, $\text{WS} = \text{RS} \cup S$ (see Claims 3 and 4 and Figure 4). At the end of this procedure, for each $i = 1, \dots, \alpha$, the verifier needs to evaluate each of dim_i , read_ts_i , final_cts_i at a random point, which it can do with one query to each. The verifier also needs to evaluate the multilinear extension $\tilde{t}_i$ of each sub-table T_i for each $i = 1, \dots, \alpha$ at a single point. T being SOS guarantees that the verifier can compute each of these evaluations in $O(\log(N)/c)$ time.

Prover time. Besides committing to the polynomials $\text{dim}_i, E_i, \text{read_ts}_i, \text{final_cts}_i$ for each of the α memories and producing one evaluation proof for each (in practice, these would be batched), the prover must compute its messages in the sum-check protocol used to compute Expression (16) and the grand product arguments (which can be batched). Using the linear-time sum-check protocol [CTY11, Tha13, Set20], the prover can compute its messages in the sum-check protocol used to compute Expression (16) with $O(b \cdot k \cdot \alpha \cdot m)$ field operations, where recall that $\alpha = k \cdot c$ and b is the number of monomials in g . If $k = O(1)$, then this is

T is an SOS lookup table of size N , meaning there are $\alpha = kc$ tables $T_1, \dots, T_\alpha$, each of size $N^{1/c}$, such that for any $r \in \{0, 1\}^{\log N}$, $T[r] = g(T_1[r_1], \dots, T_k[r_1], T_{k+1}[r_2], \dots, T_{2k}[r_2], \dots, T_{\alpha-k+1}[r_c], \dots, T_\alpha[r_c])$. During the commit phase, $\mathcal{P}$ commits to c multilinear polynomials $\text{dim}_1, \dots, \text{dim}_c$, each over $\log m$ variables. dim_i is purported to provide the indices of $T_{(i-1)k+1}, \dots, T_{ik}$ the natural algorithm computing $\sum_{i \in \{0, 1\}^{\log m}} \tilde{\text{eq}}(i, r) \cdot T[\text{nz}[i]]$ (see Equation (15)).

// $\mathcal{V}$ requests $\langle u, t \rangle$, where the i th entry of t is $T[i]$ and the y th entry of u is $\widetilde{M}(r, y)$.

1. $\mathcal{P} \rightarrow \mathcal{V}$: 2α different $(\log m)$ -variate multilinear polynomials $E_1, \dots, E_\alpha$, $\text{read_ts}_1, \dots, \text{read_ts}_\alpha$ and α different $(\log(N)/c)$ -variate multilinear polynomials $\text{final_cts}_1, \dots, \text{final_cts}_\alpha$.
// E_i is purported to specify the values of each of the m reads into T_i .
// $\text{read_ts}_1, \dots, \text{read_ts}_\alpha$ and $\text{final_cts}_1, \dots, \text{final_cts}_\alpha$, are “counter polynomials” for each of the α sub-tables T_i .
2. $\mathcal{V}$ and $\mathcal{P}$ apply the sum-check protocol to the polynomial $h(k) := \tilde{\text{eq}}(r, k) \cdot g(E_1(k), \dots, E_\alpha(k))$, which reduces the check that $v = \sum_{k \in \{0, 1\}^{\log m}} g(E_1(k), \dots, E_\alpha(k))$ to checking that the following equations hold, where $r_z \in \mathbb{F}^{\log m}$ chosen at random by the verifier over the course of the sum-check protocol:
 - $E_i(r_z) \stackrel{?}{=} v_{E_i}$ for $i = 1, \dots, \alpha$. Here, $v_{E_1}, \dots, v_{E_\alpha}$ are values provided by the prover at the end of the sum-check protocol.
3. $\mathcal{V}$: check if the above equalities hold with one oracle query to each E_i .
4. // The following checks if E_i is well-formed, i.e., that $E_i(j)$ equals $T_i[\text{dim}_i(j)]$ for all $j \in \{0, 1\}^{\log m}$.
5. $\mathcal{V} \rightarrow \mathcal{P}$: $\tau, \gamma \in_R \mathbb{F}$.
// In practice, one would apply a single sum-check protocol to a random linear combination of the below polynomials. For brevity, we describe the protocol as invoking c independent instances of sum-check.
6. $\mathcal{V} \leftrightarrow \mathcal{P}$: For $i = 1, \dots, \alpha$, run a sum-check-based protocol for “grand products” ([Tha13, Proposition2] or [SL20, Section 5 or 6]) to reduce the check that $\mathcal{H}_{\tau, \gamma}(\text{WS}) = \mathcal{H}_{\tau, \gamma}(\text{RS}) \cdot \mathcal{H}_{\tau, \gamma}(S)$, where RS, WS, S are as defined in Claim 3 and $\mathcal{H}$ is defined in Claim 4 to checking if the following hold, where $r_i'' \in \mathbb{F}^\ell$, $r_i''' \in \mathbb{F}^{\log m}$ are chosen at random by the verifier over the course of the sum-check protocol:
 - $E_i(r_i''') \stackrel{?}{=} v_{E_i}$
 - $\text{dim}_i(r_i''') \stackrel{?}{=} v_i$; $\text{read_ts}_i(r_i''') \stackrel{?}{=} v_{\text{read_ts}_i}$; and $\text{final_cts}_i(r_i'') \stackrel{?}{=} v_{\text{final_cts}_i}$
7. $\mathcal{V}$: Check the equations hold with an oracle query to each of $E_i, \text{dim}_i, \text{read_ts}_i, \text{final_cts}_i$.

Figure 5: Surge’s polynomial IOP for proving that $\sum_{y \in \{0, 1\}^{\log N}} \widetilde{M}(r, y) T[y] = v$.

- Input: A polynomial commitment to the multilinear polynomials $\tilde{a}: \mathbb{F}^{\log m} \rightarrow \mathbb{F}$, and a description of an SOS table T of size N .
- The prover $\mathcal{P}$ sends a **Surge**-commitment to the multilinear extension $\tilde{M}$ of a matrix $M \in \{0, 1\}^{m \times N}$. This consists of c different $(\log(m))$ -variate multilinear polynomials $\text{dim}_1, \dots, \text{dim}_c$ (see Figure 5 for details).
- The verifier $\mathcal{V}$ picks a random $r \in \mathbb{F}^{\log m}$ and sends r to $\mathcal{P}$. The verifier makes one evaluation query to $\tilde{a}$, to learn $\tilde{a}(r)$.
- $\mathcal{P}$ and $\mathcal{V}$ apply **Surge** (Figure 5), allowing $\mathcal{P}$ to prove that $\sum_{y \in \{0, 1\}^{\log N}} \tilde{M}(r, y) T[y] = \tilde{a}(r)$.

Figure 6: Description of the Lasso lookup argument. Here, a denotes the vector of lookups and t the vector capturing the lookup table (Definition 1.1). A polynomial commitments to the multilinear extension polynomial $\tilde{a}: \mathbb{F}^{\log m} \rightarrow \mathbb{F}$ is given to the verifier as input. If t is unstructured, then c will be set to 1.

$O(b \cdot c \cdot m)$ time. For many tables of practical interest, the factor b can be eliminated (e.g., if the *total degree* of g is a constant independent of b , such as 1 or 2). The costs for the prover in the memory checking argument is similar to **Spark**: $O(\alpha \cdot m + \alpha \cdot N^{1/c})$ field operations, plus committing to a low-order number of field elements.

Verification costs. The sum-check protocol used to compute Expression (16) consists of $\log m$ rounds in which the prover sends a univariate polynomial of degree at most $1 + \alpha$ in each round. Hence, the prover sends $O(c \cdot k \cdot \log m)$ field elements, and the verifier performs $O(k \cdot \log m)$ field operations. The costs of the memory checking argument (which can be batched) for the verifier are identical to **Spark**.

Completeness and knowledge soundness of the polynomial IOP. Completeness holds by design and by the completeness of the sum-check protocol, and of the memory checking argument.

By the soundness of the sum-check protocol and the memory checking argument, if the prover passes the verifier’s checks in the polynomial IOP with probability more than an appropriately chosen threshold $\gamma = O(m + N^{1/c}/|\mathbb{F}|)$, then $\sum_{y \in \{0, 1\}^{\log N}} \tilde{M}(r, y) T[y] = v$, where $\tilde{M}$ is the multilinear extension of the following matrix M . For $i \in \{0, 1\}^{\log m}$, row i of M consists of all zeros except for entry $M_{i, j} = 1$, where $j = (j_1, \dots, j_c) \in \{0, 1, \dots, N^{1/c}\}^c$ is the unique column index such that $j_1 = \text{dim}_1(i)$, $\dots$, $j_c = \text{dim}_c(i)$.

We have established the following theorem.

Theorem 3. Figure 5 is a complete and knowledge-sound polynomial IOP for establishing that the prover knows an $m \times N$ matrix $M \in \{0, 1\}^{m \times N}$ with exactly one entry equal to 1 in each row, such that

$$\sum_{y \in \{0, 1\}^{\log N}} \tilde{M}(r, y) T[y] = v. \quad (17)$$

The discussion surrounding Equation (12) explained that checking that $Mt = a$ is equivalent, up to soundness error $\log(m)/|\mathbb{F}|$, to Equation (17) holding for a random $r \in \mathbb{F}^{\log m}$. Combining this with Theorem 3 implies that the protocol in Figure 6, i.e., Lasso, is a lookup argument.

Remark 4. Figure 6 describes an unindexed lookup argument, as for each $i \in \{0, \dots, m-1\}$, it must be the case that $a_i = t_j$ where $j = (j_1, \dots, j_c)$ is defined as in the proof of Theorem 3 above. To obtain an indexed lookup argument (Definition 1.2), one would need to additionally have to check that for each $i \in \{0, \dots, m-1\}$, $b_i = \sum_{i=1}^c (N^{1/c})^{i-1} \cdot j_i$, i.e., that the i ’th (“chunked”) index encoded by M matches the i ’th entry of the committed index vector b .

6 Acknowledgements and disclosures

Acknowledgements. We are grateful to Luís Fernando Schultz Xavier da Silveira for optimizations to an earlier version of *Lasso*. We would also like to thank Luís, Arasu Arun, Patrick Towa for insightful comments and conversations. Justin Thaler was supported in part by NSF CAREER award CCF-1845125 and by DARPA under Agreement No. HR00112020022. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author and do not necessarily reflect the views of the United States Government or DARPA.

Disclosures. Thaler is a Research Partner at a16z crypto and is an investor in various blockchain-based platforms, as well as in the crypto ecosystem more broadly (for general a16z disclosures, see <https://www.a16z.com/disclosures/>.)

References

- [AB09] Sanjeev Arora and Boaz Barak. *Computational complexity: a modern approach*. Cambridge University Press, 2009.
- [BBB⁺18] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2018.
- [BBHR18] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed-Solomon interactive oracle proofs of proximity. In *Proceedings of the International Colloquium on Automata, Languages and Programming (ICALP)*, 2018.
- [BCC⁺16] Jonathan Bootle, Andrea Cerulli, Pyrros Chaidos, Jens Groth, and Christophe Petit. Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. In *Proceedings of the International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT)*, 2016.
- [BCG⁺18] Jonathan Bootle, Andrea Cerulli, Jens Groth, Sune Jakobsen, and Mary Maller. Arya: Nearly linear-time zero-knowledge proofs for correct program execution. In *Proceedings of the International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT)*, 2018.
- [BCHO22] Jonathan Bootle, Alessandro Chiesa, Yuncong Hu, and Michele Orru. Gemini: Elastic snarks for diverse environments. In *Proceedings of the International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT)*, 2022.
- [BDFG20] Dan Boneh, Justin Drake, Ben Fisch, and Ariel Gabizon. Halo Infinite: Recursive zk-SNARKs from any Additive Polynomial Commitment Scheme. Cryptology ePrint Archive, Report 2020/1536, 2020.
- [BEG⁺91] Manuel Blum, Will Evans, Peter Gemmell, Sampath Kannan, and Moni Naor. Checking the correctness of memories. In *Proceedings of the IEEE Symposium on Foundations of Computer Science (FOCS)*, 1991.
- [BFS20] Benedikt Bünz, Ben Fisch, and Alan Szepieniec. Transparent SNARKs from DARK compilers. In *Proceedings of the International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT)*, 2020.
- [BGH19] Sean Bowe, Jack Grigg, and Daira Hopwood. Recursive proof composition without a trusted setup. Cryptology ePrint Archive, Report 2019/1021, 2019.
- [BGH20] Sean Bowe, Jack Grigg, and Daira Hopwood. Halo2, 2020. URL: <https://github.com/zcash/halo2>.
- [BMM⁺21] Benedikt Bünz, Mary Maller, Pratyush Mishra, Nirvan Tyagi, and Psi Vesely. Proofs for inner pairing products and applications. In *Proceedings of the International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT)*, 2021.
- [CBBZ23] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. HyperPlonk: Plonk with linear-time prover and high-degree custom gates. In *Proceedings of the International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT)*, 2023.
- [CDD⁺03] Dwaine Clarke, Srinivas Devadas, Marten Van Dijk, Blaise Gassend, G. Edward, and Suh Mit. Incremental multiset hash functions and their application to memory integrity checking. In *Proceedings of the International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT)*, 2003.
- [CHM⁺20] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Noah Vesely, and Nicholas Ward. Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In *Proceedings*

- of the International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT), 2020.
- [CMT12] Graham Cormode, Michael Mitzenmacher, and Justin Thaler. Practical verified computation with streaming interactive proofs. In *Proceedings of the Innovations in Theoretical Computer Science (ITCS)*, 2012.
 - [CTY11] Graham Cormode, Justin Thaler, and Ke Yi. Verifying computations with streaming interactive proofs. *Proc. VLDB Endow.*, 5(1):25–36, 2011.
 - [DGM21] Justin Drake, Ariel Gabizon, and Izaak Meckler. Checking univariate identities in linear time, 2021. <https://hackmd.io/@arielg/ryGTQXWri>.
 - [EFG22] Liam Eagen, Dario Fiore, and Ariel Gabizon. cq: Cached quotients for fast lookups. *Cryptology ePrint Archive*, 2022.
 - [FS86] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Proceedings of the International Cryptology Conference (CRYPTO)*, pages 186–194, 1986.
 - [GK22] Ariel Gabizon and Dmitry Khovratovich. flookup: Fractional decomposition-based lookups in quasi-linear time independent of table size. *Cryptology ePrint Archive*, 2022.
 - [GLS⁺21] Alexander Golovnev, Jonathan Lee, Srinath Setty, Justin Thaler, and Riad S. Wahby. Brakedown: Linear-time and post-quantum snarks for R1CS. *Cryptology ePrint Archive*, 2021.
 - [GW20a] Ariel Gabizon and Zachary Williamson. Proposal: The TurboPlonk program syntax for specifying SNARK programs, 2020.
 - [GW20b] Ariel Gabizon and Zachary J Williamson. plookup: A simplified polynomial protocol for lookup tables. 2020.
 - [GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. *ePrint Report 2019/953*, 2019.
 - [KST22] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. Nova: Recursive Zero-Knowledge Arguments from Folding Schemes. In *Proceedings of the International Cryptology Conference (CRYPTO)*, 2022.
 - [KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In *Proceedings of the International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT)*, pages 177–194, 2010.
 - [Lee21] Jonathan Lee. Dory: Efficient, transparent arguments for generalised inner products and polynomial commitments. In *Theory of Cryptography Conference*, pages 1–34. Springer, 2021.
 - [LFKN90] Carsten Lund, Lance Fortnow, Howard Karloff, and Noam Nisan. Algebraic methods for interactive proof systems. In *Proceedings of the IEEE Symposium on Foundations of Computer Science (FOCS)*, October 1990.
 - [Pip80] Nicholas Pippenger. On the evaluation of powers and monomials. *SIAM Journal on Computing*, 9(2):230–250, 1980.
 - [PK22] Jim Posen and Assimakis A Kattis. Caulk+: Table-independent lookup arguments. *Cryptology ePrint Archive*, 2022.
 - [PT12] Mihai Pătraşcu and Mikkel Thorup. The power of simple tabulation hashing. *Journal of the ACM (JACM)*, 59(3):1–50, 2012.
 - [PT13] Mihai Pătraşcu and Mikkel Thorup. Twisted tabulation hashing. In *Proceedings of the ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 209–228, 2013.

- [RIS] RISC-V Foundation. The RISC-V instruction set manual, volume I: User-Level ISA, Document Version 20180801-draft. May 2017.
- [SAGL18] Srinath Setty, Sebastian Angel, Trinabh Gupta, and Jonathan Lee. Proving the correct execution of concurrent services in zero-knowledge. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, October 2018.
- [Set20] Srinath Setty. Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In *Proceedings of the International Cryptology Conference (CRYPTO)*, 2020.
- [SL20] Srinath Setty and Jonathan Lee. Quarks: Quadruple-efficient transparent zkSNARKs. Cryptology ePrint Archive, Report 2020/1275, 2020.
- [STW23] Srinath Setty, Justin Thaler, and Riad Wahby. Customizable constraint systems for succinct arguments. Cryptology ePrint Archive, 2023.
- [Tha13] Justin Thaler. Time-optimal interactive proofs for circuit evaluation. In *Proceedings of the International Cryptology Conference (CRYPTO)*, 2013.
- [Tha22] Justin Thaler. Proofs, arguments, and zero-knowledge. *Foundations and Trends in Privacy and Security*, 4(2–4):117–660, 2022.
- [Whi] Barry Whitehat. Lookup singularity. <https://zkresearch.ch/t/lookup-singularity/65/7>.
- [WTS⁺18] Riad S. Wahby, Ioanna Tzialla, Abhi Shelat, Justin Thaler, and Michael Walfish. Doubly-efficient zkSNARKs without trusted setup. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2018.
- [XZS22] Tiancheng Xie, Yupeng Zhang, and Dawn Song. Orion: Zero knowledge proof with linear prover time. In *Proceedings of the International Cryptology Conference (CRYPTO)*, 2022.
- [ZBK⁺22] Arantxa Zapico, Vitalik Buterin, Dmitry Khovratovich, Mary Maller, Anca Nitulescu, and Mark Simkin. Caulk: Lookup arguments in sublinear time. *Cryptology ePrint Archive*, 2022.
- [ZGK⁺22] Arantxa Zapico, Ariel Gabizon, Dmitry Khovratovich, Mary Maller, and Carla Ràfols. Baloo: Nearly optimal lookup arguments. *Cryptology ePrint Archive*, 2022.
- [ZXZS20] Jiaheng Zhang, Tiancheng Xie, Yupeng Zhang, and Dawn Song. Transparent polynomial delegation and its applications to zero knowledge proof. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2020.

A Obtaining an indexed lookup argument from an unindexed one

Let $T \in \mathbb{F}^N$ be a lookup table, and let R be such that all table elements are in $\{0, 1, \dots, R-1\}$, and assume that $m \cdot N$ is less than the field characteristic. Replace each table element $T[i]$ with $i \cdot R + T[i]$ to obtain a modified table $T' \in \mathbb{F}^N$, and replace each lookup pair (b_j, a_j) with the field element $a'_j = b_j \cdot R + a_j$. Apply a range check to confirm that $a_j \in \{0, 1, \dots, R-1\}$ for all lookups. Then $T[b_j] = a_j$ implies that $a'_j = T'[b_j]$. Conversely, under the guarantee that each a_j is in $\{0, \dots, R-1\}$, if $T[b_j] \neq a_j$ then no entry of T' is equal to a'_j . Hence, one can apply a lookup argument for unindexed lookups (Definition 1.1) to a' and T' to confirm that $T[b_j] = a_j$ for all pairs (b_j, a_j) in the list of lookups.

If R is chosen to be the smallest power of 2 bounding all the table elements, then the range check can be implemented via a lookup into the (MLE-structured and decomposable) table $\{0, 1, \dots, R-1\}$, and moreover T' is decomposable or MLE-structured if and only if t is decomposable or MLE-structured. The range check on a_j can be omitted if the value a_j is provided by a party that is guaranteed to be honest.

B Comparison of Lasso's costs to prior lookup arguments

Figure B below compares the costs of Lasso to prior lookup arguments. We clarify that several prior lookup arguments refer to the cost of a general m -sized multiexponentiation as linear in m [ZGK⁺22, DGM21]. However, as discussed in Section 1.2, the fastest known multiexponentiation algorithm, due to Pippenger [Pip80], requires a number of group operations that is (slightly) superlinear in m , namely $O(m\lambda/\log(\lambda m))$, where $\lambda = \Theta(\log |\mathbb{G}|)$ is the security parameter and $\mathbb{G}$ is the group in which the multiexponentiation is occurring. Here, λ must be considered superlogarithmic in m , to ensure that adversaries running in time 2^λ are superpolynomial time. Similarly, prior works [ZGK⁺22, EFG22] refer to group exponentiations as group operations, when in fact they require up to $O(\log |\mathbb{G}|)$ many group operations.

C Details on polynomial commitment schemes

In this section, we give more information about the properties and cost profiles of the polynomial commitment schemes in Figure 2.2. Below, let $n = 2^\ell$, and assume that the prover knows all n evaluations of q over domain $\{0, 1\}^\ell$, i.e., knows $\{(x, q(x)) : x \in \{0, 1\}^\ell\}$.

- KZG + Gemini [BCHO22] transforms the KZG scheme [KZG10], which is designed for univariate polynomials, to provide a polynomial commitment scheme for multilinear polynomials. To commit to q , the prover performs a multiexponentiation each of size n . The commitment size is $O(1)$ group elements. Evaluation proofs are also $O(\log n)$ group elements. To compute the evaluation proof, the prover performs $O(n)$ field operations and a multiexponentiation of size n . Verifying the evaluation proof requires $\log n$ group operations and a few pairings.
- Hyrax [WTS⁺18] is based on the hardness of the discrete logarithm problem. To commit to q , the prover performs $\sqrt{n}$ multiexponentiations each of size $\sqrt{n}$. The commitment size is $\sqrt{n}$ group elements. Evaluation proofs are also $\sqrt{n}$ group elements. To compute the evaluation proof, the prover performs $O(n)$ field operations and a multiexponentiation of size $\sqrt{n}$. Verifying the evaluation proof requires a multiexponentiation of size $\sqrt{n}$.
- Dory [Lee21] requires pairing-friendly groups and is based on the SXDH assumption. Its primary benefit over Bulletproofs is that verifying evaluation proofs can be done with just logarithmically many group operations. In addition, *computing* an evaluation proof requires $O(n)$ field operations and roughly $O(\sqrt{n})$ cryptographic work. Dory does use a transparent pre-processing phase for the verifier that requires $O(\sqrt{n})$ cryptographic work.
- In Brakedown, Orion, and Orion+ [GLS⁺21, XZS22, CBBZ23], evaluation proofs are computed with $O(n)$ field operations and cryptographic hash evaluations. Commitments are just a single hash value. However, Brakedown proof sizes include $O(\sqrt{\lambda n})$ field elements, where λ is the security parameter. The verifier performs $O(\sqrt{\lambda n})$ field operations and also hashes this many field elements. Brakedown

Scheme	Proof size	Prover work group, field	Verifier work
Plookup [GW20b]	$5\mathbb{G}_1, 9\mathbb{F}$	$O(N), O(N \log N)$	$2P$
Halo2 [BGH20]	$6\mathbb{G}_1, 5\mathbb{F}$	$O(N), O(N \log N)$	$2P$
Caulk [ZBK ⁺ 22]	$14\mathbb{G}_1, 1\mathbb{G}_2, 4\mathbb{F}$	$15m, O(m^2 + m \log(N))$	$4P$
Caulk+ [PK22]	$7\mathbb{G}_1, 1\mathbb{G}_2, 2\mathbb{F}$	$8m, O(m^2)$	$3P$
Flookup [GK22]	$7\mathbb{G}_1, 1\mathbb{G}_2, 4\mathbb{F}$	$O(m), O(m \log^2 m)$	$3P$
Baloo [ZGK ⁺ 22]	$12\mathbb{G}_1, 1\mathbb{G}_2, 4\mathbb{F}$	$14m, O(m \log^2 m)$	$5P$
cq [EFG22]	$8\mathbb{G}_1, 3\mathbb{F}$	$7m + o(m), O(m \log m)$	$5P$
Lasso w/ Dory (SOS table)	$O(\log(m)) \mathbb{G}_T$ $\tilde{O}(\log(m)) \mathbb{F}$	$o(cm + cN^{1/c}), O(cm)$ $O(\sqrt{m}) P$	$O(\log(m)) \mathbb{G}_T$ $\tilde{O}(\log(m)) \mathbb{F}$
Lasso w/ Dory (unstructured table)	$O(\log m) \mathbb{G}_T$ $\tilde{O}(\log(m)) \mathbb{F}$	$\min\{2m + O(\sqrt{N}), m + o(N)\}, O(m + N)$ $O(\sqrt{N}) P$	$O(\log m) \mathbb{G}_T$ $\tilde{O}(\log(m)) \mathbb{F}$
Lasso w/ Sona (SOS table)	$\tilde{O}(\log(m)) \mathbb{F}$ $O(1) \mathbb{G}$	$o(cm + cN^{1/c}), O(cm)$	$\tilde{O}(\log(m)) \mathbb{F}$ $O(1) \mathbb{G}$
Lasso w/ Sona (unstructured table)	$\tilde{O}(\log(m)) \mathbb{F}$ $O(1) \mathbb{G}$	$\min\{2m + O(\sqrt{N}), N\}, O(m + N)$	$\tilde{O}(\log(m)) \mathbb{F}$ $O(1) \mathbb{G}$
Lasso w/ KZG+Gemini (SOS table)	$O(\log m) \mathbb{G}_1$ $\tilde{O}(\log(m)) \mathbb{F}$	$(c + 1)m + cN^{1/c}, O(m)$ $O(\log m) \mathbb{G}_1$	$\tilde{O}(\log(m)) \mathbb{F}$ $2P$
Lasso w/ KZG+Gemini (unstructured table)	$O(\log m) \mathbb{G}_1$ $\tilde{O}(\log(m)) \mathbb{F}$	$(c + 1)m + cN^{1/c}, O(m + N)$ $O(\log m) \mathbb{G}_1$	$\tilde{O}(\log(m)) \mathbb{F}$ $2P$

Figure 7: Dominant costs of prior lookup arguments vs. our work. Sona is the polynomial commitment scheme proposed in this work (Section 1.5). Other cost profiles for our schemes are possible by using other polynomial commitments. **Notation:** m is the number of lookups, N is the size of the lookup table. We assume $N \geq m$ for simplicity. For verification costs only, we assume that $m \leq \text{poly}(N)$, so that $\log m = \Theta(\log N)$. The notation $\tilde{O}(\log m)$ notation hides a factor of $\log \log m$. Throughout, m “group work” for the prover refers to a multiexponentiation of size m , while “ m exps” refers to m group exponentiations (m multiexponentiations are subject to a Pippenger speedup of a factor of roughly $O(\log(m\lambda))$ that m exponentiations are not). Operations involving $O(m)$ group operations (not exponentiations) are denoted via “ $o(m)$ group work” to clarify that they are cheaper than a general m -sized multiexponentiation. SOS tables refer to those to which Lasso applies. $\mathbb{F}$ refers to field operations, and $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ to relevant group elements or operations in a pairing-friendly group. P refers to pairing operations. KZG + Gemini refers to a polynomial commitment scheme for multilinear polynomials given in [BCH022] obtained by transforming the KZG scheme for univariate polynomials. Finally, c denotes an arbitrary positive integer. Plookup and Halo2 are agnostic to the choice of polynomial commitment scheme, and that the reported costs and transparency properties in these rows refer to the case of using KZG commitments.

is also *field-agnostic*—it applies to polynomials defined over any sufficiently large field $\mathbb{F}$. Orion reduces verification costs to polylogarithmic via SNARK composition, but is not field-agnostic. Neither Brakedown nor Orion are homomorphic, but they are plausibly post-quantum secure. Orion+ [CBBZ23] reduces the proof size further to logarithmic (concretely under 10 KBs) but gives up transparency and post-quantum security in addition to field-agnosticism.

D Sparse polynomial commitments with logarithmic overhead

This section describes a sparse polynomial commitment scheme that is suboptimal by a logarithmic factor. We include this merely for illustration, because this suboptimal scheme is substantially simpler than Spark (§4).

Notation. For the remainder of this section, let $\tilde{f}$ denote an ℓ -variate multilinear polynomial to be committed, with sparsity m in the Lagrange basis. Let $f: \{0, 1\}^\ell \rightarrow \mathbb{F}$ denote the function with domain equal to the Boolean hypercube that $\tilde{f}$ extends.

In this work, we only apply a sparse polynomial commitment scheme in a setting where (if the prover is honest)

$$f(x) \in \{0, 1\} \text{ for all } x \in \{0, 1\}^\ell. \quad (18)$$

We describe a commitment scheme that applies to multilinear extensions of functions of this form. This slightly simplifies the description of the scheme, and makes the bound on the prover runtime to compute the commitment slightly cleaner.¹⁹

Let $v_f \in \mathbb{F}^{m\ell}$ be the following “densified” description of f . Break v_f into m blocks each of length ℓ . Impose an arbitrary order on the set $S := \{x \in \{0, 1\}^\ell : f(x) \neq 0\}$, and let $x^{(i)}$ denote the i th element in S . Assign the i th block of v to be $x^{(i)} \in \{0, 1\}^\ell$. In other words, v_f simply lists each x such that $f(x) \neq 0$.

The commit phase. To commit to a sparse polynomial $\tilde{f}$, we apply any desired dense polynomial commitment scheme to commit to the multilinear extension $\tilde{v}_f$ of the vector v_f .

Evaluation proofs. As a warm-up, we begin with a conceptually simple high-level sketch of an evaluation proof procedure. We then specify full details of a more direct protocol with similar costs. The direct evaluation proof procedure involves a single application of the sum-check protocol.

Warm-up: a conceptually simple procedure (sketch). To reveal an evaluation $\tilde{f}(r)$ for $r \in \mathbb{F}^\ell$, we apply any sum-check-based SNARK (e.g., Spartan, Brakedown, Orion, Libra, etc.) to the natural arithmetic circuit of size $O(m \cdot \ell)$ that takes as input the densified description v_f of f and outputs $\tilde{f}(r)$. This circuit has a “uniform” wiring pattern that ensures that the verifier in any of these SNARKs will run in polylogarithmic time when applied to this circuit (without any pre-processing), plus the time to check a single evaluation proof from the dense polynomial commitment scheme applied to $\tilde{v}_f$.

Complete description of an evaluation proof procedure via direct application of sum-check. Let us assume that m and ℓ are both powers of 2. If the prover is honest, then

$$\tilde{f}(r) = \sum_{k \in \{0, 1\}^{\log m}} \prod_{j \in \{0, 1\}^{\log \ell}} (\tilde{v}_f(k, j)r_j + (1 - \tilde{v}_f(k, j))(1 - r_j)). \quad (19)$$

To compute Equation (19), we can apply the sum-check protocol to the polynomial g defined below:

$$g(k) = \prod_{j \in \{0, 1\}^{\log \ell}} (\tilde{v}_f(k, j)r_j + (1 - \tilde{v}_f(k, j))(1 - r_j)).$$

Observe that g has $\log m$ variables and degree ℓ in each of them, so the proof length of the sum-check protocol applied to g is $O(\ell \cdot \log m)$ field elements. At the end of the sum-check protocol, the verifier has to evaluate g at a random point $r' \in \mathbb{F}^{\log m}$. This can be done in $O(\ell)$ time given ℓ evaluations of $\tilde{v}_f$, namely $\tilde{v}_f(r', j)$ for each $j \in \{0, 1\}^{\log \ell}$. Standard techniques can efficiently reduce these ℓ evaluations of $\tilde{v}_f$ to a *single* evaluation of $\tilde{v}_f$. Specifically, the prover is asked to send the entire $(\log \ell)$ -variate polynomial $h(y) = \tilde{v}_f(r', y)$, i.e., the polynomial obtained from $\tilde{v}_f$ by fixing the first $\log m$ variables to r' . This costs only $\ell + 1$ field elements in communication. The verifier picks a random point r'' and confirms that $h(r'') = \tilde{v}_f(r', r'')$ with a single evaluation query to the committed polynomial $\tilde{v}_f$. By the Schwartz-Zippel lemma, if $h(y) \neq \tilde{v}_f(r', y)$, then with probability at least $1 - \log(1 + \ell)/|\mathbb{F}|$, the verifier’s check will fail.

Theorem 4. *The above protocol is an extractable polynomial commitment scheme for multilinear polynomials.*

Proof. Suppose that $\mathcal{P}$ is a prover that, with non-negligible probability, produces evaluation proofs that pass verification. By extractability of the dense polynomial commitment scheme used to commit to $\tilde{v}_f$, there is a polynomial time algorithm $\mathcal{E}$ that produces a multilinear polynomial p that explains all of $\mathcal{P}$ ’s evaluation proofs, in the following sense. If $\mathcal{P}$ is able to, with non-negligible probability, produce an evaluation

¹⁹To clarify, the scheme as described in this section does *not* guarantee that the committed polynomial satisfies Equation (18). While it cannot be used by an honest prover to commit to arbitrary polynomials, it can always be used to commit to $\tilde{f}$ if f satisfies Equation (18).

proof for the claim that the committed polynomial polynomial's evaluation at any input $(r', r'') \in \mathbb{F}^{\log m} \times \mathbb{F}^\ell$ equals value $v \in \mathbb{F}$, then $p(r', r'') = v$.

By soundness of the sum-check protocol, if the prover passes the verifier's checks with probability more than $O((\log m + \log(1 + \ell))/|\mathbb{F}|)$ then v equals

$$\sum_{k \in \{0,1\}^{\log m}} \prod_{j \in \{0,1\}^{\log \ell}} (\tilde{v}_f(k, j) r_j + (1 - \tilde{v}_f(k, j))(1 - r_j)).$$

This is a multilinear polynomial in (r', r'') .

□

Costs of the sparse polynomial commitment scheme. There are two sources of costs in the sparse polynomial commitment scheme above.

- One is applying the dense polynomial commitment scheme to commit to the multilinear extension $\tilde{v}$ of a vector $v \in \{0, 1\}^{m\ell}$, and later produce a single evaluation proof for $\tilde{v}(r', r'')$ via this commitment scheme.

Prover costs. If the dense polynomial commitment scheme used is Hyrax [WTS⁺18], Dory [Lee21], or BMMTV [BMM⁺21], the commitment can be computed with only $O(m\ell)$ group operations. Here, we exploit that the entries of v are all in $\{0, 1\}$. Dory and BMMTV require pairing-friendly groups and also require the prover to compute a multi-pairing of length- $O(\sqrt{m\ell})$.

Evaluation proofs for all three commitment schemes require $O(m \log n)$ field operations and roughly $O(\sqrt{m\ell})$ cryptographic work.

Verifier costs. Hyrax's proofs consist of $O(\sqrt{m\ell})$ group elements, and the verifier must perform a multiexponentiation of size $O(\sqrt{m\ell})$. Dory and BMMTV proofs consist of $O(\log(m\ell))$ elements of the target group $\mathbb{G}_t$, and the verifier performs $O(\log(m\ell))$ exponentiations/scalar-multiplications in $\mathbb{G}_t$.

- The other is the costs of the sum-check protocol, and the final step in reducing $1 + \ell$ evaluations of $\tilde{v}_f$ to a single evaluation. The verification costs of these two protocols is $O((\log m) \cdot \log \ell)$ field operations. Meanwhile, via standard techniques, the prover can be implemented with $O(m\ell)$ field operations in total across all rounds of the protocol.

E Additional details on the grand product argument

For completeness of exposition, we provide additional details on the grand product argument we use (Lines 6 and 10 of Figure 3) and its application in our context. Note that these details are identical to prior works [Set20, SL20, GLS⁺21].

Thaler's grand product argument [Tha13, Proposition 2] is simply an optimized application of the GKR interactive proof for circuit evaluation to a circuit computing a binary tree of multiplication gates. The prover in the interactive proof does a number of field operations that is linear in the circuit size, which in the application of the grand product argument in our lookup argument is $O(m)$. The verifier in the GKR protocol has to evaluate the MLE of the input vector to the circuit at a randomly chosen point r . In our applications of the grand product argument, the input to the circuit is either:

$$\begin{aligned} & \{a \cdot \gamma^2 + v \cdot \gamma + t - \tau : (a, v, t) \in \text{WS}\}, \\ & \{a \cdot \gamma^2 + v \cdot \gamma + t - \tau : (a, v, t) \in \text{RS}\} \cup \{a \cdot \gamma^2 + v \cdot \gamma + t - \tau : (a, v, t) \in S\}, \\ & \{a \cdot \gamma^2 + v \cdot \gamma + t - \tau : (a, v, t) \in \text{WS}'\}, \end{aligned}$$

or

$$\{a \cdot \gamma^2 + v \cdot \gamma + t - \tau : (a, v, t) \in \text{RS}'\} \cup \{a \cdot \gamma^2 + v \cdot \gamma + t - \tau : (a, v, t) \in S'\}.$$

For simplicity of notation, let us assume that $N^{1/c} = m$, and let $k = (k_1, \dots, k_{\log m})$ be variables, and let us focus for illustration on the second case above. In this second case above, the multilinear extension of the input to the circuit is

$$g(k_0, k_1, \dots, k_{\log m}) = k_0 \cdot (\gamma^2 \text{row}(k) + \gamma E_{rx}(k) + \text{read_ts}_{\text{row}}(k)) + (1 - k_0) \cdot \left(\gamma^2 \left(\sum_{i=1}^{\log N^{1/c}} 2^{i-1} \cdot k_i \right) + \gamma \cdot \tilde{\text{eq}}(k, r_x) + \text{final_cts}_{\text{row}}(k) \right) - \tau.$$

Indeed, by the definition of RS and S in Claim 2, the expression above is multilinear and agrees with the input to the circuit whenever $(k_0, \dots, k_{\log m}) \in \{0, 1\}^{\log m}$. Here, k_0 acts a selector bit—when $k_0 = 1$ (respectively, $k_0 = 0$), it indicates that $(k_1, \dots, k_{\log m})$ index into the set RS' (respectively, S'). Hence, it must equal the unique multilinear extension of the input. The expression above can be evaluated at any point $(k_0, \dots, k_{\log m}) \in \mathbb{F}^{1+\log m}$ in logarithmic time by the verifier, with one evaluation query to each of row , E_{rx} , $\text{read_ts}_{\text{row}}$ and $\text{final_cts}_{\text{row}}$. A similar expression holds in the other three cases above.

We propose to use Setty and Lee’s grand product argument [SL20, Section 6], which reduces the proof size of Thaler’s to $O(\log(m) \cdot \log \log m)$ at the cost of committing to an additional, say, $m/\log^3(m)$, field elements. The rough idea is that the prover cryptographically commits to the values of the gates at all layers of circuit (the binary-tree of multiplication gates), except for the $O(\log \log m)$ layers closest to the inputs. While the committed gates account for *most of the layers* of the circuit, they account for a tiny fraction of the *gates* in the circuit, as there are only $m/\log^3 m$ gates at these layers. This commitment enables the prover to apply a Spartan-like SNARK to the committed layers, resulting in just logarithmic communication cost (whereas Thaler’s interactive proof applied to those layers would have communication cost $O(\log^2 n)$).

Then Thaler’s protocol is used to handle the $O(\log \log m)$ layers that were not committed. The total communication cost of applying Thaler’s protocol just to these layers is $O(\log(m) \cdot \log \log m)$.

F GeneralizedLasso: Beyond decomposable tables

This section describes how GeneralizedLasso checks Equation (5) to provide a lookup argument. We first provide an overview of the main technical component in GeneralizedLasso that is not present in Lasso.

F.1 Sparse-dense sum-check protocol

Equation (5) can be computed with the sum-check protocol of Lund, Fortnow, Karloff, and Nisan [LFKN90], so long as the verifier can evaluate each of the polynomials in the equation at a random point. The key question for this technical overview is: how fast can the prover be implemented in this application of the sum-check protocol?

The challenge is that this protocol is computing the inner product between two vectors in $u, t \in \mathbb{F}^N$, and we are unsatisfied with a prover time of $O(N)$ field operations. Here, entries of u are indexed by vectors $y \in \{0, 1\}^{\log N}$ and the y ’th entry of u equals $\tilde{M}(r, y)$. Fortunately, we are *guaranteed* that at most m entries of u are non-zero. We leverage this guarantee to show how to implement the prover in this sum-check protocol with only $O(cm)$ field operations where c is such that $N = M^c$, so long as t is structured.

Brief overview of existing linear-time sum-check provers. In each round j of the sum-check protocol, the j th input to the polynomials $\tilde{u}(y_1, \dots, y_{\log N})$ and $\tilde{t}(y_1, \dots, y_{\log N})$ gets “bound” to a random field element r_j of the verifier’s choosing. Existing linear-time sum-check protocols [CTY11, Tha13] applied to compute $\langle u, t \rangle$ achieve a prover that runs in $O(N)$ time by treating two or more entries of u (and of t) as a single entity once all their “bit-differences” are bound. That is, if $y, y' \in \{0, 1\}^{\log N}$ agree on their last ℓ entries, then existing prover implementations treat u_y and $u_{y'}$ as a “single entity” for the final ℓ rounds of the protocol. This ensures that in each round j , the prover only needs to process $N/2^j$ entries, yielding total runtime of $O\left(\sum_{j=1}^{\log N} N/2^j\right) = O(N)$.

Earlier techniques [CMT12] can reduce the prover time instead to $O(m \cdot \text{polylog} N)$. However, achieving $O(m)$ prover time is substantially more challenging.

Brief overview of our sparse-dense sum-check prover. We reduce the prover time to $O(cm)$ as follows. Whereas prior works on the linear-time sum-check provers treat two indices $y, y' \in \{0, 1\}^{\log N}$ of u as a single entity *after their last* “bit-difference” gets bound, our key idea (and fundamentally new technique) is to give a way treat any two indices y, y' as a single entity *until their first* bit-difference gets bound. For example, in round 1, the indices are split into just two entities: those with high-order bit equal to 0 and those with high-order bit equal to 1. In round 2, they are split into four entities, based on their highest-order two bits. And so forth.

This observation lets the prover handle each round $j = 1, \dots, \log m$ in time $O(2^j)$. $\mathcal{P}$ begins to run into a problem once the protocol passes round $\log m$, since then $O(2^j)$ time starts to become larger than the $O(m)$ time bound we wish to satisfy. We think of this phenomenon, of the number of “relevant entities” to be tracked by the prover potentially doubling in each round, as *expansion*.

The idea then is to apply the techniques from the existing linear-time sum-check protocols, spending $O(m)$ time to “make the first $\log m$ bits of the index of each non-zero entry u_y of u no longer relevant to the protocol”, by updating the entries of u to “incorporate” the binding of the first $\log m$ variables of $\tilde{u}$ to the values $r_1, \dots, r_{\log m}$. We call this procedure *consolidation*, as it reduces the number of “relevant entities” tracked by the prover from m down to 1.

The above procedure can be repeated every $\log m$ rounds. That is, for each contiguous “chunk” of input variables to $\tilde{u}$ of size $\log m$, $\mathcal{P}$ spends $O(m)$ field work processing the rounds that bind variables in that chunk. This is $O(c \cdot m)$ field work total if $N = m^c$, i.e., if there are $\log N = c \cdot \log m$ variables.

In the above procedure, there’s a tension whereby the more rounds that go by without consolidating, the more time $\mathcal{P}$ is paying each round. But consolidating costs $O(m)$ work. Hence, $\mathcal{P}$ does not want to consolidate too frequently. The optimal approach is to let expansion occur unchecked for $\log m$ rounds at a time—this balances the cost of consolidating vs. deferring consolidation.

F.2 The GeneralizedLasso protocol

The polynomial IOP. In the polynomial IOP, the prover sends $\tilde{M}$ to the verifier as the first message in the protocol. (In the succinct arguments resulting from this polynomial IOP, the prover will commit to $\tilde{M}$ using our specialized version of Spark, which additionally ensures that each row of M is a unit vector.) Below, we check that $M \cdot t = a$ using the sparse-dense sum-check protocol.

Following the same approach as Surge (Section 5), define $b = M \cdot t$, and let $\tilde{b}$ denote the multilinear extension of b . Then it is easy to see that:

$$\tilde{b}(r) = \sum_{j \in \{0,1\}^{\log N}} \tilde{M}(r, j) \cdot \tilde{t}(j). \quad (20)$$

Indeed, the RHS is a multilinear polynomial in the variables of r , and by the definition of matrix-vector multiplication, it agrees with b at all inputs in $\{0, 1\}^{\log m}$. Hence, the RHS is the unique multilinear polynomial extending b .

Accordingly, to confirm that $M \cdot t = a$, it suffices to confirm that $\tilde{b}$ and $\tilde{a}$ are the same polynomial. To do this, it suffices for the verifier to pick a random input $r \in \mathbb{F}^{\log m}$ and confirm that $\tilde{b}(r) = \tilde{a}(r)$ (up to soundness error $\log(m)/|\mathbb{F}|$, by the Schwartz-Zippel lemma). The verifier can learn $\tilde{a}(r)$ with one evaluation query to $\tilde{a}$.

To learn $\tilde{b}(r)$, the verifier applies the sumcheck protocol to the $(\log N)$ -variate polynomial $g(j) := \tilde{M}(r, j) \cdot \tilde{t}(j)$, in order to compute the right hand side of Equation (20). At the end of the sum-check protocol, the verifier needs to evaluate $\tilde{M}(r, r')$ and $\tilde{t}(r')$ for a randomly chosen point $r' \in \mathbb{F}^{\log N}$. This can be done with one evaluation query to $\tilde{M}$ and one to $\tilde{t}$.

As explained later (Appendix G), $\tilde{M}(r, j) = 0$ for all but at most m values of $j \in \{0, 1\}^{\log N}$. Hence, standard techniques [CMT12] suffice to implement the prover in the application of the sum-check protocol

to $g(j) := \widetilde{M}(r, j) \cdot \widetilde{t}(j)$ with a number of field operations that is *quasilinear* in m . However, we wish to lower this to $O(m)$, especially considering that we would like to apply **GeneralizedLasso** to (MLE-structured) tables so large that $\log N$ is well over one hundred. We call this $O(m)$ -time prover algorithm the *sparse-dense sum-check protocol*. We defer a detailed description of the algorithm to Section [G](#). The consequence of this result is captured in Theorem [5](#).

- Polynomial commitments to the multilinear polynomials $\widetilde{a}: \mathbb{F}^{\log m} \rightarrow \mathbb{F}$ and $\widetilde{t}: \mathbb{F}^{\log N} \rightarrow \mathbb{F}$ are given to the verifier as input. The commitment to $\widetilde{t}$ is omitted if $\widetilde{t}(r)$ can be evaluated at any point $r \in \mathbb{F}^{\log N}$ in logarithmic time.
- The prover $\mathcal{P}$ sends a polynomial commitment to the MLE $\widetilde{M}$ of a matrix $M \in \{0, 1\}^{m \times N}$ using our specialized version of Spartan's sparse polynomial commitment scheme.
- The verifier $\mathcal{V}$ picks a random $r \in \mathbb{F}^{\log m}$ and sends r to $\mathcal{P}$. The verifier makes one evaluation query to $\widetilde{a}$, to learn $\widetilde{a}(r)$.
- $\mathcal{P}$ and $\mathcal{V}$ apply the (sparse-dense) sum-check protocol to the $(\log N)$ -variate polynomial $g(j) := \widetilde{M}(r, j) \cdot \widetilde{t}(j)$, to confirm that

$$\widetilde{a}(r) = \sum_{j \in \{0, 1\}^{\log N}} g(j).$$

- At the end of the sum-check protocol, the verifier needs to evaluate $\widetilde{M}(r, r')$ and $\widetilde{t}(r')$ for a random point $r' \in \mathbb{F}^{\log N}$ that is chosen entry-by-entry over the course of the sum-check protocol. This costs one evaluation query to $\widetilde{M}$ and one to $\widetilde{t}$.

Figure 8: Description of the **GeneralizedLasso** lookup argument. Here, a denotes the vector of lookups and t the vector capturing the lookup table (Definition [1.1](#)). Polynomial commitments to the multilinear extension polynomials $\widetilde{a}: \mathbb{F}^{\log m} \rightarrow \mathbb{F}$ and $\widetilde{t}: \mathbb{F}^{\log N} \rightarrow \mathbb{F}$ are given to the verifier as input (the commitment to $\widetilde{t}$ can be omitted if $\widetilde{t}$ can be evaluated at any point in logarithmic time).

Theorem 5. *There is a polynomial IOP that can be combined with an appropriate commitment scheme for sparse polynomials to obtain a lookup argument for m lookups into a table of size N . The polynomial IOP requires that the characteristic of the field $\mathbb{F}$ over which the lookup argument is defined is at least $\max\{m, N\}$. The polynomial IOP has soundness error at most*

$$(\log m + 2 \log N)/|\mathbb{F}|.$$

The honest prover sends one polynomial $\widetilde{M}$, which is the multilinear extension of a matrix in $\{0, 1\}^{m \times N}$ in which each row is a unit vector. The verifier queries the polynomials $\widetilde{a}$, $\widetilde{t}$, and $\widetilde{M}$ once each, to obtain the values $\widetilde{a}(r)$, $\widetilde{t}(r')$, and $\widetilde{M}(r, r')$. The proof length and verifier time is $O(\log m + \log N)$ field elements and operations respectively, plus the time to query the aforementioned polynomials. The prover time is $O(m)$ field operations if the table satisfies the properties of Theorem [9](#), plus the time to answer the above queries to the polynomials $\widetilde{a}$, $\widetilde{t}$, and $\widetilde{M}$.

Proof. Completeness holds because, if the prover is honest, then $Mt = a$, and the verifier's checks pass with probability 1.

Soundness holds by the following reasoning. If the prover's claim is false, then $Mt \neq a$. By the Schwartz-Zippel lemma, with probability at least $1 - \log(m)/|\mathbb{F}|$, $\widetilde{b}(r) \neq \widetilde{a}(r)$. The sum-check protocol (bulletpoint four of Figure [8](#)) forces the prover to provide $\widetilde{b}(r)$, and it has soundness error $2 \log(N)/|\mathbb{F}|$. By a union bound, the total soundness error is at most

$$\frac{\log(m) + 2 \log(N)}{|\mathbb{F}|}.$$

The prover runtime assertion is immediate from Theorem [9](#), which is stated in proved in Section [G.5.2](#) $\square$

We remark that while the soundness error of the polynomial IOP in Figure 8 is $O(\log(N)/|\mathbb{F}|)$, actual SNARKs derived from the polynomial IOP will use the sparse polynomial commitment scheme of Section 4 (Spark) to commit to $\tilde{M}$. And this sparse polynomial commitment scheme is itself based on a polynomial IOP with soundness error $O(N/|\mathbb{F}|)$. So the resulting lookup argument will need to work over fields of size substantially larger than N to ensure adequate soundness error.

F.3 Details on what is a “structured table” for sparse-dense sum-check

Recall that our *sparse-dense sum-check protocol* exploits structure in the table in two ways: to implement the prover in the sum-check protocol in only $O(m)$ field operations, and to ensure that the verifier in the protocol, on its own, can quickly compute the information it needs about the table. Details follow, starting with how the verifier computes the information about the table that it needs to check the proof.

F.3.1 Ensuring the verifier can quickly compute the information it needs about the table

For many natural lookup tables, the multilinear polynomial $\tilde{t}$ that “captures” the table in our protocol can be directly evaluated by the verifier at any desired point $r \in \mathbb{F}^{\log N}$ in $O(\log N)$ time. We call tables satisfying this property *MLE-structured*. In fact, often the evaluation procedure only involves finite field additions and multiplication by powers of 2, rather than general finite field multiplications.

Hence, the verifier’s work is inexpensive even if $\tilde{t}$ is not cryptographically committed by the prover. In contrast, all prior lookup arguments require the prover to cryptographically commit to some polynomial “encoding” the table, which requires cryptographic costs at least linear in the table size.

Our companion work, Jolt, demonstrates that the evaluation tables of essentially all primitive RISC-V instructions are MLE-structured. For illustration, we mention some specific examples below.

- All integers between 0 and $N - 1$ (this table enables range checks).
- All even (or all odd) integers between 0 and $2N$.
- All integers between 0 and N^2 whose natural binary representations have all even-indexed (or odd-indexed) bits set to 0. This table was used in early work on representing bitwise operations within constraint systems defined over large prime-order fields [BCG⁺18] including bitwise XOR, OR, and AND.

The key technical property that all of the above tables have in common is that the i ’th table entry is a specific linear combination of the individual bits of the binary representation of i . Moreover, lookup tables that are the union of $O(\log N)$ many tables of the form above also fall into the class. That is, for such tables, the polynomial $\tilde{t}$ used in GeneralizedLasso can be evaluated by the verifier itself in $O(\log N)$ time.

For illustration, consider the table consisting of all finite field elements between 0 and $N - 1$. If we index the table entries by $i \in \{0, 1\}^{\log N}$, then the i ’th table element is simply the field element $\sum_{j=0}^{\log(N)-1} 2^i \cdot i_j$. As we explain later, this means that for arbitrary finite field elements $(r_1, \dots, r_{\log N}) \in \mathbb{F}$,

$$\tilde{t}(r_1, \dots, r_{\log n}) = \sum_{j=0}^{\log N} 2^i \cdot r_j. \quad (21)$$

Clearly, Equation (21) can be evaluated in $O(\log N)$ time, and in fact only requires multiplications-by-powers-of-two and finite field additions.

Two more examples. An illustrative example that is somewhat more complicated than any of the above, is a lookup table introduced in our companion paper, Jolt, to handle bitwise operations more efficiently than prior works. For bitwise AND over b -bit inputs $x, y \in \{0, 1\}^b$, the (x, y) ’th entry of the appropriate (ordered)

table is $\sum_{i=1}^b 2^{i-1} \cdot x_i \cdot y_i$, implying that

$$\tilde{t}(x, y) = \sum_{i=1}^b 2^{i-1} \cdot x_i \cdot y_i. \quad (22)$$

To derive the above expression for the table entries, observe that for any two bits $a, b \in \{0, 1\}$, $\text{AND}(a, b) = a \cdot b$, and then take the appropriate weighted sum of the bitwise AND of x and y to transform it into the associated integer. The result is that any $(x, y) \in \{0, 1\}^b \times \{0, 1\}^b$ gets mapped to the field element with binary representation equal to the bitwise AND of x and y .

Observe that the (x, y) 'th entry of the evaluation table for the bitwise AND operation is *not* a weighted sum of the bits of x and y , because the function has total degree 2. Nonetheless, the multilinear extension $\tilde{t}$ of this table can be evaluated by the verifier in logarithmic time, and the prover in the sparse-dense sum-check protocol applied to this table (Theorem 9) runs in $O(m)$ time.

As a final, more complicated example, Jolt also considers the lookup table associated with the integer comparison instruction LT (short for “less than”). This instruction takes two 64-bit inputs x and y , interprets them as (say, unsigned) integers, and outputs 1 if and only if $x \geq y$. The appropriate lookup table has (x, y) 'th entry equal to the output of the LT instruction when run on x and y . As with the table AND above, we show that the multilinear extension of this table can be evaluated by the verifier in logarithmic time, and the prover in the sparse-dense sum-check protocol applied to this table (Theorem 9) runs in $O(cm)$ time when the table size is at most $O(m^c)$.

F.3.2 Ensuring the sum-check prover runs in time close to m

Depending on just how “structured” the table y is, we prove two results regarding how fast the prover can be implemented in the sparse-dense sum-check protocol. First, using prior techniques [CMT12], we bound the prover time in the sparse-dense sum-check protocol by

$$O(m \cdot \log N \cdot \text{evaltime}(\tilde{t})),$$

where $\text{evaltime}(\tilde{t})$ denotes the time required to evaluate $\tilde{t}(r')$ for any point $r' \in \mathbb{F}^{\log N}$. In particular, if $\text{evaltime}(\tilde{t}) = O(\log N)$, then this means $O(m \cdot \log^2 N)$ field operations for the prover.

Theorem 6. *The prover in the sparse-dense sum-check protocol applied to compute $\langle u, t \rangle$ can be implemented in $O(m \cdot \log N \cdot \text{evaltime}(\tilde{t}))$ field operations.*

Using the same techniques, we are in fact able to reduce the prover runtime for all tables considered in Section F.3.1 to $O(m \log N)$ field operations (see Section G.4).

Reducing the prover’s runtime to the optimal $O(cm)$ field operations is a substantially more challenging task. Intuitively, this requires the prover to spend a *constant* amount of work per non-zero entry of u , in total across *all* $\log N$ rounds of the protocol. This is a far more exacting task than achieving a constant amount of work per non-zero entry of u in *each* of the $\log N$ rounds, which is what the previous paragraph achieved.²⁰

Hence, fundamentally new algorithmic techniques (outlined in Section F.1) are required to reduce the prover’s runtime to the optimal $O(m)$ field operations. We achieve this in Theorem 7 below for a large class of tables that captures all of those considered in Section F.3.1. Informally, the key property that we require to achieve $O(m)$ prover time is that, given any evaluation $\tilde{t}(r_1, \dots, r_{\log N})$ of $\tilde{t}$, changing the value of one variable from r_j to r'_j has a “simple” effect on the evaluation of $\tilde{t}$.

Theorem 7 (Informal version of Theorem 9 in Section G.5.2). *Suppose there is some constant $C > 0$ such that $m \leq O(N^C)$. Suppose that $\tilde{t}: \mathbb{F}^{\log N} \rightarrow \mathbb{F}$ satisfies the following property. For any $(r_1, \dots, r_{\log N}) \in \mathbb{F}^{\log N}$ and any $r' \in \mathbb{F}^{\log N}$,*

$$\tilde{t}(r_1, \dots, r_{j-1}, r'_j, r_{j+1}, \dots, r_{\log N}) = m \cdot \tilde{t}(r_1, \dots, r_{j-1}, r_j, r_{j+1}, \dots, r_{\log N}) + a$$

²⁰Because cryptographic operations are one or more orders of magnitude more expensive than field operations, we believe that even $O(m \log N)$ field work would not be a bottleneck for the GeneralizedLasso prover, compared to the work of committing to $O(m)$ field elements, unless $\log N$ is in the thousands or larger.

where $\mathbf{m}$ and $\mathbf{a}$ are field elements that depend only on $j, r_1, \dots, r_j, r_{j+1}$, and r'_j and can be computed in $O(1)$ time. Then the sparse-dense sum-check protocol prover can be implemented in $O(m)$ field operations.

In **GeneralizedLasso**, the $O(Cm)$ field operations incurred by the prover in the sparse-dense sum-check protocol will not be a bottleneck for the prover relative to the task of applying a sparse polynomial commitment scheme to commit to $O(cm)$ field elements (Section 1.1).

When the sparse-dense sum-check protocol is applied to compute $\langle u, t \rangle$ where u is m -sparse, the prover ultimately has to provide the verifier with the value $\tilde{u}(r)$ for a randomly chosen $r \in \mathbb{F}^{\log N}$, where $\tilde{u}$ is the multilinear extension polynomial of u . The fastest known algorithm for evaluating $\tilde{u}(r)$ requires $O(Cm)$ field work. In fact this algorithm underlies **Spark**, our sparse polynomial commitment scheme (it is described in Section 3.1). Hence, if this algorithm for evaluating sparse multilinear polynomials is optimal, then so is the $O(Cm)$ -time algorithm of implementing the sparse-dense sum-check prover.

F.4 Beyond multilinear extensions, and a generic speedup over bit-decomposition

In our lookup argument (Figure 8) there is nothing special about using the *multilinear* extension $\tilde{t}$ of t . We can replace $\tilde{t}$ with *any* extension polynomial $\hat{t}$ of t (recall from Section 2.1 that $\hat{t}$ extends t if $\hat{t}(i) = t(i)$ for all $i \in \{0, 1\}^{\log N}$).

Indeed, the key equality (Equation 20) that for $b = M \cdot t$ that

$$\tilde{b}(r) = \sum_{j \in \{0,1\}^{\log N}} \tilde{M}(r, j) \cdot \tilde{t}(j).$$

holds with $\tilde{t}$ replaced by *any* extension $\hat{t}$ of t . This is because the right hand side of the equation is multilinear in r regardless of whether or not $\tilde{t}(j)$ is multilinear in j .

This means that if the multilinear extension $\tilde{t}$ of t cannot be evaluated by the verifier sufficiently quickly (say, in polylogarithmic time), we can replace $\tilde{t}$ with another extension $\hat{t}$ that can be. This does potentially increase the costs of the sum-check protocol applied to compute

$$\sum_{j \in \{0,1\}^{\log N}} \tilde{M}(r, j) \cdot \tilde{t}(j)$$

(see Figure 9). In particular, the length of each message j from prover to verifier across the $\log N$ rounds of the sum-check protocol grows from 3 field elements (specifying a degree-2 univariate polynomial) to $1 + d_j$ where d_j is the degree of $\hat{t}$ in its j 'th variable. Assuming that $d_j \leq \text{polylog}(N)$ for each variable $j = 1, \dots, \log N$, and applying standard techniques to implement the sum-check protocol prover [CMT12], we obtain a prover performing $O(m \cdot \text{polylog}(N))$ field operations.

This result can be viewed as formalizing the following intuitive statement: any operation that can be “efficiently performed via bit-decomposition” (meaning there is an arithmetic or Boolean formula of size polynomial in the number of bits in the bit-decomposition that outputs the result of the operation) can be solved by **GeneralizedLasso** with $\mathcal{P}$ only needing to cryptographically commit to $3c$ many field elements per lookup ($\mathcal{P}$ also performs polylogarithmic field operations per lookup). In contrast, as explained in Section 1, naive bit-decomposition of integers in the range $\{0, 1, \dots, R-1\}$ requires the prover to commit to $\log R$ many field elements.

G Details of the sparse-dense sum-check protocol

Let $u \in \mathbb{F}^N$ be a vector with at most m non-zero entries and $t \in \mathbb{F}^N$ be another vector (which may have N non-zero entries). Let $\tilde{u}$ and $\tilde{t}$ denote their multilinear extensions. Throughout, we assume that there is some constant $C > 0$ such that $N \leq m^C$.

- Polynomial commitment to the multilinear polynomial $\tilde{a}: \mathbb{F}^{\log m} \rightarrow \mathbb{F}$.
- The prover $\mathcal{P}$ sends a polynomial commitment to the MLE $\tilde{M}$ of a matrix $M \in \{0, 1\}^{m \times N}$ using our specialized version of Spartan's sparse polynomial commitment scheme.
- The verifier $\mathcal{V}$ picks a random $r \in \mathbb{F}^{\log m}$ and sends r to $\mathcal{P}$. The verifier makes one evaluation query to $\tilde{a}$, to learn $\tilde{a}(r)$.
- $\mathcal{P}$ and $\mathcal{V}$ apply the sum-check protocol to the $(\log N)$ -variate polynomial $g(j) := \tilde{M}(r, j) \cdot \hat{t}(j)$, to confirm that

$$\tilde{a}(r) = \sum_{j \in \{0, 1\}^{\log N}} g(j).$$

- At the end of the sum-check protocol, the verifier needs to evaluate $\tilde{M}(r, r')$ and $\hat{t}(r')$ for a random point $r' \in \mathbb{F}^{\log N}$ that is chosen entry-by-entry over the course of the sum-check protocol. This costs one evaluation query to $\tilde{M}$ and one to $\hat{t}$.

Figure 9: Description of **GeneralizedLasso** when using an extension polynomial $\hat{t}$ of the table vector t , where $\hat{t}$ may not be multilinear. A polynomial commitment to the multilinear polynomial $\tilde{a}: \mathbb{F}^{\log m} \rightarrow \mathbb{F}$ is given to the verifier as input.

G.1 Establishing that for any $r \in \mathbb{F}^{\log m}$, $\tilde{M}(r, y)$ is m -sparse

In **GeneralizedLasso**, $t \in \mathbb{F}^N$ will be the table (Definition 1.1), while $\tilde{u}$ will be $\tilde{M}(r, y)$. If the prover is honest, each row of M has exactly one non-zero entry, and accordingly $\tilde{M}(r, y) \neq 0$ for at most m values of $y \in \{0, 1\}^{\log N}$ by the following reasoning. Let

$$\chi_i(r) = \prod_{k=1}^{\log m} (r_k i_k + (1 - r_k)(1 - i_k))$$

denote the i 'th Lagrange basis polynomial, which maps i to 1 and maps all other inputs in $\{0, 1\}^{\log m}$ to zero. Standard Lagrange interpolation for multilinear polynomials (see [Cha22, Chapter 3]) states that

$$\tilde{M}(r, y) = \sum_{(i, j) \in \{0, 1\}^{\log m + \log N}} M_{i, j} \cdot \chi_i(r) \cdot \chi_j(y). \quad (23)$$

Since for any i , $M_{i, j} \neq 0$ for exactly one j , the right hand side of Equation (23) can be non-zero for at most m values of $y \in \{0, 1\}^{\log N}$, namely those y 's indexing columns of M with at least one non-zero entry.

G.2 Background on the sum-check protocol

For each round $j = 1, \dots, \log N$ of the sum-check protocol, the prescribed prover message is the degree-2 univariate polynomial s_j where

$$s_j(c) = \sum_{(b_{j+1}, b_{j+2}, \dots, b_{\log N}) \in \{0, 1\}^{\log(N)-j}} \tilde{u}(r_1, \dots, r_{j-1}, c, b_{j+1}, \dots, b_{\log N}) \cdot \tilde{t}(r_1, \dots, r_{j-1}, c, b_{j+1}, \dots, b_{\log N}). \quad (24)$$

Here, $r_1, \dots, r_{j-1}$ are random field elements chosen by the verifier in rounds $1, 2, \dots, j-1$. The prover will specify s_j by sending its evaluations at 3 inputs, say, $s_j(0)$, $s_j(1)$, and $s_j(-1)$.

G.3 Proof of Theorem 6

Proof of Theorem 6. Observe that

$$\tilde{u}(r_1, b_2, \dots, b_{\log N}) = (1 - r_1) \cdot \tilde{u}(0, b_2, \dots, b_{\log N}) + r_1 \cdot \tilde{u}(1, b_2, \dots, b_{\log N})$$

This holds because the left hand side and right hand sides are both multilinear polynomials that agree on all inputs $(r_1, b_2, \dots, b_{\log N}) \in \{0, 1\}^{\log N}$.

Let $S_u = \{i = (i_1, \dots, i_{\log N}) \in \{0, 1\}^{\log N} : u_i \neq 0\}$ denote the non-zero entries of u . For every $i \in S_u$, and every round j of the sum-check protocol, $\mathcal{P}$ will store the value

$$\chi_i(r_1, \dots, r_{j-1}, i_j, i_{j+1}, \dots, i_{\log N}) = \prod_{k=1}^{j-1} (i_k \cdot r_k + (1 - i_k) \cdot (1 - r_k)). \quad (25)$$

Note that given all such values for round j , the prover can compute all the relevant values for round $j+1$ in time $O(m)$. This is because there are m elements in $i \in S_u$ and computing $\chi_i(r_1, \dots, r_{j-1}, r_j, i_{j+1}, \dots, i_{\log N})$ given $\chi_i(r_1, \dots, r_{j-1}, i_j, i_{j+1}, \dots, i_{\log N})$ can be done with just one field multiplication. Hence, maintaining all such values across all rounds j takes time $O(m \log N)$ in total for the prover.

By standard multilinear Lagrange interpolation (see [Tha22, Chapter 3]),

$$\tilde{u}(r_1, \dots, r_{\log N}) = \sum_{i \in S_u} u_i \cdot \chi_i(r_1, \dots, r_{\log N}), \quad (26)$$

where u_i denotes the i 'th entry of u when its entries are indexed by bit-vectors $\{0, 1\}^{\log N}$. Hence, for any $c \in \mathbb{F}$,

$$\begin{aligned} s_j(c) &= \sum_{(b_{j+1}, b_{j+2}, \dots, b_{\log N}) \in \{0, 1\}^{\log(N)-j}} \tilde{u}(r_1, \dots, r_{j-1}, c, b_{j+1}, \dots, b_{\log N}) \cdot \tilde{t}(r_1, \dots, r_{j-1}, c, b_{j+1}, \dots, b_{\log N}) \\ &= \sum_{(b_{j+1}, b_{j+2}, \dots, b_{\log N}) \in \{0, 1\}^{\log(N)-j}} \left(\sum_{i \in S_u} u_i \cdot \chi_i(r_1, \dots, r_{j-1}, c, b_{j+1}, \dots, b_{\log N}) \right) \cdot \tilde{t}(r_1, \dots, r_{j-1}, c, b_{j+1}, \dots, b_{\log N}) \\ &= \sum_{i \in S_u} u_i \cdot \chi_i(r_1, \dots, r_{j-1}, c, i_{j+1}, \dots, i_{\log N}) \cdot \tilde{t}(r_1, \dots, r_{j-1}, c, i_{j+1}, \dots, i_{\log N}). \end{aligned} \quad (27)$$

The final equality above exploits the fact that for any $(b_{j+1}, \dots, b_{\log N}) \in \{0, 1\}^{\log(N)-j}$, if

$$(i_{j+1}, \dots, i_{\log N}) \neq (b_{j+1}, \dots, b_{\log N}),$$

then

$$\chi_i(r_1, \dots, r_{j-1}, c, b_{j+1}, \dots, b_{\log N}) = 0.$$

To see this, observe that

$$\chi_i(x_1, \dots, x_{\log N}) = \prod_{k=1}^{\log N} (i_k x_k + (1 - i_k)(1 - x_k)),$$

and if $i_k = 1$ and $x_k = 0$ or vice versa then the k 'th term of this product is zero.

So to compute $s_j(c)$ for $c \in \{0, 1, -1\}$, the prover directly computes Expression (27). Given that the prover maintains in each round j the values in Expression (25), in round j computing $s_j(c)$ takes time

$$O(m \cdot \text{evaltime}(\tilde{t})).$$

Across all $\log N$ rounds of the sum-check protocol, this entails total prover time

$$O(m \cdot \text{evaltime}(\tilde{t})).$$

□

G.4 Improving the runtime to $O(m \log N)$ if $\tilde{t}$ has additional structure

In the proof of Theorem 6, the prover time is bottlenecked by evaluating $\tilde{t}$ at all points of the form

$$(r_1, \dots, r_{j-1}, c, i_{j+1}, \dots, i_{\log N}),$$

where $i = (i_1, \dots, i_{\log N})$ ranges over $i \in S_u$ and $c \in \{0, 1, -1\}$. For all the example tables in Section F.3.1 given

$$\tilde{t}(r_1, \dots, r_{j-1}, i_j, i_{j+1}, \dots, i_{\log N}),$$

it takes only constant time (in most cases, just one multiplication by a power of two and one field addition) to compute

$$\tilde{t}(r_1, \dots, r_{j-1}, r_j, i_{j+1}, \dots, i_{\log N}).$$

This reduces the prover time for all such tables from $O(m \log^2 N)$ of Theorem 6 down to $O(m \log N)$.

G.5 Improving the runtime to $O(cm)$

Fundamentally different techniques are required to reduce the prover's runtime to $O(m)$.

We begin by explaining how to implement the prover in $O(m)$ time under the assumption that $\tilde{t}$ has total degree one, i.e.,

$$\tilde{t}(r_1, \dots, r_{\log N}) = \sum_{j=1}^{\log N} d_j r_j.$$

This is sufficient to capture most of the example tables considered in Section F.3.1. Later (Section G.5.2) we explain how to implement the prover in $O(m)$ time for a larger class of tables.

G.5.1 Handling tables for which $\tilde{t}$ has total degree 1

Theorem 8. *Suppose that the multilinear extension $\tilde{t}$ of t has total degree 1, i.e., can be written as $\tilde{t}(r_1, \dots, r_{\log N}) = \sum_{k=1}^{\log N} d_k \cdot r_k$ for some field elements $d_1, \dots, d_{\log N} \in \mathbb{F}$. Then the prover in the sparse-dense sum-check protocol can be implemented in $O(m)$ field operations.²¹*

Proof. We begin by showing that $\tilde{t}$ having total degree 1 ensures that altering the value of a single variable leads to “simple” changes in the output of $\tilde{t}$.

Understanding the effect on an evaluation of $\tilde{t}$ of altering the j 'th variable. Let

$$\tilde{t}(r_1, \dots, r_{\log N}) = \sum_{j=1}^{\log N} d_j r_j.$$

This ensures that for any $c \in \mathbb{F}$, and any $(r_1, \dots, r_{j-1}, b_j, b_{j+1}, \dots, b_{\log N}) \in \mathbb{F}^j \times \{0, 1\}^{\log(N)-j}$,

$$\tilde{t}(r_1, \dots, r_{j-1}, c, b_{j+1}, \dots, b_{\log N}) = \tilde{t}(r_1, \dots, r_{j-1}, b_j, b_{j+1}, \dots, b_{\log N}) + (c - b_j) \cdot d_j. \quad (28)$$

The important implication of Equation (28) is that “revising” the j 'th variable value from b_j to c affects the evaluation of $\tilde{t}$ by an additive term $(c - b_j) \cdot d_j$. The key here is that this term can be computed in $O(1)$ time, and does *not* depend on the variables $b_{\log(m)+1}, \dots, b_{\log N}$.

In Section G.5.2, we extend our techniques to achieve $O(m)$ prover time whenever $\tilde{t}$ can be decomposed into a sum of $\eta = O(1)$ polynomials $\tilde{t}_1, \dots, \tilde{t}_\eta$ such that the following holds. There exist values $\mathbf{a}_\ell(c, j, b_j)$, $\mathbf{m}_\ell(c, j, b_j)$ such that “revising” the j 'th variable from b_j to c affects the evaluation of $\tilde{t}_\ell$ by a multiplicative factor of $\mathbf{m}_\ell(c, j, b_j)$ and an additive factor of $\mathbf{a}_\ell(c, j, b_j)$. Moreover, these factors $\mathbf{a}_\ell(c, j, b_j)$ and $\mathbf{m}_\ell(c, j, b_j)$ can be

²¹To clarify, Theorem 8 also assumes that for each index b , the b 'th table entry, t_b , can be computed in constant time (otherwise, it is impossible to even compute the correct answer $\langle u, t \rangle$ in the sparse-dense sum-check protocol in $O(m)$ time).

evaluated in $O(1)$ time and do *not* depend on the variables $b_{\log(m)+1}, \dots, b_{\log N}$. The special case of $\tilde{t}$ having total degree 1, which we consider first, corresponds to $\eta = 1$, $\mathbf{a}_\ell(c, j, b_j) = (c - b_j) \cdot d_j$ and $\mathbf{m}_\ell(c, j, b_j) = 1$.

Values computed by the prover at the start of the protocol. For every $k \in \{0, 1\}^{\log m}$, let $\text{extend}_\ell(k)$ denote the set of all vectors in $\{0, 1\}^\ell$ whose first $\log m$ entries equal k . At the start of the protocol, the prover computes the following two values q_k and z_k for every $k \in \{0, 1\}^{\log m}$:

$$q_k := \sum_{y \in \text{extend}_{\log N}(k)} \tilde{u}(y) \cdot \tilde{t}(y) \quad (29)$$

$$z_k := \sum_{y \in \text{extend}_{\log N}(k)} \tilde{u}(y). \quad (30)$$

Since u is m -sparse, all m quantities q_k and z_k can be computed in $O(m)$ total time.

The prover also computes “a binary tree of aggregations” of the above values. Specifically, let us think of the m different q_k (respectively, z_k) values as the roots of a binary tree Q (respectively, Z), and assign each node in Q and Z the value equal to the sum of its leaves. These values can be computed in $O(m)$ time in total.

For example, the roots of Q and Z respectively store

$$\sum_{k \in \{0, 1\}^{\log m}} q_k = \sum_{y \in \{0, 1\}^{\log N}} \tilde{u}(y) \cdot \tilde{t}(y),$$

and

$$\sum_{k \in \{0, 1\}^{\log m}} z_k = \sum_{y \in \{0, 1\}^{\log N}} \tilde{u}(y).$$

Likewise, the two children of the root in Q store:

$$\sum_{k=(k_1, \dots, k_{\log m}) \in \{0, 1\}^{\log m} : k_1=0} q_k, \quad (31)$$

and

$$\sum_{k=(k_1, \dots, k_{\log m}) \in \{0, 1\}^{\log m} : k_1=1} q_k, \quad (32)$$

and similarly for Z . In general, let $Q^{(j)} \in \mathbb{F}^{2^j} \in \mathbb{F}^{2^j}$ is the vector of values assigned to nodes at at depth j of Q , and similarly let $Z^{(j)}$ denote the corresponding vector of values for Z . For example, $Q^{(1)}$ is the length-2 vector whose two entries are given in Equations (31) and (32).

The prover’s workflow in the first $\log m$ rounds. Recall from Equation (24) that

$$s_j(c) = \sum_{(b_{j+1}, b_{j+2}, \dots, b_{\log N}) \in \{0, 1\}^{\log(N)-j}} \tilde{u}(r_1, \dots, r_{j-1}, c, b_{j+1}, \dots, b_{\log N}) \cdot \tilde{t}(r_1, \dots, r_{j-1}, c, b_{j+1}, \dots, b_{\log N}).$$

And recall from Equation (26) that

$$\tilde{u}(r_1, \dots, r_{\log N}) = \sum_{i \in S_u} u_i \cdot \chi_i(r_1, \dots, r_{\log N}).$$

Moreover, for any $i = (i_1, \dots, i_{\log N}) \in \{0, 1\}^{\log N}$, if $i_j = 1$, then

$$\chi_i(r_1, \dots, r_{j-1}, c, i_{j+1}, \dots, i_{\log N}) = c \cdot \chi_i(r_1, \dots, r_{j-1}, i_j, i_{j+1}, \dots, i_{\log N}),$$

and if $i_j = 0$ then

$$\chi_i(r_1, \dots, r_{j-1}, c, i_{j+1}, \dots, i_{\log N}) = (1 - c) \cdot \chi_i(r_1, \dots, r_{j-1}, i_j, i_{j+1}, \dots, i_{\log N}).$$

Based on the above equations, it can be derived that

$$\begin{aligned}
s_1(c) &= \sum_{y \in \{0,1\}^{\log N}} \tilde{u}(c, y_2, \dots, y_{\log N}) \cdot \tilde{t}(c, y_2, \dots, y_{\log N}) \\
&= \sum_{y \in \{0,1\}^{\log N}} \left(\sum_{i=(i_1, \dots, i_{\log N}) \in S_u} u_i \cdot \chi_i(c, y_2, \dots, y_{\log N}) \right) \cdot \tilde{t}(c, y_2, \dots, y_{\log N}) \tag{33}
\end{aligned}$$

$$= \sum_{i=(i_1, \dots, i_{\log N}) \in S_u} u_i \cdot \chi_i(c, i_2, \dots, i_{\log N}) \cdot \tilde{t}(c, i_2, \dots, i_{\log N}) \tag{34}$$

$$= \sum_{i=(i_1, \dots, i_{\log N}) \in S_u} \chi_{i_1}(c) \cdot u_i \cdot \chi_i(i_1, i_2, \dots, i_{\log N}) \cdot (\tilde{t}(i_1, i_2, \dots, i_{\log N}) + (c - i_1) d_1) \tag{35}$$

$$= \sum_{i=(i_1, \dots, i_{\log N}) \in S_u : i_1=0} (1 - c) \cdot u_i \cdot (\tilde{t}(i_1, i_2, \dots, i_{\log N}) + c d_1) \tag{36}$$

$$+ \sum_{i=(i_1, \dots, i_{\log N}) \in S_u : i_1=1} c \cdot u_i \cdot (\tilde{t}(i_1, i_2, \dots, i_{\log N}) + (c - 1) d_1). \tag{37}$$

Here, Equation (33) invokes Equation (26). Equation (34) holds because $\chi_i(j) = 0$ whenever there is even a single index $\ell \in 2, 3, \dots, \log N$ such that $i_\ell, j_\ell \in \{0, 1\}$ and $i_\ell \neq j_\ell$. Equation (35) holds by Equation (28). Equation (36) holds because $\chi_i(i) = 1$ for all $i \in \{0, 1\}^{\log N}$. Equation (37) holds by definition of χ_{i_1} (Equation (3)).

Round 1 computation. Expression (37) above equals:

$$\begin{aligned}
&\sum_{k=(k_1, \dots, k_{\log m}) \in \{0,1\}^{\log m} : k_1=0} ((1 - c) \cdot q_k + (1 - c) \cdot c \cdot d_1 \cdot z_k) \\
&+ \sum_{k=(k_1, \dots, k_{\log m}) \in \{0,1\}^{\log m} : k_1=1} (c \cdot q_k + c \cdot (c - 1) \cdot d_1 \cdot z_k)
\end{aligned}$$

For each $c \in \{-1, 0, 1\}$, computing this expression requires just a constant amount of work given the values of the two children of the root vertex of the trees Q (Equations (31) and (32)) and Z .

Round $j > 1$ computation. A similar calculation to the above reveals that $s_j(c)$ equals:

$$\begin{aligned}
& \sum_{(y_{j+1}, \dots, y_{\log N}) \in \{0,1\}^{\log(N)-j}} \tilde{u}(r_1, \dots, r_{j-1}, c, y_{j+1}, \dots, y_{\log N}) \cdot \tilde{t}(r_1, \dots, r_{j-1}, c, y_{j+1}, \dots, y_{\log N}) \\
&= \sum_{(y_{j+1}, \dots, y_{\log N}) \in \{0,1\}^{\log(N)-j}} \left(\sum_{i=(i_1, \dots, i_{\log N}) \in S_u} u_i \cdot \chi_i(r_1, \dots, r_{j-1}, c, y_{j+1}, \dots, y_{\log N}) \right) \cdot \tilde{t}(r_1, \dots, r_{j-1}, c, y_{j+1}, \dots, y_{\log N}) \\
&= \sum_{i=(i_1, \dots, i_{\log N}) \in S_u} u_i \cdot \chi_i(r_1, \dots, r_{j-1}, c, i_{j+1}, \dots, i_{\log N}) \cdot \tilde{t}(r_1, \dots, r_{j-1}, c, i_{j+1}, \dots, i_{\log N}) \\
&= \sum_{i=(i_1, \dots, i_{\log N}) \in S_u} \chi_{(i_1, \dots, i_j)}(r_1, \dots, r_{j-1}, c) \cdot u_i \cdot \left(\tilde{t}(i_1, i_2, \dots, i_{\log N}) + \sum_{k=1}^{j-1} (r_k - i_k) d_k + (c - i_j) d_j \right) \\
&= \sum_{(b_1, \dots, b_j) \in \{0,1\}^j} \sum_{i=(i_1, \dots, i_{\log N}) \in S_u : (i_1, \dots, i_j) = (b_1, \dots, b_j)} u_i \cdot \chi_{(b_1, \dots, b_j)}(r_1, \dots, r_{j-1}, c) \cdot \left(\tilde{t}(i_1, i_2, \dots, i_{\log N}) + \sum_{k=1}^{j-1} (r_k - i_k) d_k + (c - i_j) d_j \right) \\
&= \sum_{(b_1, \dots, b_j) \in \{0,1\}^j} \sum_{i=(i_1, \dots, i_{\log N}) \in S_u : (i_1, \dots, i_j) = (b_1, \dots, b_j)} u_i \cdot \chi_{(b_1, \dots, b_j)}(r_1, \dots, r_{j-1}, c) \cdot \tilde{t}(i_1, i_2, \dots, i_{\log N}) \quad (38) \\
&+ \sum_{(b_1, \dots, b_j) \in \{0,1\}^j} \sum_{i=(i_1, \dots, i_{\log N}) \in S_u : (i_1, \dots, i_j) = (b_1, \dots, b_j)} u_i \cdot \chi_{(b_1, \dots, b_j)}(r_1, \dots, r_{j-1}, c) \cdot \left(\sum_{k=1}^{j-1} (r_k - i_k) d_k + (c - i_j) d_j \right) \quad (39)
\end{aligned}$$

Recall that $Q^{(j)} \in \mathbb{F}^{2^j}$ (respectively $Z^{(j)}$) is the vector of values assigned to nodes at depth j of Q . Let $v^{(j)}$ be the length- 2^j vector with entries indexed by $(b_1, \dots, b_j) \in \{0,1\}^j$, with $(b_1, \dots, b_j)$ 'th entry given by

$$\chi_{(b_1, \dots, b_j)}(r_1, \dots, r_j). \quad (40)$$

Let $(v')^{(j)}$ be the vector with $(b_1, \dots, b_j)$ 'th entry given by

$$\chi_{(b_1, \dots, b_j)}(r_1, \dots, r_{j-1}, c) = v^{(j-1)}[b_1, \dots, b_{j-1}] \cdot (b_j c + (1 - b_j)(1 - c)).$$

Note that $(v')^{(j)}$ can be computed in $O(2^j)$ time given $v^{(j-1)}$. And Expression (38) equals

$$\langle (v')^{(j)}, Q^{(j)} \rangle.$$

Similarly, let $w^{(j)}$ be the length- 2^j vector with entries indexed by $(b_1, \dots, b_j) \in \{0,1\}^j$, with $(b_1, \dots, b_j)$ 'th entry given by

$$\sum_{k=1}^{j-1} (r_k - i_k) d_k.$$

Let $(w')^{(j)}$ be the vector with $(b_1, \dots, b_j)$ 'th entry given by $w^{(j-1)}[b_1, \dots, b_j] + (c - b_j) \cdot d_j$. Then Expression (39) equals

$$\langle (v')^{(j)} \circ (w')^{(j)}, Z^{(j)} \rangle,$$

where $(v')^{(j)} \circ (w')^{(j)}$ denotes the Hadamard (i.e., entry-wise) product of $(v')^{(j)}$ and $(w')^{(j)}$.

In summary, we have shown that for $j = 1, \dots, m$, the sum-check prover's j 'th message s_j can be computed in time $O(2^j)$ so long as the prover can compute $v^{(j)}$ and $w^{(j)}$ in this time bound. And indeed this is the case. For $v^{(j)}$, observe that, given all entries of $v^{(j-1)}$, one can compute $v^{(j)}$ in time $O(2^j)$. This is because

$$v^{(j)}[b_1, \dots, b_j] = v^{(j-1)}[b_1, \dots, b_{j-1}] \cdot (r_j b_j + (1 - r_j)(1 - b_j)).$$

Similarly, for $w^{(j)}$, observe that, given all entries of $w^{(j-1)}$, one can compute $w^{(j)}$ in time $O(2^j)$. This is because

$$w^{(j)}[b_1, \dots, b_j] = w^{(j-1)}[b_1, \dots, b_{j-1}] + (r_j - b_j) \cdot d_j.$$

Rounds $\log(m) + 1, \dots, 2\log(m)$. Because the prover's runtime in round j takes time $O(2^j)$, by the time we reach round $\log m$, the prover is requiring $O(m)$ time per round. Hence, before the prover proceeds to round $m + 1$, the prover needs to perform a “condensation” operation so that round $\log(m) + 1$ behaves like round 1 in terms of prover complexity.

Round $j = \log(m) + 1$ of the sparse-dense sum-check protocol is equivalent to round 1 of the sparse-dense sum-check protocol with the $(\log(N))$ -variate polynomials $\tilde{u}$ and $\tilde{t}$ replaced by the following $(\log(N) - \log(m))$ -variate polynomials $\tilde{u}'$ and $\tilde{t}'$:

$$\begin{aligned}\tilde{u}'(b_{\log(m)+1}, \dots, b_{\log N}) &:= \tilde{u}(r_1, \dots, r_{\log m}, b_{\log(m)+1}, \dots, b_{\log N}), \\ \tilde{t}'(b_{\log(m)+1}, \dots, b_{\log N}) &:= \tilde{t}(r_1, \dots, r_{\log m}, b_{\log(m)+1}, \dots, b_{\log N}).\end{aligned}$$

So we merely need to show that in round $m + 1$ of the sparse-dense sum-check protocol, the prover can in $O(m)$ time compute the necessary data structures about $\tilde{u}'$ and $\tilde{t}'$, namely (per Equations (29) and (30)) the following quantities:

$$q'_k := \sum_{y \in \text{extend}_{\log(N) - \log(m)}(k)} \tilde{u}'(y) \cdot \tilde{t}'(y) \quad (41)$$

$$z'_k := \sum_{y \in \text{extend}_{\log(N) - \log(m)}(k)} \tilde{u}'(y). \quad (42)$$

All m of the z'_k can be computed in $O(m)$ time, as $z'_k = z_k \cdot \chi_k(r_1, \dots, r_{\log m})$ and the values

$$\{\chi_k(r_1, \dots, r_{\log m}) : k \in \{0, 1\}^{\log m}\} \quad (43)$$

can all be computed in $O(m)$ total time, and in fact are precisely the contents of the vector $v^{(\log m)}$ (see Equation (40)) computed by the prover anyway during the course of the first $\log m$ rounds of sum-check.

All m of the q'_k values can also be computed in $O(m)$ total time. To see this, recall that there are at most m values $y = (y_1, \dots, y_{\log N}) \in \{0, 1\}^{\log N}$ such that $\tilde{u}(y) \neq 0$. For each such y , let $y = (y', y'') \in \{0, 1\}^{\log m} \times \{0, 1\}^{\log(N) - \log(m)}$. Then $\tilde{u}'(y') = \sum_{k \in \{0, 1\}^{\log m}} \chi_k(r_1, \dots, r_{\log m}) \cdot \tilde{u}(k, y')$. Since the $\chi_k(r_1, \dots, r_{\log m})$ values (Equation (43)) can all be computed in $O(m)$ total time, this means that all non-zero $\tilde{u}'(y')$ values can in turn all be computed in $O(m)$ time. Let S be the set

$$\{y' \in \{0, 1\}^{\log(N) - \log(m)} : \tilde{u}'(y') \neq 0\},$$

and recall that S has size at most m . For all $y' \in S$, we can also compute $\tilde{t}'(y')$ in $O(m)$ total time, by the following reasoning.

First, recall from Equation (28) that for round $j = \log m$ and $(b_1, \dots, b_{\log N}) \in \{0, 1\}^{\log N}$,

$$\tilde{t}_\ell(r_1, \dots, r_j, b_{j+1}, \dots, b_{\log N}) = \tilde{t}_\ell(b_1, \dots, b_{\log N}) + \sum_{k=1}^j (r_k - b_k) \cdot d_k$$

Using dynamic programming, the following values can be computed in total time $O(2^j) = O(m)$ for all $(b_1, \dots, b_j) \in \{0, 1\}^j$:

$$\sum_{k=1}^j (r_k - b_k) \cdot d_k. \quad (44)$$

Indeed, let $H^{(j)}$ be the length 2^j -array with entries indexed by $(b_1, \dots, b_j) \in \{0, 1\}^j$ and such that $H^{(j)}[b_1, \dots, b_j] = \sum_{k=1}^j (r_k - b_k) \cdot d_k$. Then $H^{(j+1)}$ can be computed from $H^{(j)}$ in $O(2^{j+1})$ time, since $H^{(j)}[b_1, \dots, b_j, b_{j+1}] = H^{(j)}[b_1, \dots, b_j] + (r_{j+1} - b_{j+1}) \cdot d_{j+1}$. This means $H^{(\log m)}$ can be computed in $O(\sum_{j=1}^{\log m} 2^j) = O(m)$ time in total.

The prover, having computed and stored $\tilde{t}_\ell(y)$ for all $y \in S$ in $O(m)$ total time at the start of the protocol (see Footnote [21](#)), can use these values to compute

$$\tilde{t}(r_1, \dots, r_j, b_{j+1}, \dots, b_{\log N})$$

for all $y \in S$ in $O(m)$ total time.

Remaining rounds. The prover's computation in the remaining rounds $(2 \log(m) + 1, \dots, \log N)$ proceeds analogously to rounds $\log m, \dots, 2 \log m$. Every $\log m$ rounds, the prover perform a “condensation” operation so that subsequent round behaves like round 1 in terms of prover complexity. This entails computing quantities $q'_{k,\ell}$ and z'_k for each $k \in \{0, 1\}^{\log m}$, defined analogously to Equations [\(41\)](#) and [\(42\)](#). As above, these m quantities can all be computed in time $O(m)$ per condensation operation. In total, the prover performs $O(C)$ condensation operations, so $O(Cm)$ time is spent on condensation operations. Outside of the condensation operations, the prover implements each “chunk” of $\log m$ rounds in $O(m)$ time. Since there are $O(C)$ chunks, this means that the total prover time is $O(Cm)$. □

G.5.2 An $O(cm)$ -time prover for tables with $\tilde{t}$ having total degree larger than one

Theorem 9. Suppose that $\tilde{t}$ can be decomposed into a sum of $\eta = O(1)$ polynomials $\tilde{t}_1, \dots, \tilde{t}_\eta$, i.e.,

$$\tilde{t} = \sum_{\ell=1}^{\eta} \tilde{t}_\ell \quad (45)$$

and such that the following holds. For any $(r_1, \dots, r_{j-1}, c) \in \mathbb{F}^j$ and any $(b_j, \dots, b_{\log N}) \in \{0, 1\}^{\log(N)-j+1}$, there exist values $\mathbf{a}_\ell(c, j, b_j)$, $\mathbf{m}_\ell(c, j, b_j)$, each of which can be evaluated in $O(1)$ time, and which do not depend on the variables $b_{j+1}, \dots, b_{\log N}$, such that:

$$\tilde{t}_\ell(r_1, \dots, r_{j-1}, c, b_{j+1}, \dots, b_{\log N}) = \mathbf{m}_\ell(c, j, b_j) \cdot \tilde{t}_\ell(r_1, \dots, r_{j-1}, b_j, b_{j+1}, \dots, b_{\log N}) + \mathbf{a}_\ell(c, j, b_j). \quad (46)$$

Moreover, assume that for each $y \in \{0, 1\}^{\log N}$ such that $\tilde{t}(y) \neq 0$, and each $\ell = 1, \dots, \kappa$, it holds that $\tilde{t}_\ell(y)$ can be computed by the prover in $O(1)$ time [22](#). Then the prover in the sparse-dense sum-check protocol can be implemented in $O(m)$ field operations.

The theorem continues to hold if the values $\mathbf{a}_\ell$ and $\mathbf{m}_\ell$ depend on $(r_1, \dots, r_{j-1})$ in addition to c, j , and b_j . It also holds if $\mathbf{a}_\ell$ and $\mathbf{m}_\ell$ depend on $b_{j+1}, \dots, b_{j+\gamma}$ for some $\gamma = O(1)$ (in addition to c, j, b_j and $(r_1, \dots, r_{j-1})$).

The last sentence of Theorem [9](#) is needed to capture the two final example tables in Section [F.3.1](#) namely those capturing bitwise AND and Less-Than (LT) operations (for both of these examples, it suffices to take $\gamma = 1$). For example, recall from Equation [\(22\)](#) that for the table t capturing bitwise AND evaluations on b -bit inputs, the following holds:

$$\tilde{t}(x, y) = \sum_{i=1}^b 2^{i-1} \cdot x_i \cdot y_i.$$

Unfortunately, Theorem [6](#) does not apply to $\tilde{t}$, which has total degree 2.

Let us order the variables of (x, y) so that x_1 comes first and y_1 comes second, followed x_2 in third and y_2 in fourth, and so on. Then for even values of $j = 2k$, changing the value of the j 'th variable from y_k to r_j leads to an additive update to $\tilde{t}_1(x, y)$ of

$$2^{2b+k-1} \cdot r_{j-1} \cdot (r_j - y_k),$$

²²As with Footnote [21](#) if $\tilde{t}(y)$ itself cannot be computed in $O(1)$ time for all $y \in \{0, 1\}^{\log N}$ then the correct answer $\langle u, t \rangle$ cannot necessarily be computed by the prover in $O(m)$ time.

which depends only on $j = 2k$, r_{j-1}, r_j and y_k . However, if $j = 2k - 1$ is odd, then the additive “effect” on $\tilde{t}_1$ when changing the value of the j th variable from x_j to r_j is

$$2^{2b+k-1} \cdot (r_j - x_k) \cdot y_k.$$

This depends on variable $j + 1$ (i.e., on y_k).

Conceptually, this means that the value of y_k cannot be “ignored” by the sum-check prover algorithm during round $j = 2k - 1$, which is the round in which variable x_k is “processed”. However, the proof of Theorem 9 shows that this does not substantially affect prover time, essentially because there are only two possible values of $y_{k+1} \in \{0, 1\}$ that the algorithm needs to contemplate.

Proof of Theorem 9. We will prove the theorem assuming Equation (46) holds, and then explain what modifications are necessary if $\mathbf{a}_\ell(c, j, b_j)$ and $\mathbf{m}_\ell(c, j, b_j)$ have additional dependencies as per the last two sentences of the theorem statement.

A consequence of Equation (46) is that for any $j \in \{1, \dots, \log N\}$, and any $(b_1, \dots, b_{\log N}) \in \{0, 1\}^{\log N}$,

$$\tilde{t}_\ell(r_1, \dots, r_j, b_{j+1}, \dots, b_{\log N}) = \left(\prod_{k=1}^j \mathbf{m}_\ell(r_k, k, b_k) \right) \cdot \tilde{t}_\ell(b_1, \dots, b_{\log N}) + \sum_{k=1}^j \mathbf{a}_\ell(r_k, k, b_k). \quad (47)$$

Values computed by the prover at the start of the protocol. At the start of the protocol, for each polynomial $\tilde{t}_\ell$ in the decomposition of $\tilde{t}$ of Equation (45), the prover computes the exact same $q_{k,\ell}$ and z_k values as in Section G.5.1 and builds binary trees Q_ℓ and Z over them exactly as in Section G.5.1. That is, for each $\ell = 1, \dots, \kappa$,

$$q_{k,\ell} := \sum_{y \in \text{extend}_{\log N}(k)} \tilde{u}(y) \cdot \tilde{t}_\ell(y) \quad (48)$$

and z_k is defined exactly as in Definition (30). Q_ℓ is a binary tree over the $q_{k,\ell}$ values, where each internal node stores the sum of its two children, and Z is a binary tree over the z_k values.

The prover’s workflow in the first $\log m$ rounds. Following the derivation in Section G.5.1, we calculate the following convenient expression for the prover’s first message polynomial s_1 :

$$\begin{aligned} s_1(c) &= \sum_{y \in \{0,1\}^{\log N}} \tilde{u}(c, y_2, \dots, y_{\log N}) \cdot \tilde{t}(c, y_2, \dots, y_{\log N}) \\ &= \sum_{y \in \{0,1\}^{\log N}} \left(\sum_{i=(i_1, \dots, i_{\log N}) \in S_u} u_i \cdot \chi_i(c, y_2, \dots, y_{\log N}) \right) \cdot \tilde{t}(c, y_2, \dots, y_{\log N}) \end{aligned} \quad (49)$$

$$\begin{aligned} &= \sum_{i=(i_1, \dots, i_{\log N}) \in S_u} u_i \cdot \chi_i(c, i_2, \dots, i_{\log N}) \cdot \tilde{t}(c, i_2, \dots, i_{\log N}) \\ &= \sum_{i=(i_1, \dots, i_{\log N}) \in S_u} \chi_{i_1}(c) \cdot u_i \cdot \chi_i(i_1, i_2, \dots, i_{\log N}) \cdot \left(\sum_{\ell=1}^{\eta} (\mathbf{m}_\ell(c, 1, i_1) \cdot \tilde{t}_\ell(i_1, i_2, \dots, i_{\log N}) + \mathbf{a}_\ell(c, 1, i_1)) \right) \end{aligned} \quad (50)$$

Here, Equation (49) invokes Equation (26) and Equation (50) holds by Equations (45) and (46).

Round 1 computation. Expression (50) above equals:

$$\begin{aligned}
& \sum_{k=(k_1, \dots, k_{\log m}) \in \{0,1\}^{\log m} : k_1=0} \sum_{\ell=1}^{\kappa} (\chi_0(c) \cdot \mathbf{m}_\ell(c, 1, 0) \cdot q_{k,\ell} + \chi_0(c) \cdot \mathbf{a}_\ell(c, 1, 0) \cdot z_k) \\
& + \sum_{k=(k_1, \dots, k_{\log m}) \in \{0,1\}^{\log m} : k_1=1} \sum_{\ell=1}^{\kappa} (\chi_1(c) \cdot \mathbf{m}_\ell(c, 1, 1) \cdot q_{k,\ell} + \chi_1(c) \cdot \mathbf{a}_\ell(c, 1, 1) \cdot z_k)
\end{aligned} \tag{51}$$

For each $c \in \{-1, 0, 1\}$, computing this expression requires just a constant amount of work given the values of the two children of the root vertex of the trees $Q_1, \dots, Q_\kappa$ (Equations (31) and (32)) and $Z_1, \dots, Z_\kappa$.

Round $j > 1$ computation. A similar calculation to the above reveals that $s_j(c)$ equals:

$$\begin{aligned}
& \sum_{(y_{j+1}, \dots, y_{\log N}) \in \{0,1\}^{\log(N)-j}} \tilde{u}(r_1, \dots, r_{j-1}, c, y_{j+1}, \dots, y_{\log N}) \cdot \tilde{t}(r_1, \dots, r_{j-1}, c, y_{j+1}, \dots, y_{\log N}) \\
& = \sum_{(y_{j+1}, \dots, y_{\log N}) \in \{0,1\}^{\log(N)-j}} \left(\sum_{i=(i_1, \dots, i_{\log N}) \in S_u} u_i \cdot \chi_i(r_1, \dots, r_{j-1}, c, y_{j+1}, \dots, y_{\log N}) \right) \cdot \tilde{t}(r_1, \dots, r_{j-1}, c, y_{j+1}, \dots, y_{\log N}) \\
& = \sum_{i=(i_1, \dots, i_{\log N}) \in S_u} u_i \cdot \chi_i(r_1, \dots, r_{j-1}, c, i_{j+1}, \dots, i_{\log N}) \cdot \tilde{t}(r_1, \dots, r_{j-1}, c, i_{j+1}, \dots, i_{\log N}) \\
& = \sum_{i=(i_1, \dots, i_{\log N}) \in S_u} \chi_{(i_1, \dots, i_j)}(r_1, \dots, r_{j-1}, c) \cdot u_i \cdot \left(\sum_{\ell=1}^{\kappa} \left(\prod_{k=1}^{j-1} \mathbf{m}_\ell(r_k, k, b_k) \right) \tilde{t}_\ell(i_1, i_2, \dots, i_{\log N}) + \sum_{k=1}^{j-1} \mathbf{a}_\ell(r_k, k, b_k) \right) \\
& = \sum_{(b_1, \dots, b_j) \in \{0,1\}^j} \sum_{i=(b_1, \dots, b_j, i_{j+1}, \dots, i_{\log N}) \in S_u} u_i \cdot \chi_{(b_1, \dots, b_j)}(r_1, \dots, r_{j-1}, c) \cdot \sum_{\ell=1}^{\kappa} \left(\left(\prod_{k=1}^{j-1} \mathbf{m}_\ell(r_k, k, b_k) \right) \tilde{t}_\ell(i_1, i_2, \dots, i_{\log N}) + \sum_{k=1}^{j-1} \mathbf{a}_\ell(r_k, k, b_k) \right) \\
& = \sum_{(b_1, \dots, b_j) \in \{0,1\}^j} \sum_{i=(b_1, \dots, b_j, i_{j+1}, \dots, i_{\log N}) \in S_u} u_i \cdot \chi_{(b_1, \dots, b_j)}(r_1, \dots, r_{j-1}, c) \cdot \left(\sum_{\ell=1}^{\kappa} \left(\prod_{k=1}^{j-1} \mathbf{m}_\ell(r_k, k, b_k) \right) \tilde{t}_\ell(i_1, i_2, \dots, i_{\log N}) \right)
\end{aligned} \tag{52}$$

$$+ \sum_{(b_1, \dots, b_j) \in \{0,1\}^j} \sum_{i=(b_1, \dots, b_j, i_{j+1}, \dots, i_{\log N}) \in S_u} u_i \cdot \chi_{(b_1, \dots, b_j)}(r_1, \dots, r_{j-1}, c) \cdot \left(\sum_{\ell=1}^{\kappa} \sum_{k=1}^{j-1} \mathbf{a}_\ell(r_k, k, b_k) \right). \tag{53}$$

Recall that for each $\ell = 1, \dots, \kappa$, $Q_\ell^{(j)} \in \mathbb{F}^{2^j}$ (respectively $Z^{(j)}$) is the vector of values assigned to nodes at depth j of Q_ℓ . As in Section G.5.1 let $v^{(j)}$ be the length- 2^j vector with entries indexed by $(b_1, \dots, b_j) \in \{0,1\}^j$, with $(b_1, \dots, b_j)$ 'th entry given by

$$\chi_{(b_1, \dots, b_j)}(r_1, \dots, r_j).$$

Let $(v')^{(j)}$ be the vector with $(b_1, \dots, b_j)$ 'th entry given by

$$\chi_{(b_1, \dots, b_j)}(r_1, \dots, r_{j-1}, c) = v^{(j-1, \ell)}[b_1, \dots, b_{j-1}] \cdot (b_j c + (1 - b_j)(1 - c)).$$

Note that $(v')^{(j)}$ can be computed in $O(2^j)$ time given $v^{(j-1)}$. For each $\ell = 1, \dots, \kappa$, let $x^{(j, \ell)}$ denote the vector with entries indexed by $b = (b_1, \dots, b_j) \in \{0,1\}^j$ whose b 'th entry is $\prod_{k=1}^j \mathbf{m}_\ell(r_k, k, b_k)$, and let $(x')^{(j, \ell)}$ equal

$$x^{(j-1, \ell)}[b_1, \dots, b_{j-1}] \cdot \mathbf{m}_\ell(c, j, b_j).$$

Then Expression (52) equals

$$\sum_{\ell=1}^{\kappa} \langle (v')^{(j)} \circ (x')^{(j, \ell)}, Q_\ell^{(j)} \rangle,$$

where $(v')^{(j)} \circ (x')^{(j, \ell)}$ denotes the Hadamard (i.e., entry-wise) product of $(v')^{(j)}$ and $(x')^{(j, \ell)}$.

Similarly, let $w^{(j)}$ be the length- 2^j vector with entries indexed by $(b_1, \dots, b_j) \in \{0, 1\}^j$, with $(b_1, \dots, b_j)$ 'th entry given by

$$\sum_{\ell=1}^{\kappa} \sum_{k=1}^{j-1} a_{\ell}(r_k, k, b_k)$$

Let $(w')^{(j)}$ be the vector with $(b_1, \dots, b_j)$ 'th entry given by $w^{(j-1)}[b_1, \dots, b_j] + \sum_{\ell=1}^{\kappa} a_{\ell}(c, j, b_j)$. Then Expression (53) equals

$$\langle (v')^{(j)} \circ (w')^{(j)}, Z^{(j)} \rangle.$$

In summary, we have shown that for $j = 1, \dots, m$, the sum-check prover's j 'th message s_j can be computed in time $O(\kappa \cdot 2^j)$ so long as the prover can compute $v^{(j)}$, $w^{(j)}$, and $x^{(j, \ell)}$ for $\ell = 1, \dots, \kappa$ in this time bound. And indeed this is the case. This was explained for $v^{(j)}$ in Section G.5.1. For $x^{(j, \ell)}$, observe that, given all entries of $x^{(j-1, \ell)}$, one can compute $x^{(j, \ell)}$ in time $O(2^j)$. This is because

$$x^{(j, \ell)}[b_1, \dots, b_j] = v^{(j-1)}[b_1, \dots, b_{j-1}] \cdot m_{\ell}(r_j, j, b_j),$$

and we have assumed that $m_{\ell}(r_j, j, b_j)$ can be computed in constant time. The case of $w^{(j)}$ is similar.

Rounds $\log(m) + 1, \dots, 2 \log(m)$. As in Section G.5.1, round $j = \log(m) + 1$ of the sparse-dense sum-check protocol is equivalent to round 1 of the sparse-dense sum-check protocol with the $(\log(N))$ -variate polynomials $\tilde{u}$ and $\tilde{t}$ replaced by the following $(\log(N) - \log(m))$ -variate polynomials $\tilde{u}'$ and $\tilde{t}'$:

$$\tilde{u}'(b_{\log(m)+1}, \dots, b_{\log N}) := \tilde{u}(r_1, \dots, r_{\log m}, b_{\log(m)+1}, \dots, b_{\log N}),$$

$$\tilde{t}'(b_{\log(m)+1}, \dots, b_{\log N}) := \tilde{t}(r_1, \dots, r_{\log m}, b_{\log(m)+1}, \dots, b_{\log N}).$$

For $\ell = 1, \dots, \kappa$, let

$$\tilde{t}_{\ell}(b_{\log(m)+1}, \dots, b_{\log N}) = \tilde{t}'_{\ell}(r_1, \dots, r_{\log m}, b_{\log(m)+1}, \dots, b_{\log N})$$

So we merely need to show that in round $m + 1$ of the sparse-dense sum-check protocol, the prover can in $O(m)$ time compute the necessary data structures about $\tilde{u}'$ and $\tilde{t}'_1, \dots, \tilde{t}'_{\ell}$, namely (per Equations (48) and (30)) the following quantities:

$$(q')_{k, \ell} := \sum_{y \in \text{extend}_{\log(N) - \log(m)}(k)} \tilde{u}'(y) \cdot \tilde{t}'_{\ell}(y) \quad (54)$$

$$z'_k := \sum_{y \in \text{extend}_{\log(N) - \log(m)}(k)} \tilde{u}'(y). \quad (55)$$

All m of the z'_k can be computed in $O(m)$ time, as $z'_k = z_k \cdot \chi_k(r_1, \dots, r_{\log m})$ exactly as per Section G.5.1. All $m \cdot \kappa$ of the $q'_{k, \ell}$ values can also be computed in $O(m)$ total time. To see this, recall that there are at most m values $y = (y_1, \dots, y_{\log N}) \in \{0, 1\}^{\log N}$ such that $\tilde{u}(y) \neq 0$. For each such y , let $y = (y', y'') \in \{0, 1\}^{\log m} \times \{0, 1\}^{\log(N) - \log(m)}$. Then $\tilde{u}'(y') = \sum_{k \in \{0, 1\}^{\log m}} \chi_k(r_1, \dots, r_{\log m}) \cdot \tilde{u}(k, y')$. Since the $\chi_k(r_1, \dots, r_{\log m})$ values (Equation (43)) can all be computed in $O(m)$ total time, this means that all non-zero $\tilde{u}'(y')$ values can in turn all be computed in $O(m)$ time. Let S be the set

$$\{y' \in \{0, 1\}^{\log(N) - \log(m)} : \tilde{u}'(y') \neq 0\},$$

and recall that S has size at most m . For all $y' \in S$, we can also compute $\tilde{t}'_{\ell}(y')$ and $\ell = 1, \dots, \kappa$ in $O(\kappa \cdot m)$ total time, by the following reasoning. First, recall from Equation (47) that for round $j = \log m$ and $(b_1, \dots, b_{\log N}) \in \{0, 1\}^{\log N}$,

$$\tilde{t}_\ell(r_1, \dots, r_j, b_{j+1}, \dots, b_{\log N}) = \left(\prod_{k=1}^j m_\ell(r_k, k, b_k) \right) \cdot \tilde{t}_\ell(b_1, \dots, b_{\log N}) + \sum_{k=1}^j a_\ell(r_k, k, b_k).$$

Using dynamic programming, the following values can be computed in total time $O(2^j) = O(m)$ for all $(b_1, \dots, b_j) \in \{0, 1\}^j$:

$$\prod_{k=1}^j m_\ell(r_k, k, b_k) \quad (56)$$

and

$$\sum_{k=1}^j a_\ell(r_k, k, b_k). \quad (57)$$

The prover, having computed and stored $\tilde{t}_\ell(y)$ for all $y \in S$ in $O(m)$ total time at the start of the protocol (see Footnote [22](#)), can use these values to compute

$$\tilde{t}_\ell(r_1, \dots, r_j, b_{j+1}, \dots, b_{\log N})$$

for all $y \in S$ in $O(m)$ total time.

Remaining rounds. The prover's computation in the remaining rounds $(2 \log(m) + 1, \dots, \log N)$ proceeds analogously to rounds $\log m, \dots, 2 \log m$. Every $\log m$ rounds, the prover perform a “condensation” operation so that subsequent round behaves like round 1 in terms of prover complexity. This entails computing quantities $q'_{k,\ell}$ and z'_k for each $k \in \{0, 1\}^{\log m}$. The total prover runtime is $O(Cm)$ as in Section [G.5.1](#).

Modifications if $a_\ell(c, j, b_j)$ and $m_\ell(c, j, b_j)$ have additional dependencies. If $a_\ell(c, j, b_j)$ and $m_\ell(c, j, b_j)$ depend on $(r_1, \dots, r_{j-1})$, in addition to c, j , and b_j , nothing about the prover's computation needs to change because in each round j , there is only one vector $(r_1, \dots, r_{j-1})$ for the algorithm to consider.

If a_ℓ and m_ℓ also depend on $b_{j+1}, \dots, b_{j+\gamma}$ for some $\gamma = O(1)$ (in addition to c, j, b_j and $(r_1, \dots, r_{j-1})$), then some modifications are required. Conceptually, in each round j , the prover computation groups those $y \in \{0, 1\}^{\log N}$ such that $\tilde{u}(y) \neq 0$ so that elements of the same group all have a_ℓ and m_ℓ equal to the same quantities. These groups correspond to the internal nodes at level j of the binary trees Q_ℓ and Z . If a_ℓ and m_ℓ depend on $b_{j+1}, \dots, b_{j+\gamma}$, then the grouping needs to incorporate $b_{j+1}, \dots, b_{j+\gamma}$ as well. Fortunately, the number of groups under consideration in each round j grows by at most a factor of 2^γ because $(b_{j+1}, \dots, b_{j+\gamma}) \in \{0, 1\}^\gamma$ can only take 2^γ values.

Specifically, Equations [\(51\)](#) (capturing the prover's round-one message) is updated to have $2^{1+\gamma}$ sums rather than 2 sums. Associating each sum with a bit-vector in $\{0, 1\}^\gamma$, the i 'th sum is over terms $k \in \{0, 1\}^{\log m}$ that have with $(k_1, \dots, k_\gamma) = i$ (Equation [\(51\)](#) itself corresponds to the case $\gamma = 0$). Explicitly, Equation [\(51\)](#) becomes:

$$\sum_{i=(i_1, \dots, i_\gamma) \in \{0, 1\}^\gamma} \sum_{k=(k_1, \dots, k_{\log m}) \in \{0, 1\}^{\log m} : (k_1, \dots, k_\gamma) = i} \sum_{\ell=1}^{\kappa} (\chi_i(c, k_2, \dots, k_\gamma) \cdot m_\ell \cdot q_{k,\ell} + \chi_i(c, k_2, \dots, k_\gamma) \cdot a_\ell \cdot z_k),$$

where the quantities a_ℓ and m_ℓ may depend on $c, k_1, \dots, k_\gamma$.

Similarly [\(52\)](#), and [\(53\)](#) are updated to become:

$$\begin{aligned} & \sum_{(b_1, \dots, b_j) \in \{0, 1\}^j} \sum_{i=(b_1, \dots, b_j, i_{j+1}, \dots, i_{\log N}) \in S_u} u_i \cdot \chi_{(b_1, \dots, b_{j+\gamma})}(r_1, \dots, r_{j-1}, c, b_{j+1}, \dots, b_{j+\gamma}) \left(\sum_{\ell=1}^{\kappa} \left(\prod_{k=1}^{j-1} m_\ell(r_k, k, b_k) \right) \tilde{t}_\ell(i_1, i_2, \dots, i_{\log N}) \right) \\ & + \sum_{(b_1, \dots, b_{j+\gamma}) \in \{0, 1\}^{j+\gamma}} \sum_{i=(b_1, \dots, b_{j+\gamma}, i_{j+\gamma+1}, \dots, i_{\log N}) \in S_u} u_i \cdot \chi_{(b_1, \dots, b_{j+\gamma})}(r_1, \dots, r_{j-1}, c, b_{j+1}, \dots, b_{j+\gamma}) \left(\sum_{\ell=1}^{\kappa} \sum_{k=1}^{j-1} a_\ell(r_k, k, b_k) \right). \end{aligned}$$

Here, we write $\mathbf{a}_\ell(r_k, k, b_k)$ and $\mathbf{m}_\ell(r_k, k, b_k)$ for simplicity and consistency with Equations (52) and (53), but these quantities may in general depend on $k, r_1, \dots, r_k$ and $b_{k+1}, \dots, b_{k+\gamma}$. $\square$

Applications to the AND and LT tables. The discussion following the statement of Theorem 9 explained that the theorem applied to the lookup table for the AND instruction, whose (x, y) 'th entry is $\sum_{i=1}^b 2^{i-1} x_i \cdot y_i$.

Recall from Section F.3.1 that LT denotes the function that takes two b -bit inputs x and y as input and outputs 1 if $x < y$ and 0 otherwise. The appropriate lookup table to capture this function has (x, y) 'th entry equal to $\text{LT}(x, y)$. Let $\widetilde{\text{LT}}$ denote the multilinear extension of the function $\text{LT}: \{0, 1\}^b \times \{0, 1\}^b \rightarrow \mathbb{F}$.

Deriving an expression for $\widetilde{\text{LT}}$. Let $x = (x_1, \dots, x_b)$ and $y = (y_1, \dots, y_b)$.

For $i = 2, \dots, b$, define $x_{>i} = (x_{i+1}, \dots, x_b)$, and define

$$\widetilde{\text{LT}}_i(x, y) = (1 - x_i) \cdot y_i \cdot \widetilde{\text{eq}}(x_{>i}, y_{>i}),$$

where recall that $\widetilde{\text{eq}}$ was defined in Equation (1).

We claim that

$$\widetilde{\text{LT}}(x, y) = \sum_{i=1}^b \widetilde{\text{LT}}_i(x, y). \quad (58)$$

Indeed, the right hand side of this equation is multilinear and agrees with $\text{LT}(x, y)$ at all inputs $x, y \in \{0, 1\}^b$. This is because, if j is the highest-order bit that differs between x and y , then $\widetilde{\text{LT}}_{j'}(x, y) = 0$ for all $j' \neq j$, and $\widetilde{\text{LT}}_j(x, y) = 1$ if and only if $x_j = 0$ and $y_j = 1$.

Showing that $\widetilde{\text{LT}}(x, y)$ satisfies the requirement of Theorem 9 so long as $m \geq \log N$. Let us order the $2b$ variables of (x, y) (i.e., the variables of the polynomial $\widetilde{\text{LT}}$) so that the low-order bits x_1 and y_1 get bound in the first two rounds of sparse-dense sum-check, x_2 and y_2 get bound in the next two rounds, and so on. For any $(r_1, \dots, r_{j-1}, r_j) \in \mathbb{F}^j$ and any $(z_j, \dots, z_{2b}) \in \{0, 1\}^{2b-j+1}$, we must show that there exist values $\mathbf{a}_\ell, \mathbf{m}_\ell$, each of which can be evaluated in $O(1)$ time, and which do *not* depend on the variables $z_{j+2}, \dots, z_{2b}$, such that:

$$\widetilde{\text{LT}}(r_1, \dots, r_{j-1}, r_j, z_{j+1}, \dots, z_{2b}) = \mathbf{m} \cdot \widetilde{\text{LT}}(r_1, \dots, r_{j-1}, z_j, z_{j+1}, \dots, z_{2b}) + \mathbf{a}. \quad (59)$$

Say that $j = 2k$ is even and let us write

$$x = (r_1, r_3, r_5, \dots, r_{j-1}, z_{j+1}, \dots, z_{2b-1})$$

and

$$y' = (r_2, r_4, r_6, \dots, r_j, z_{j+2}, \dots, z_{2b}).$$

Let

$$y = (r_2, r_4, r_6, \dots, r_{j-1}, z_j, z_{j+2}, \dots, z_{2b}).$$

Let k^* be the highest-order bit such that $x_{k^*} \neq y_{k^*}$.

- If $k < k^*$, then $\widetilde{\text{LT}}(x, y) - \widetilde{\text{LT}}(x, y') = 0$. This holds by the following reasoning. For all $i \leq k$, $\widetilde{\text{LT}}_i(x, y) = \widetilde{\text{LT}}_i(x, y') = 0$ due to the factor $\widetilde{\text{eq}}(x_{>i}, y_{>i})$ appearing in $\widetilde{\text{LT}}_i$ and the fact that $x_{k^*} \neq y_{k^*}$. And for $i > k$, $\widetilde{\text{LT}}_i(x, y) = \widetilde{\text{LT}}_i(x, y')$ because y and y' differ only in the k 'th coordinate, and for each $i > k$, $\widetilde{\text{LT}}_i$ does not depend on inputs $1, \dots, i-1$.

- Suppose $k \geq k^*$. Then:

$$\begin{aligned}
\widetilde{\text{LT}}(x, y') &= \sum_{i=1}^b \widetilde{\text{LT}}_i(x, y') = \sum_{i=1}^k (1 - r_{2i-1}) \cdot r_{2i} \cdot \widetilde{\text{eq}}(x_{>i}, y'_{>i}) + \sum_{i=k+1}^b \text{LT}_i(x, y') \\
&= \sum_{i=1}^k \left((1 - r_{2i-1}) \cdot r_{2i} \cdot \prod_{j=i+1}^b (x_i y'_i + (1 - x_i)(1 - y'_i)) \right) + \sum_{j=k+1}^b \text{LT}_j(x, y') \\
&= \sum_{i=1}^k \left((1 - r_{2i-1}) \cdot r_{2i} \cdot \prod_{j=i+1}^b (x_i y'_i + (1 - x_i)(1 - y'_i)) \right) \quad (60)
\end{aligned}$$

Here, Equation (60) holds because $x_i = y_i = y'_i$ for all $i > k$ by assumption that k^* is the most significant index at which x and y differ.

- Suppose $k > k^*$. Then (assuming each $r_j \notin \{0, 1\}$) Equation (60) equals

$$\text{LT}(x, y) \cdot \frac{r_{j-1}z_j + (1 - r_{j-1})(1 - z_j)}{r_{j-1}r_j + (1 - r_{j-1})(1 - r_j)}.$$

- Suppose $k = k^*$. Then Equation (60) equals

$$\begin{aligned}
&\sum_{i=1}^{k-1} \left((1 - r_{2i-1}) \cdot r_{2i} \cdot \prod_{j=i+1}^k (x_i y'_i + (1 - x_i)(1 - y'_i)) \right) + (1 - r_{2k-1}) \cdot r_{2k} \\
&= (1 - r_{j-1}) \cdot r_j + \sum_{i=1}^{k-1} \left((1 - r_{2i-1}) \cdot r_{2i} \cdot \left(\prod_{j=i+1}^{k-1} (x_i y'_i + (1 - x_i)(1 - y'_i)) \right) \right) \cdot (r_{j-1}r_j + (1 - r_{j-1})(1 - r_j)).
\end{aligned}$$

Here, we have used that $x_i = y_i$ and $x_i, y_i \in \{0, 1\}$ for all $i > k^*$. Meanwhile, $\text{LT}(x, y)$ equals

$$(1 - r_{j-1}) \cdot z_j + \sum_{i=1}^{k-1} \left((1 - r_{2i-1}) \cdot r_{2i} \cdot \left(\prod_{j=i+1}^{k-1} (x_i y'_i + (1 - x_i)(1 - y'_i)) \right) \right) \cdot (r_{j-1}z_j + (1 - r_{j-1})(1 - z_j)).$$

Let

$$\beta = \frac{r_{j-1}r_j + (1 - r_{j-1})(1 - r_j)}{r_{j-1}z_j + (1 - r_{j-1})(1 - z_j)}. \quad (61)$$

Hence,

$$\text{LT}(x, y') = \text{LT}(x, y) \cdot \beta + (1 - r_{2k-1}) \cdot r_{2k} - \beta(1 - r_{j-1}) \cdot z_j.$$

Summarizing, if we are in round $j = 2k$ with $k < k^*$ where k^* is the most significant where x and y differ, then $\mathbf{a} = 0$ and $\mathbf{m} = 1$. If $k > k^*$ then $\mathbf{a} = 0$ and $\mathbf{m} = \beta$.

$$\mathbf{m} = \frac{r_{j-1}r_j + (1 - r_{j-1})(1 - r_j)}{r_{j-1}z_j + (1 - r_{j-1})(1 - z_j)}.$$

If $k = k^*$ then $\mathbf{m} = \beta$ (defined in Equation (61)) and

$$\mathbf{a} = (1 - r_{2k-1}) \cdot r_{2k} - \beta(1 - r_{j-1}) \cdot z_j.$$

²³Note that while multiplicative inverses take super-constant time to compute, the denominator of the fraction only involves r_{j-1} , and z_j , and hence only takes two different values, as r_{j-1} is fixed by the verifier before starting round j and z_j is only ever 0 or 1. That is, the prover does not have to compute a different inverse for each tuple (x, y) with $\tilde{u}(x, y) \neq 0$.

The case of $j = 2k - 1$ is similar and we highlight the main differences. In this case, let us write

$$x' = (r_1, r_3, r_5, \dots, r_j, z_{j+2}, \dots, z_{2b-1})$$

and

$$x = (r_1, r_3, r_5, \dots, z_j, z_{j+2}, \dots, z_{2b-1}),$$

and

$$y = (r_2, r_4, r_6, \dots, r_{j-1}, z_j, z_{j+2}, \dots, z_{2b}).$$

Then we have the following analog of Equation (60):

$$\begin{aligned} \widetilde{\text{LT}}(x', y) &= \sum_{i=1}^b \widetilde{\text{LT}}_i(x', y) = \sum_{i=1}^k (1 - r_{2i-1}) \cdot r_{2i} \cdot \widetilde{\text{eq}}(x_{>i}, y'_{>i}) + \sum_{i=k+1}^b \text{LT}_i(x', y) \\ &= \sum_{i=1}^k \left((1 - r_{2i-1}) \cdot r_{2i} \cdot \prod_{j=i+1}^b (x'_i y_i + (1 - x'_i)(1 - y_i)) \right) + \sum_{j=k+1}^b \text{LT}_j(x', y) \\ &= \sum_{i=1}^k \left((1 - r_{2i-1}) \cdot r_{2i} \cdot \prod_{j=i+1}^b (x'_i y_i + (1 - x'_i)(1 - y_i)) \right) \end{aligned} \quad (62)$$

The main difference when $j = 2k - 1$ compared to the analysis for $j = 2k$ lies is in the case that $k = k^*$. In this case, Equation (62) equals

$$\begin{aligned} &\sum_{i=1}^{k-1} \left((1 - r_{2i-1}) \cdot r_{2i} \cdot \prod_{j=i+1}^k (x'_i y_i + (1 - x'_i)(1 - y_i)) \right) + (1 - r_{2k-1}) \cdot y_k \\ &= (1 - r_j) \cdot y_k + \sum_{i=1}^{k-1} \left((1 - r_{2i-1}) \cdot r_{2i} \cdot \left(\prod_{j=i+1}^{k-1} (x'_i y_i + (1 - x'_i)(1 - y_i)) \right) \right) \cdot (r_j y_k + (1 - r_j)(1 - y_k)) \end{aligned}$$

Meanwhile, $\text{LT}(x, y)$ equals

$$(1 - x_k) \cdot y_k + \sum_{i=1}^{k-1} \left((1 - r_{2i-1}) \cdot r_{2i} \cdot \left(\prod_{j=i+1}^{k-1} (x'_i y_i + (1 - x'_i)(1 - y_i)) \right) \right) \cdot (x_k y_k + (1 - x_k)(1 - y_k)) = -0,$$

where we have used the fact that $k = k^*$ and $x_{k^*} \neq y_{k^*}$ by assumption.

Hence,

$$\text{LT}(x', y) = \text{LT}(x, y) + (1 - r_j) \cdot y_j - (1 - x_j) \cdot y_j + \eta$$

where η equals

$$\begin{aligned} &= \sum_{i=1}^{k-1} \left((1 - r_{2i-1}) \cdot r_{2i} \cdot \left(\prod_{j=i+1}^{k-1} (x'_i y_i + (1 - x'_i)(1 - y_i)) \right) \right) \cdot (r_j y_k + (1 - r_j)(1 - y_k)) \\ &\quad \sum_{i=1}^{k-1} \left((1 - r_{2i-1}) \cdot r_{2i} \cdot \left(\prod_{j=i+1}^{k-1} (r_{2i-1} r_{2i} + (1 - r_{2i-1})(1 - r_{2i})) \right) \right) \cdot (r_j y_k + (1 - r_j)(1 - y_k)). \end{aligned} \quad (63)$$

In this case, we can set $\mathbf{a} = (1 - r_j) \cdot y_j - (1 - x_j) \cdot y_j + \eta$ and $\mathbf{m} = 1$. The prover can devote $O(\log N)$ total time over the *entire course* of the sum-check protocol to ensure that η can always be computed in $O(1)$ time.

That is, at all odd rounds $j = 2k - 1$ of the sparse-dense sum-check protocol it will update the quantity

$$\sum_{i=1}^{k-1} \left((1 - r_{2i-1}) \cdot r_{2i} \cdot \left(\prod_{j=i+1}^{k-1} (r_{2j-1}r_{2j} + (1 - r_{2j-1})(1 - r_{2j})) \right) \right).$$

Note that when moving from round j to round $j + 2$ this quantity can be updated in $O(1)$ time. Moreover, given this quantity, for either of the two possible values of $y_k \in \{0, 1\}$, η can be computed in $O(1)$ time.